

PEMROGRAMAN BERORIENTASI OBJECT

Konsep Dasar OOP

Rules



Don't be late



Don't call (WA Only)

Monday-Friday(07.00 - 18.00 WIB)



Don't cheat

Discount 50%

Outline

- PBO vs Struktural
- Konsep dasar PBO

Matakuliah PBO

- Pemrograman Berbasis Objek (2 SKS/4x50 menit)
- Praktikum Pemrograman Berbasis Objek (2 SKS/4x50 menit)
- Capaian Pembelajaran
 - ▣ Mampu membuat program dengan menggunakan prinsip-prinsip OOP menggunakan bahasa pemrograman Java

Silabus

Pertemuan Ke-	Pembahasan
1	Pengantar Konsep Dasar OOP
2	Class dan Object
3	Enkapsulasi
4	Relasi Class
5	Kuis 1
6	Inheritance
7	Overriding dan Overloading
8	Ujian Tengah Semester (UTS)
9	Abstract Class
10	Interface
11	Polimorfisme
12	Kuis 2
13-16	OOP pada Bahasa pemrograman lain
17	Ujian Akhir Semester (UAS)

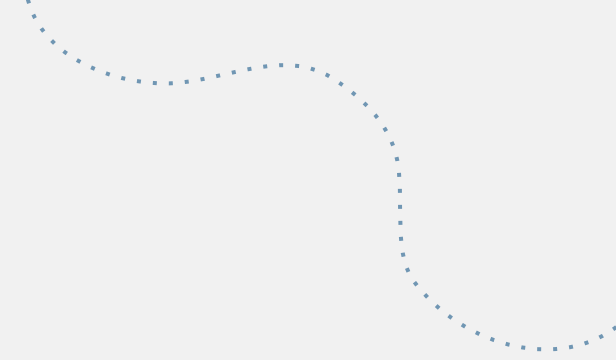


**Pemrograman
Berorientasi Objek**

Vs

**Pemrograman
Struktural**

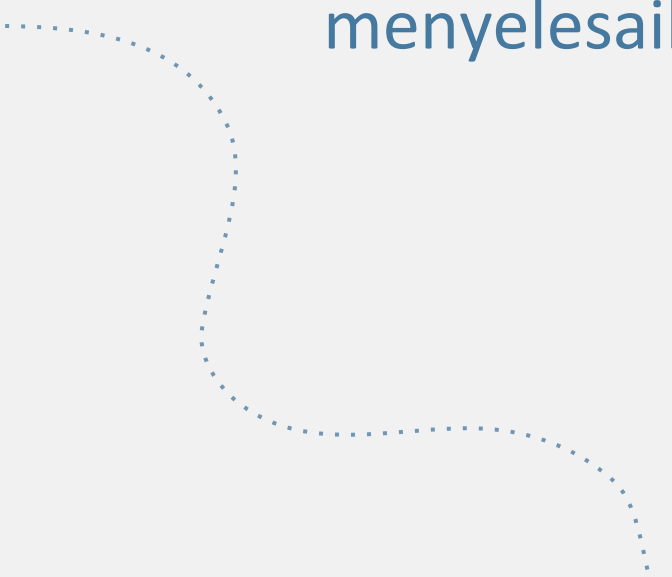




— PEMROGRAMAN

STRUKTURAL

Paradigma pemrograman dengan konsep bahwa program merupakan urutan langkah-langkah yang dilakukan untuk menyelesaikan suatu masalah



Pemrograman Struktural

- Contoh:
- Kita akan membuat program game simulasi sepeda, didalamnya ada objek sepeda yang memiliki kecepatan, gear dan merk.
- Bagaimana membangun game tersebut dengan metode **struktural**?
 - ▣ Langkah pertama kita buat variabelnya, misal kecepatan, gear, merk
 - ▣ Langkah berikutnya kita buat fungsi-fungsinya, tambah kecepatan, kurangi kecepatan.
 - ▣ Langkah berikutnya kita coba mengoperasikan sepeda tersebut secara sederhana, yaitu memanipulasi kecepatan, gear, merk nya, didalam fungsi main, kemudian kita cetak ke layar.
- Kode program →...


```
public static void main(String[] args)
{
    String merek;
    int kecepatan, gear;

    merek = "Poligone";
    kecepatan = 10;
    gear = 1;

    kecepatan = tambahKecepatan(kecepatan, 10);

    System.out.println("Merek: " + merek);
    System.out.println("Kecepatan: " + kecepatan);
}

public static int tambahKecepatan(int kecepatan, int increment)
{
    kecepatan += increment;

    return kecepatan;
}

public static int kurangiKecepatan(int kecepatan, int decrement)
{
    kecepatan -= decrement;

    return kecepatan;
}
```

Pemrograman Struktural

- Bagaimana jika ada dua sepeda di game?
 - ▣ Tambahkan variabel merek2, kecepatan2, gear2
 - ▣ Coba manipulasi nilai-nilai variabelnya kemudian tampilkan ke layar
- Kode program → ...

```
public static void main(String[] args)
{
    String merek, merek2;
    int kecepatan, kecepatan2, gear, gear2;

    merek = "Poligone";
    kecepatan = 10;
    gear = 1;

    merek2 = "Wiim Cycle";
    kecepatan2 = 15;
    gear2 = 3;

    kecepatan = tambahKecepatan(kecepatan, 10);
    kecepatan2 = tambahKecepatan(kecepatan2, 5);

    System.out.println("Merek: " + merek);
    System.out.println("Kecepatan: " + kecepatan);

    System.out.println("Merek: " + merek2);
    System.out.println("Kecepatan: " + kecepatan2);
}

public static int tambahKecepatan(int kecepatan, int increment)
{
    kecepatan += increment;

    return kecepatan;
}

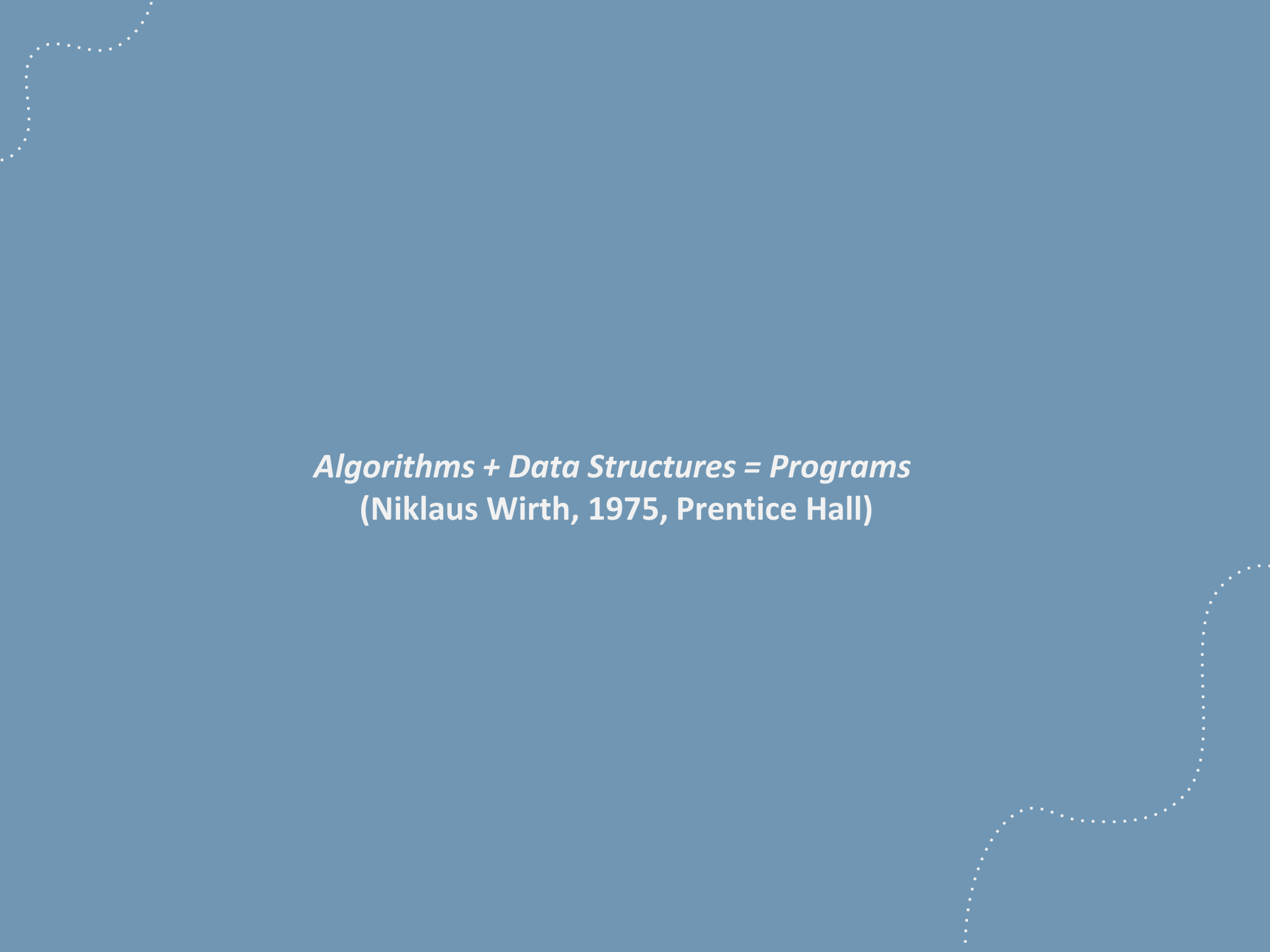
public static int kurangiKecepatan(int kecepatan, int decrement)
{
    kecepatan -= decrement;

    return kecepatan;
}
```

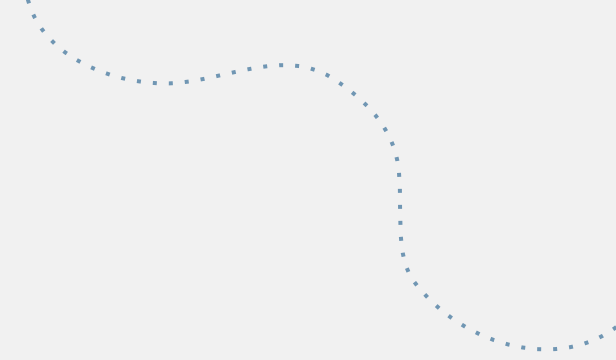
12
baris

Objek Oriented vs Struktural

- Bagaimana jika ada **sepuluh** sepeda?
 - ▣ Tambahkan variabel merek3, kecepatan3, gear3
..... merek10, kecepatan10, gear10



Algorithms + Data Structures = Programs
(Niklaus Wirth, 1975, Prentice Hall)



— PEMROGRAMAN **BERORIENTASI OBJEK**

Paradigma pemrograman yang berfokus pada objek

Data + Algorithm

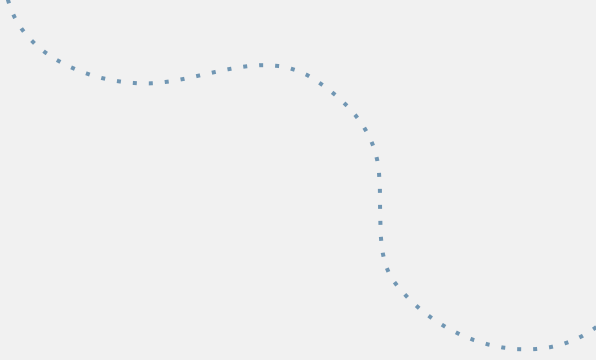


Konsep OOP

- Beberapa aspek dalam OOP:
 - ▣ Object
 - ▣ Class
 - ▣ Enkapsulasi
 - ▣ Inheritance
 - ▣ Polimorfisme

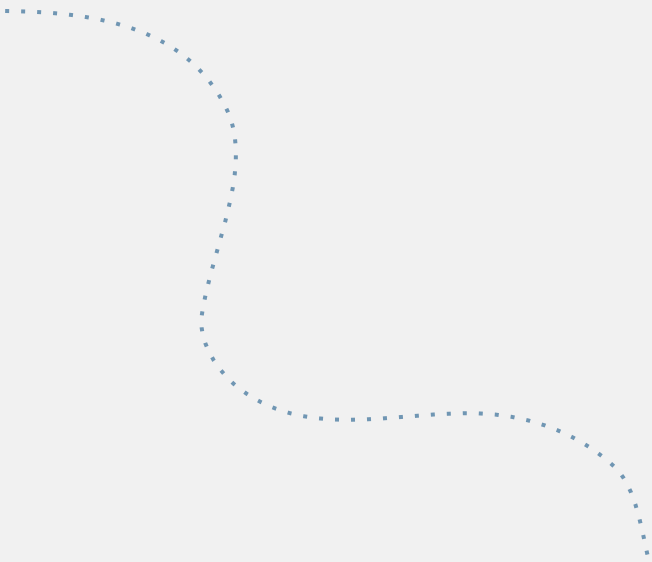
The background features two decorative dotted lines in a light blue color. One line starts at the top left, curves upwards and then downwards towards the right. The other line starts at the top right, curves upwards and then downwards towards the left. There are two light blue plus signs: one in the top right area and one in the bottom left area.

CLASS & OBJECT



— OBJEK

Siapa dan apa saja yang terlibat dalam business process?



SISTEM INFORMASI AKADEMIK

Aljabar Linear



AKUNTANSI



Kalkulus 2



Analisis Numerik
Matematika Diskrit



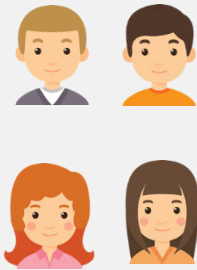
TEKNIK SIPIL
TEKNIK KIMIA



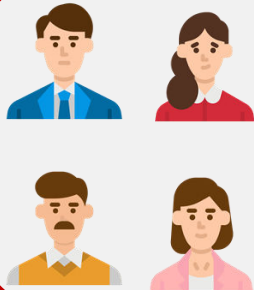
AKUNTANSI



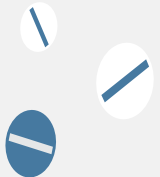
SISTEM INFORMASI **AKADEMIK**



Analisis Numerik
Matematika Diskrit
Aljabar Linear
Kalkulus 2

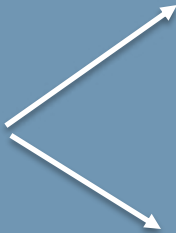


AKUNTANSI
TEKNIK INFORMATIKA
TEKNIK SIPIL
TEKNIK KIMIA





KARAKTERISTIK



Apa yang dimiliki?

Apa yang bisa dilakukan?



Apa yang dimiliki sebuah objek mahasiswa?



- Nama
- NIM
- Tanggal Lahir
- Jenis Kelamin
- Alamat



—**ATRIBUT:**

Variabel/properti/ciri/status
/sifat yang **dimiliki** oleh
suatu objek



	productID	productName	categoryName	supplier
▶	1	Chai	Beverages	Specialty Biscuits, Ltd.
	2	Chang	Beverages	Exotic Liquids
	3	Aniseed Syrup	Condiments	Exotic Liquids
	4	Chef Anton's Cajun Seasoning	Condiments	New Orleans Cajun Delights
	5	Chef Anton's Gumbo Mix	Condiments	New Orleans Cajun Delights
	6	Grandma's Boysenberry Spread	Condiments	Grandma Kelly's Homestead
	7	Uncle Bob's Organic Dried Pears	Produce	Grandma Kelly's Homestead
	8	Northwoods Cranberry Sauce	Condiments	Grandma Kelly's Homestead
	9	Mishi Kobe Niku	Meat/Poultry	Tokyo Traders
	10	Ikura	Seafood	Tokyo Traders

Apa yang bisa dilakukan oleh/terhadap objek mahasiswa?

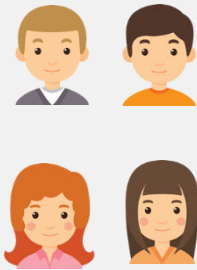


- Memilih mata kuliah
- Melihat nilai
- Mengajukan cuti akademik

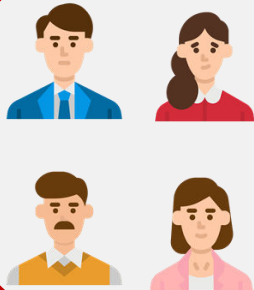
—METHOD:

Prosedur/fungsi/perilaku/proses yang bisa **dilakukan** oleh/terhadap suatu objek

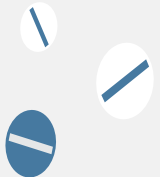
SISTEM INFORMASI AKADEMIK

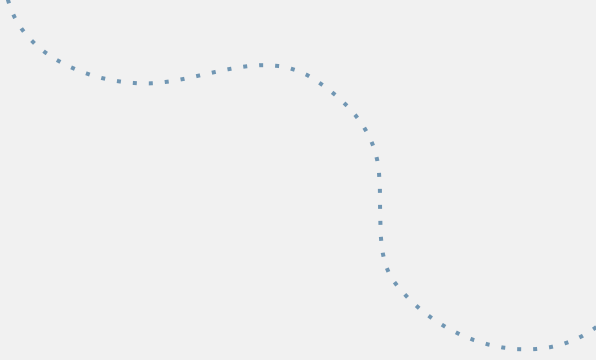


Analisis Numerik
Matematika Diskrit
Aljabar Linear
Kalkulus 2



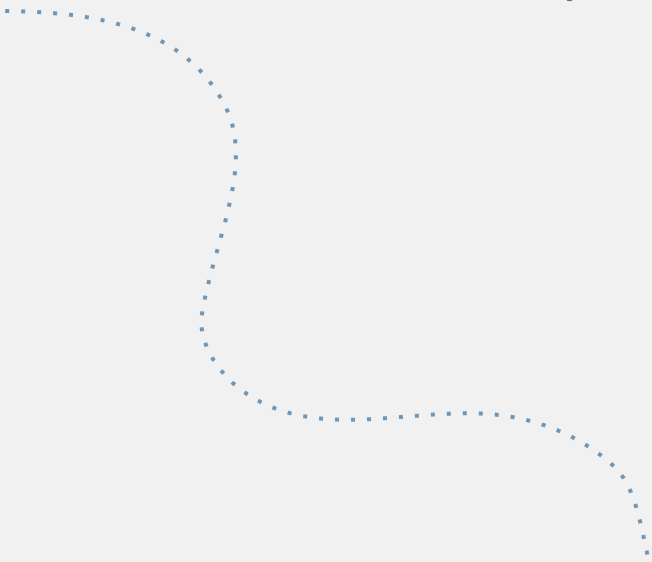
AKUNTANSI
TEKNIK INFORMATIKA
TEKNIK SIPIL
MATEMATIKA





— CLASS

Blueprint/template/cetakan yang mendefinisikan karakteristik (atribut dan method) objek pada class



— OBJECT

Objek adalah representasi dari setiap entitas yang terlibat dalam sistem (baik yang nyata maupun tidak nyata)

Setiap objek akan memiliki sekumpulan variabel/ciri/status dengan nilai yang melekat padanya

Class Donat



6 Objek pada Class Donat



Class Rumah



3 Objek pada Class Rumah



```
public class Sepeda {  
    public String merk;  
    public int kecepatan, gear;  
  
    public Sepeda(String m, int k, int g){  
        merk = m;  
        kecepatan = k;  
        gear = g;  
    }  
  
    public int tambahKecepatan(int increment){  
        kecepatan += increment;  
        return kecepatan;  
    }  
  
    public int kurangKecepatan(int decrement){  
        kecepatan -= decrement;  
        return kecepatan;  
    }  
  
    public void info(){  
        System.out.println("Merk: " + merk);  
        System.out.println("Kecepatan: " + kecepatan);  
    }  
  
    public static void main(String[] args) {  
        // sepeda pertama  
        Sepeda spd1 = new Sepeda("Poligon", 10, 1);  
  
        spd1.tambahKecepatan(10);  
        spd1.info();  
  
        //sepeda kedua  
        Sepeda spd2 = new Sepeda("Wim Cycle", 15, 3);  
  
        spd2.tambahKecepatan(5);  
        spd2.info();  
    }  
}
```

6
baris

Kesimpulan

□ Struktural

- ▣ Program dipecah ke dalam fungsi/prosedur
- ▣ Perubahan fitur → kemungkinan mengganggu keseluruhan program

□ Object Oriented

- ▣ Program dipecah ke dalam object
 - Didalamnya terdapat state dan behavior
- ▣ Perubahan fitur → tidak mengganggu keseluruhan program

Latihan

- Carilah objek apa saja di dunia nyata sebanyak 5.
- Tuliskan state/atribut dan behavior/method objek tersebut. Makin banyak state dan behavior makin baik. Contoh:
- Televisi
 - ▣ State:
 - Merek
 - Ukuran layar
 - Channel
 - Volume
 - ▣ Behavior:
 - Nyalakan
 - Matikan
 - Pindah channel
 - Tambah volume
 - Kurangi volume

Referensi

- Horstmann, C. S., & Cornell, G. (2007). *Core Java Volume I—Fundamentals, Eighth Edition*. Network Circle, Santa Clara: Prentice Hall.
- Horstmann, C. S., & Cornell, G. (2008). *Core Java Volume II—Advanced Features, Eighth Edition*. Network Circle, Santa Clara: Prentice Hall.
- <https://www.javatpoint.com/java-oops-concepts>
- Dapat di download di <http://libgen.io/>

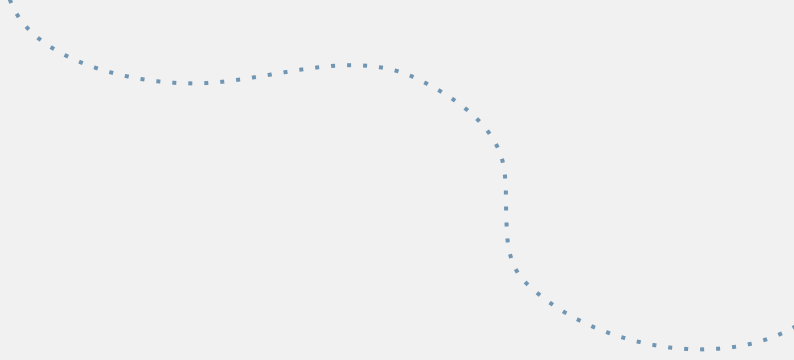


Class dan Object

PEMROGRAMAN BERORIENTASI OBJEK

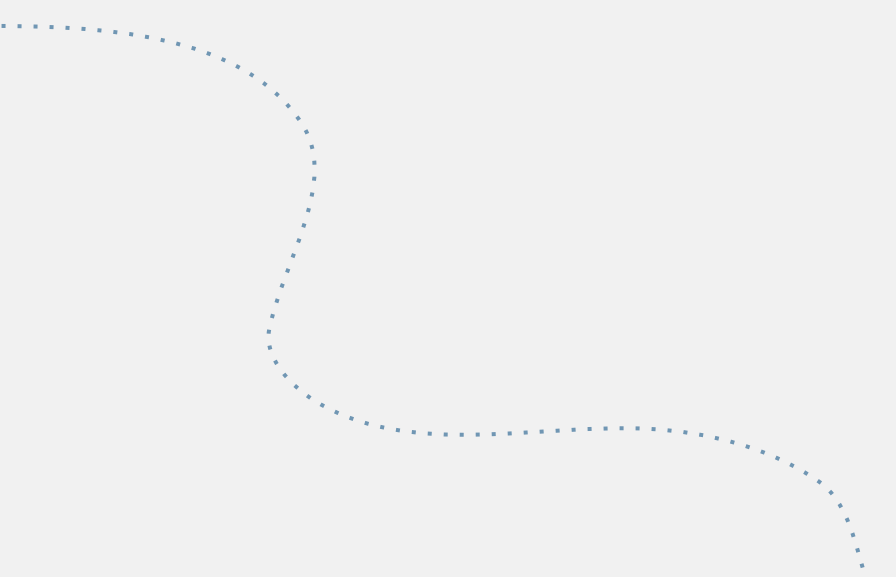
JURUSAN TEKNOLOGI INFORMASI

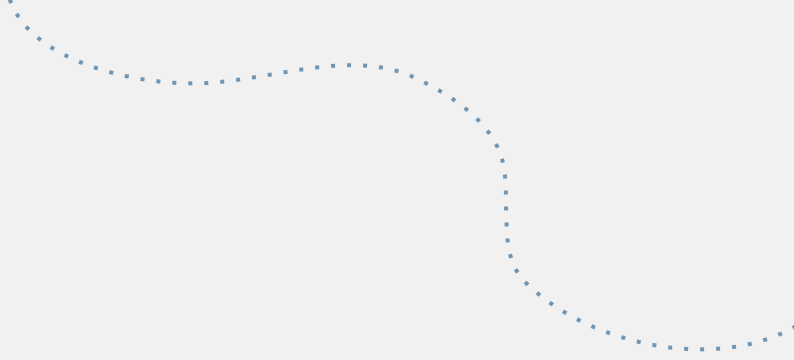
POLITEKNIK NEGERI MALANG



— OBJECT

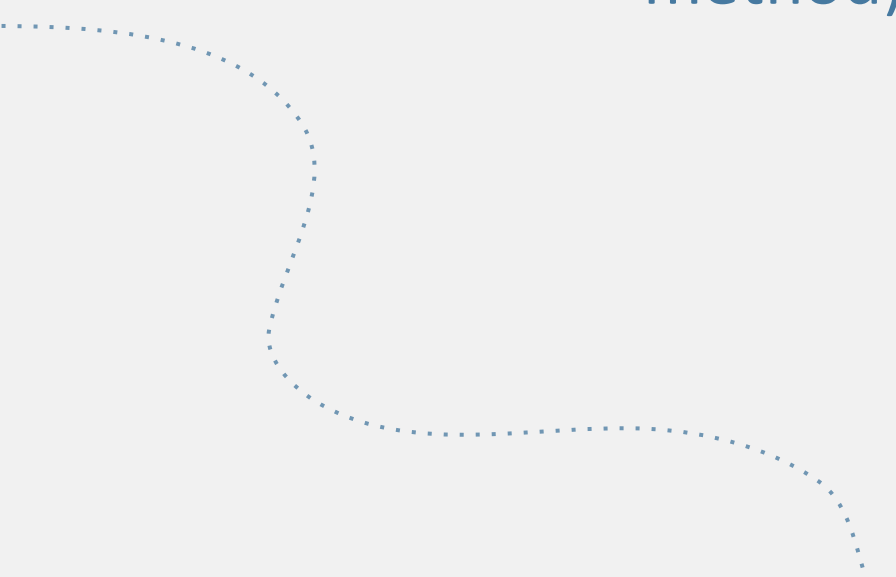
Objek adalah representasi dari setiap entitas yang terlibat dalam sistem





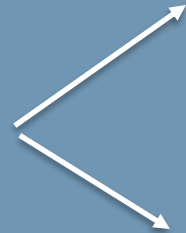
— CLASS

Blueprint/template/cetakan yang mendefinisikan karakteristik (atribut dan method) objek pada class



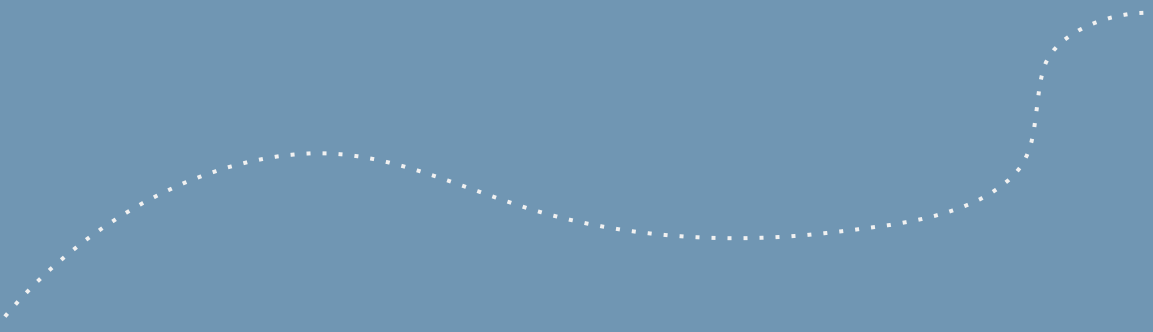


KARAKTERISTIK



Apa yang dimiliki?

Apa yang bisa dilakukan?



Apa yang dimiliki sebuah objek mahasiswa?



- Nama
- NIM
- Tanggal Lahir
- Jenis Kelamin
- Alamat



—**ATRIBUT:**

Variabel/ciri/status/sifat
yang **dimiliki** oleh suatu
objek

Apa yang bisa dilakukan oleh/terhadap objek mahasiswa?



- Memilih mata kuliah
- Melihat nilai
- Mengajukan cuti akademik

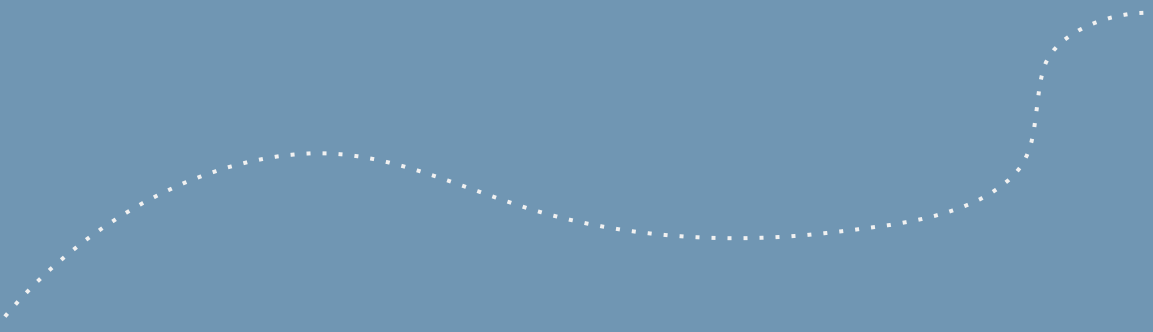


—METHOD:

Prosedur/fungsi/perilaku/
proses yang bisa **dilakukan**
oleh/terhadap suatu objek



Tidak diawali angka
Underscore?



Aturan penulisan nama class

- Berupa kata benda
- Diawali dengan **HURUF KAPITAL**,
- Jika terdiri dari lebih dari 1 kata, maka setiap kata **disambungkan**, dan huruf awal dari tiap kata menggunakan **HURUF KAPITAL**
- Contoh: Mahasiswa, TenagaKependidikan

Implementasi Class

```
<modifier> class <nama_class> {  
  
    //deklarasi atribut dan method  
  
}
```

- Untuk mendeklarasikan suatu class, digunakan keyword class lalu diikuti dengan nama class yang akan dibuat, e.g:

```
public class Mahasiswa{  
  
  
  
}
```

Aturan penulisan nama atribut

- Berupa **kata benda** atau **kata sifat**,
- Diawali dengan **HURUF KECIL**
- Jika terdiri dari lebih dari 1 kata, maka setiap kata **disambungkan**, dan huruf awal dari tiap kata menggunakan **HURUF BESAR**.

Deklarasi Atribut

- Untuk melakukan deklarasi atribut:

```
<modifier> <tipe> <nama_atribut> ;
```

- Exp :

```
public String nim;
```

```
public String nama;
```

```
public String alamat;
```

Aturan penulisan nama method

- Berupa **kata kerja**,
- Diawali dengan **HURUF KECIL**,
- Jika terdiri dari lebih dari 1 kata, maka setiap kata **disambungkan**, dan huruf awal dari tiap kata menggunakan **HURUF BESAR**.

Deklarasi Method

- Untuk melakukan pendeklarasian method dapat dilakukan dengan sintaks sebagai berikut :

```
<modifier> <return_type> <nama method>(param1, param2, ...)
{
    //statements
}
```

Return Type

- Method dengan **return type void**, berarti tidak memiliki return value, sehingga tidak memerlukan kata kunci **return** di dalamnya.
- Method dengan **return type selain void**, berarti memerlukan return value, sehingga harus ada nilai yang dikembalikan dengan kata kunci **return** di dalamnya

```
public void sayHello(){  
    System.out.println("Hello World!!");  
}
```

TIDAK PERLU
RETURN /
TIDAK ADA NILAI
KEMBALIAN

```
public int tambah (int a, int b){  
    int hasil = a+b;  
    return hasil;  
}
```

- TIPE DATA METHOD INT, BERARTI METHOD TSB HARUS MENGEMBALIKAN NILAI INT
- HARUS ADA RETURN

Constructor

- Constructor → method istimewa yang digunakan untuk melakukan instansiasi objek (membuat objek baru)
- Istimewa:
 1. Nama method sama dengan nama class
 2. Tidak memiliki return type
 3. Hanya bisa dijalankan/dipanggil pada proses instansiasi
- Jika sebuah class tidak memiliki constructor secara eksplisit, maka secara default Java compiler akan menyediakan constructor tanpa parameter

Instansiasi Object

- Object adalah instansiasi dari sebuah class
- Instansiasi dilakukan dengan memanggil constructor menggunakan keyword **new**

```
NamaClass namaObject = new NamaClass();
```

- Contoh:

```
Mahasiswa mahasiswa1 = new Mahasiswa();
```

```
Mahasiswa ani = new Mahasiswa();
```

Contoh

```
public class Mahasiswa{  
    public String nim;  
    public String nama;  
    public String alamat;  
  
    public void cetakBiodataMahasiswa(){  
        System.out.println("Biodata Mahasiswa");  
        System.out.println("Nim      : " + nim);  
        System.out.println("Nama      : " + nama);  
        System.out.println("Alamat      : " + alamat);  
    }  
}
```

Instansiasi Objek

```
public class DemoMahasiswa{  
    public static void main(String[] args){  
        Mahasiswa mhs = new Mahasiswa();  
        mhs.nim = "14324";  
        mhs.nama = "Very Sugiarto";  
        mhs.alamat = "Malang";  
        mhs.cetakBiodataMahasiswa();  
    }  
}
```

Constructor

```
public class Donat{  
    public String topping;  
}
```



```
public class Donat{  
    public String topping;  
  
    public Donat(){  
    }  
}
```

Instansiasi Objek

```
public class DemoDonat{
```

```
    public static void main(String[] args){
```

```
        Donat donat1 = new Donat();
```

```
        donat1.topping = "Strawberry sprinkles";
```

```
    }
```

```
}
```



Constructor berparameter

- Constructor berparameter digunakan untuk menginstansiasi objek baru dengan kondisi/nilai tertentu.
- Java compiler tidak akan menyediakan default constructor (tanpa parameter) jika constructor lain sudah dibuat oleh programmer

```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```

Instansiasi Objek

```
public class DemoDonat{  
    public static void main(String[] args){  
        Donat donat1 = new Donat("Strawberry sprinkles");  
    }  
}
```



```
public class Donat{  
    public String topping;  
}
```



```
public class Donat{  
    public String topping;  
  
    public Donat(){  
    }  
}
```

Jika sebuah class tidak memiliki constructor secara eksplisit, maka secara default Java compiler akan menyediakan constructor tanpa parameter

```
public class DemoDonat{  
    public static void main(String[] args){  
        Donat donat1 = new Donat();  
        donat1.topping = "Strawberry sprinkles";  
    }  
}
```


Java compiler tidak akan menyediakan default constructor (tanpa parameter) jika constructor lain sudah dibuat oleh programmer

```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```



```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```

Class Diagram

NamaClass

atribut1: tipeData1
atribut2: tipeData2

method1(parameter1:tipeData1): returnType1
method2(): returnType2

```

public class Sepeda {
    String merk;
    int kecepatan;

    public int tambahKecepatan(float increment) {
        kecepatan += increment;
        return kecepatan;
    }

    public int kurangiKecepatan(float decrement) {
        kecepatan -= decrement;
        return kecepatan;
    }

    public void cetakInfo() {
        System.out.println("Merk: " + merk);
        System.out.println("Kecepatan: " + kecepatan);
    }
}

```

Sepeda
merk: String kecepatan: int
tambahKecepatan(increment:int): int kurangiKecepatan(decrement:int): int cetakInfo(): void

```

public class Sepeda {
    String merk;
    int kecepatan;

    public int tambahKecepatan(float increment) {
        kecepatan += increment;
        return kecepatan;
    }

    public int kurangiKecepatan(float decrement) {
        kecepatan -= decrement;
        return kecepatan;
    }

    public void cetakInfo() {
        System.out.println("Merk: " + merk);
        System.out.println("Kecepatan: " + kecepatan);
    }
}

```

Sepeda
merk: String kecepatan: int
tambahKecepatan(): int kurangiKecepatan(): int cetakInfo(): void

Latihan

- Carilah objek apa saja di dunia nyata sebanyak 5.
- Tuliskan state/atribut dan behavior/method objek tersebut. Makin banyak state dan behavior makin baik. Contoh:
- Televisi
 - ▣ State:
 - Merek
 - Ukuran layar
 - Channel
 - Volume
 - ▣ Behavior:
 - Nyalakan
 - Matikan
 - Pindah channel
 - Tambah volume
 - Kurangi volume

Latihan

- Buatlah class diagram untuk setiap class yang dibuat pada Tugas 01
- Perhatikan best practice penamaan class, method, dan atribut (kata benda/sifat/kerja dan huruf kapital)

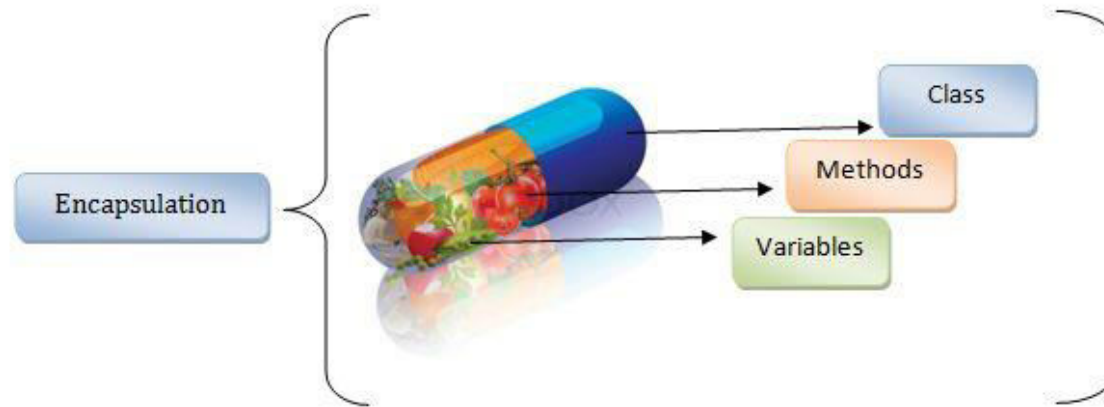
ENKAPSULASI

Minggu ke-3 PBO

Overview

- Enkapsulasi
 - ▣ Definisi
 - ▣ Tujuan
 - ▣ Mekanisme
 - ▣ Access Level Modifier
 - ▣ Setter & Getter
- Notasi pada UML Class Diagram
- Constructor

Enkapsulasi



Enkapsulasi

- Definisi:
 - ▣ Penyatuan/penggabungan atribut dan method dari suatu objek menjadi suatu kesatuan
 - ▣ Pembatasan akses langsung terhadap komponen dari suatu objek

Tujuan Enkapsulasi

- Penyembunyian struktur internal dari suatu objek → information hiding/data hiding
- Melindungi atribut dari perubahan di luar class secara random. Atribut dapat dibuat menjadi read-only atau write-only
- Mempermudah implementasi perubahan requirements
- Mempermudah pengujian unit sistem

Mekanisme Enkapsulasi

- Enkapsulasi bisa dilakukan dengan mengeset access level modifier menjadi private sehingga tidak dapat diakses secara langsung di luar class
- Menyediakan getter dan setter sebagai cara untuk mengakses atau memodifikasi private attribute

Access Level Modifier

- Terdapat 4 access level modifier yaitu:
 - ▣ public – dapat diakses dari mana saja
 - ▣ protected – dapat diakses di luar package asal merupakan subclass (inherit terhadap class nya)
 - ▣ no modifier (*package-private*) – hanya dapat diakses di dalam package yang sama
 - ▣ private – hanya dapat diakses di dalam kelas yang sama
- Class hanya memiliki 2 jenis access level modifier yaitu public dan no modifier

Akses Modifier

Modifier	Class	Package	Subclass	Outside Package
public	v	v	v	v
protected	v	v	v	
no modifier	v	v		
private	v			

Setter dan Getter

□ Getter

- ▣ Public method yang berfungsi mengembalikan nilai dari atribut private
- ▣ Ada return value

□ Setter

- ▣ Public method yang berfungsi untuk memanipulasi nilai dari atribut private
- ▣ Tidak ada return value

Tanpa Enkapsulasi

```
public class MobilDemo{  
    public String merek;  
    public int kecepatan;  
}
```

```
public class MotorDemo {  
    public static void main(String[] args){  
        Mobil mobil1 = new Mobil();  
        mobil1.merek = "Suzuki";  
        mobil1.kecepatan = 100;  
  
        System.out.println(mobil1.merek);  
        System.out.println(mobil1.kecepatan);  
    }  
}
```


Dengan Enkapsulasi

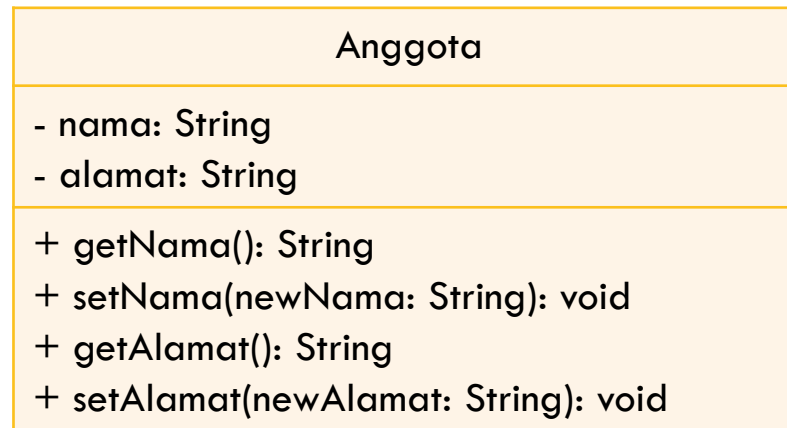
```
public class Motor {  
    private String merek;  
    private int kecepatan;  
  
    public void setMerek(String merek){  
        this.merek = merek;  
    }  
  
    public String getMerek(){  
        return merek;  
    }  
  
    public void setKecepatan(int kecepatan){  
        this.kecepatan = kecepatan;  
    }  
  
    public int getKecepatan(){  
        return kecepatan;  
    }  
}
```

```
public class MotorDemo {  
    public static void main(String[] args){  
        Mobil mobil1 = new Mobil();  
        mobil1.setMerek("Suzuki");  
        mobil1.setKecepatan(100);  
  
        System.out.println(mobil1.getMerek());  
        System.out.println(mobil1.getKecepatan());  
    }  
}
```

Read-Only & Write-Only Attribute

- Read-only attribute → hanya memiliki getter, tetapi tidak memiliki setter
- Write-only attribute → hanya memiliki setter, tetapi tidak memiliki getter

Notasi Pada UML Class Diagram



Notasi Pada UML Class Diagram

- Notasi access level modifier pada UML class diagram adalah sebagai berikut:
 - ▣ Tanda plus (+) untuk public
 - ▣ Tanda pagar (#) untuk protected
 - ▣ Tanda minus (-) untuk private
 - ▣ Untuk no-modifier tidak diberi notasi

Constructor

- Method yang digunakan untuk menginstansiasi objek dari suatu class
- Beberapa aturan deklarasi constructor:
 - ▣ Nama constructor harus sama dengan nama class
 - ▣ Constructor tidak memiliki return type

Constructor

Contoh

```
public class Anggota {  
    private String nama;  
    private String alamat;  
    private float simpanan;  
  
    //constructor  
    Anggota(String nama, String alamat){  
        this.nama = nama;  
        this.alamat = alamat;  
        this.simpanan = 0;  
    }  
}
```

Instansiasi Objek

```
public class KoperasiDemo {  
    public static void main(String[] args) {  
        Anggota anggota1 = new Anggota("Iwan","Jalan Mawar");  
  
        anggota1.setNama("Iwan Setiawan");  
        anggota1.setAlamat("Jalan Sukarno Hatta no 10");  
        anggota1.setor(100000);  
    }  
}
```

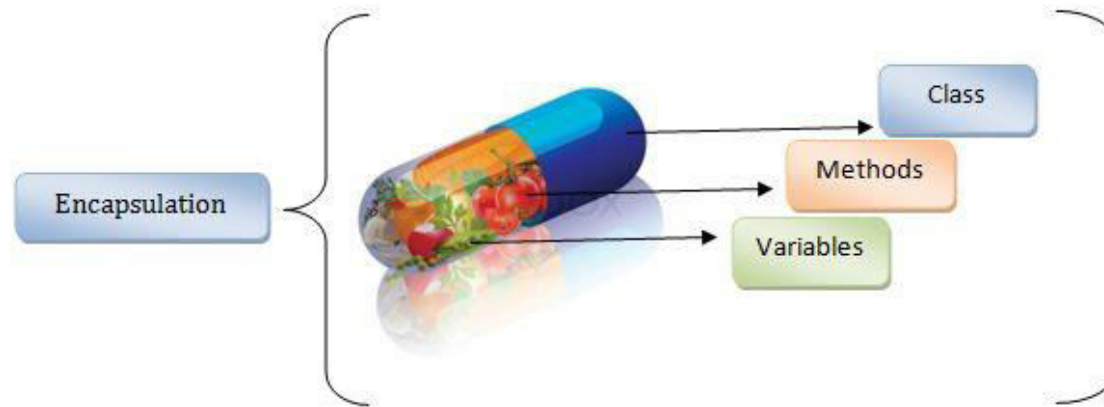
ENKAPSULASI

Minggu ke-3 PBO

Overview

- Enkapsulasi
 - ▣ Definisi
 - ▣ Tujuan
 - ▣ Mekanisme
 - ▣ Access Level Modifier
 - ▣ Setter & Getter
- Notasi pada UML Class Diagram

Enkapsulasi



Enkapsulasi

- Definisi:
 - ▣ Penyatuan/penggabungan atribut dan method dari suatu objek menjadi suatu kesatuan
 - ▣ Pembatasan akses langsung terhadap komponen dari suatu objek

Tujuan Enkapsulasi

- Penyembunyian struktur internal dari suatu objek → information hiding/data hiding
- Melindungi atribut dari perubahan di luar class secara random. Atribut dapat dibuat menjadi read-only atau write-only
- Mempermudah implementasi perubahan requirements
- Mempermudah pengujian unit sistem

Mekanisme Enkapsulasi

- Set access level modifier menjadi private sehingga tidak dapat diakses secara langsung di luar class
- Sediakan getter dan setter sebagai cara untuk mengakses atau memodifikasi private attribute

Access Level Modifier

- 4 access level modifier untuk **atribut dan method**:
 - public, protected, no modifier, private
- 2 access level modifier untuk **class**:
 - public dan no modifier

Access Level Modifier

- Terdapat 4 access level modifier yaitu:
 - ▣ public – dapat diakses dari mana saja
 - ▣ protected – dapat diakses di luar package asal merupakan subclass (inherit terhadap class nya)
 - ▣ no modifier (*package-private*) – hanya dapat diakses di dalam package yang sama
 - ▣ private – hanya dapat diakses di dalam kelas yang sama

Akses Modifier

Modifier	Class	Package	Subclass	Outside Package
public	v	v	v	v
protected	v	v	v	
no modifier	v	v		
private	v			

Setter dan Getter

□ Getter

- Public method yang berfungsi mengembalikan nilai dari atribut private
- Ada return value

□ Setter

- Public method yang berfungsi untuk memanipulasi nilai dari atribut private
- Tidak ada return value

Tanpa Enkapsulasi

```
public class Mobil{  
    public String merek;  
    public int kecepatan;  
}
```

```
public class MobilDemo {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil();  
        mobil1.merek = "Suzuki";  
        mobil1.kecepatan = 100;  
  
        System.out.println(mobil1.merek);  
        System.out.println(mobil1.kecepatan);  
    }  
}
```

Dengan Enkapsulasi

```
public class Mobil {  
    private String merek;  
    private int kecepatan;  
  
    public void setMerek(String merek) {  
        this.merek = merek;  
    }  
  
    public String getMerek() {  
        return merek;  
    }  
  
    public void setKecepatan(int kecepatan) {  
        this.kecepatan = kecepatan;  
    }  
  
    public int getKecepatan() {  
        return kecepatan;  
    }  
}
```

```
public class MobilDemo {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil();  
        mobil1.setMerek("Suzuki");  
        mobil1.setKecepatan(100);  
  
        System.out.println(mobil1.getMerek());  
        System.out.println(mobil1.getKecepatan());  
    }  
}
```

this keyword

- Keyword *this* merujuk pada current object
- Keyword ini biasanya digunakan untuk mengeliminasi kebingungan antara atribut dan parameter dengan nama yang sama

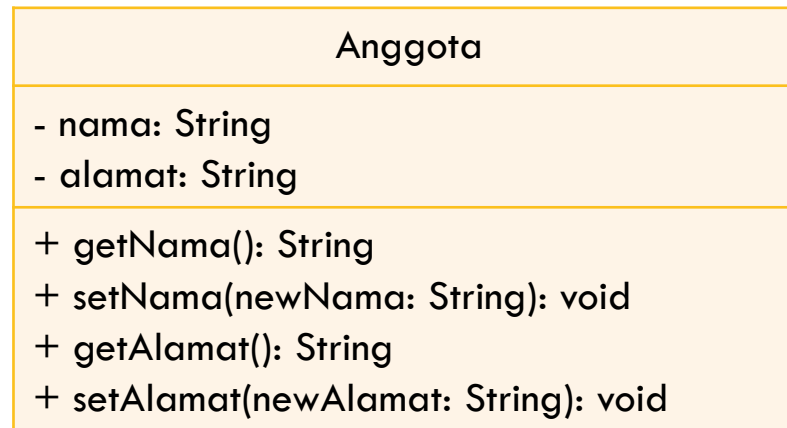
Read-Only & Write-Only Attribute

- Read-only attribute → hanya memiliki getter, tetapi tidak memiliki setter
- Write-only attribute → hanya memiliki setter, tetapi tidak memiliki getter

Notasi Pada UML Class Diagram

- Notasi access level modifier pada UML class diagram adalah sebagai berikut:
 - ▣ Tanda plus (+) untuk public
 - ▣ Tanda pagar (#) untuk protected
 - ▣ Tanda minus (-) untuk private
 - ▣ Untuk no-modifier tidak diberi notasi

Notasi Pada UML Class Diagram





CLASS RELATION (RELASI KELAS)

PEMROGRAMAN BERBASIS OBJEK – PERTEMUAN 4

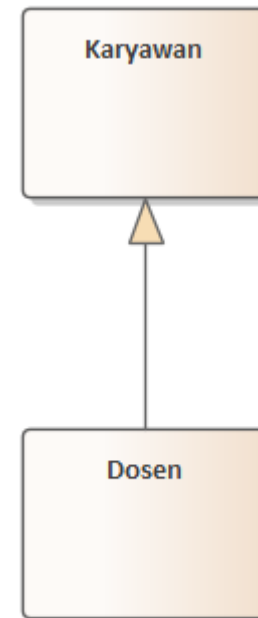


JENIS RELASI

- Ada 4 jenis relasi dasar antar class:
 1. Inheritance
 2. Association
 3. Aggregation
 4. Composition

INHERITANCE

- Menggambarkan relasi antara class yang lebih umum (parent/superclass) dan class yang lebih khusus (child/subclass)
- “is-a” relationship
- Contoh:
 - Dosen *adalah* seorang Karyawan
 - Kucing *adalah* seekor Binatang
 - Gedung *adalah* sebuah Bangunan
 - Mobil *merupakan* sebuah Kendaraan

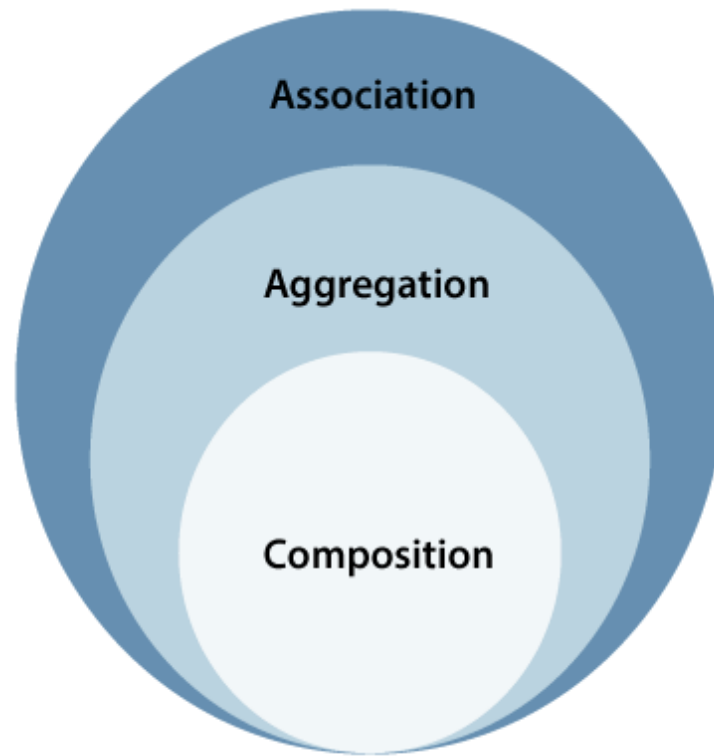


ASSOCIATION

- Asosiasi merupakan relasi antar objek yang saling berkaitan dalam suatu sistem
- “has-a” relationship
- Contoh:
 - Dosen dan MataKuliah
 - Mahasiswa dan MataKuliah

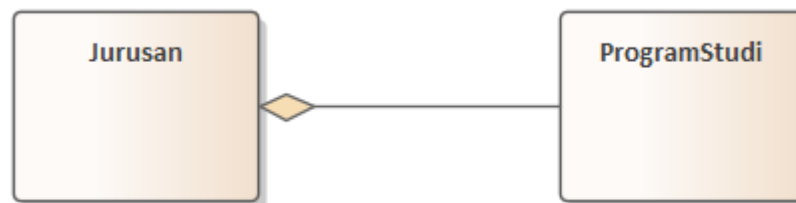


ASSOCIATION



AGGREGATION

- Aggregation merupakan bentuk khusus dari Association dimana terdapat ownership antara satu class terhadap class lain, namun masing-masing objek memiliki life-cycle nya sendiri
- Unidirectional association/one-way relationship
- “has-a” + “whole-part” relationship
- Ownership artinya ada kepemilikan antara kedua class tersebut. Contoh:
 - Jurusan *terdiri dari* beberapa Program Studi
 - Program Studi merupakan *bagian dari* Jurusan
- Jurusan dan Program Studi memiliki life-cyle nya masing-masing, artinya jika Jurusan dihapus, Program Studi tidak secara otomatis terhapus juga.



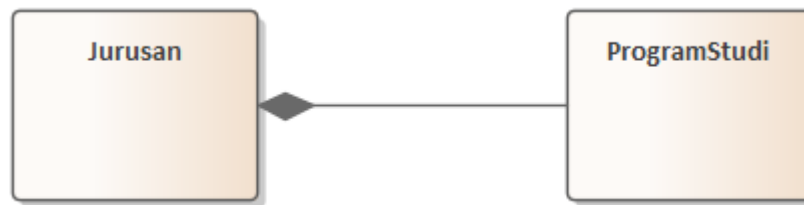
COMPOSITION

- Composition merupakan bentuk khusus dari Aggregation dimana terdapat ownership antarkelas tetapi child object tidak memiliki life-cycle sendiri tanpa parent object
- Tidak memiliki life cycle sendiri artinya, child object tidak dapat dibuat tanpa ada parent object; child object juga akan terhapus jika parent object dihapus.
- Contoh:
 - Gedung *terdiri dari* Ruangan
 - Karyawan *memiliki* Tanggungan
 - E-mail *memiliki* Attachment

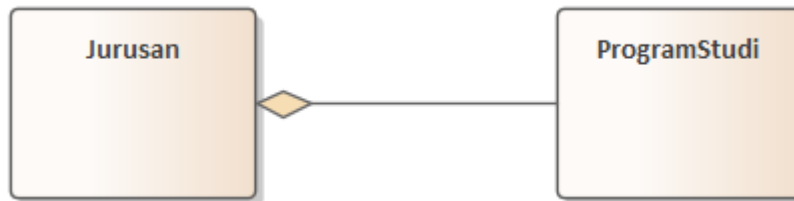


JURUSAN & PROGRAM STUDI

- Jika jurusan dihapus, maka otomatis program studi dihapus → composition



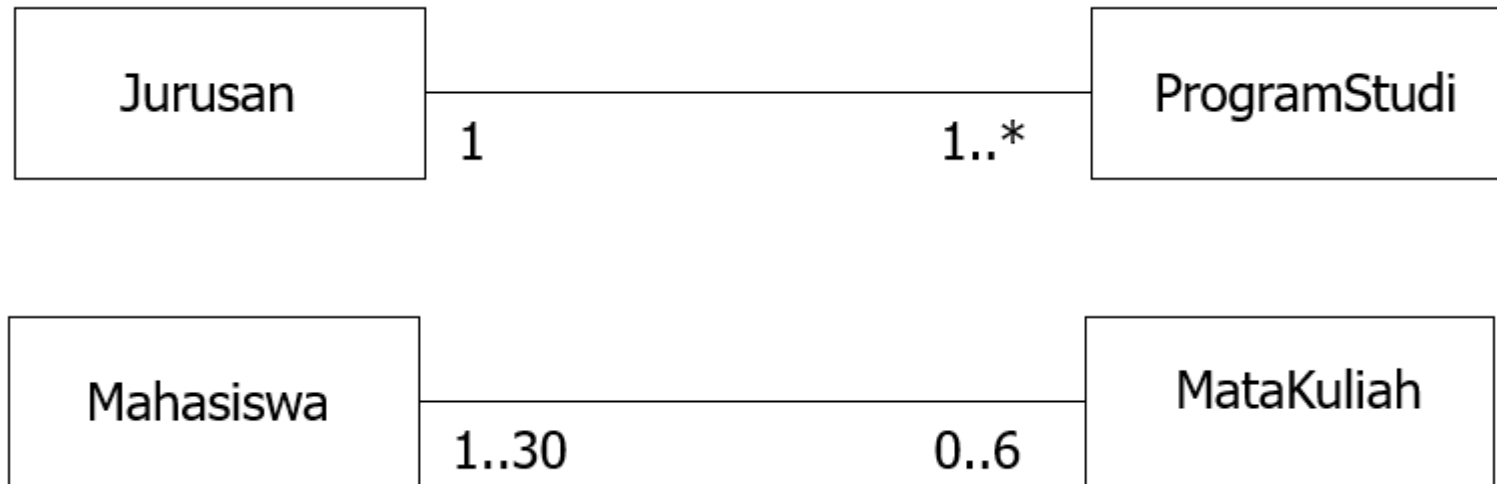
- Jika jurusan dihapus, program studi bisa berpindah ke jurusan lain → aggregation

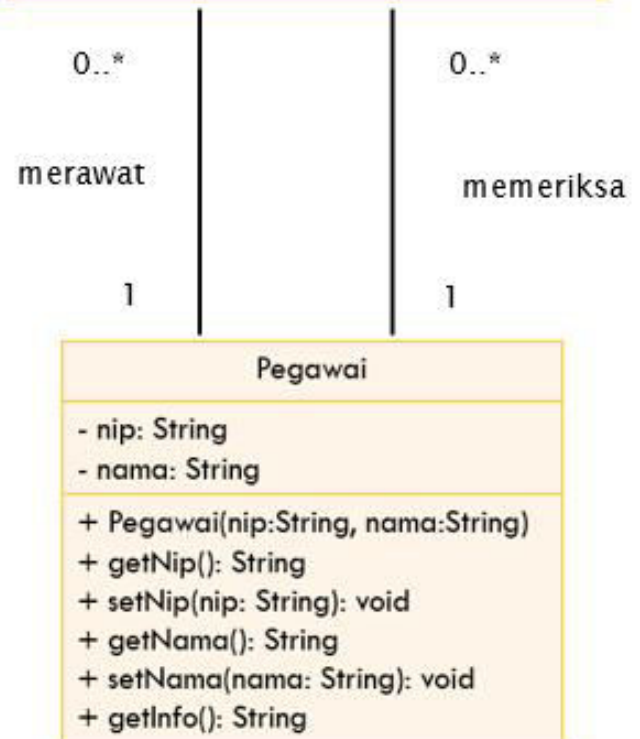
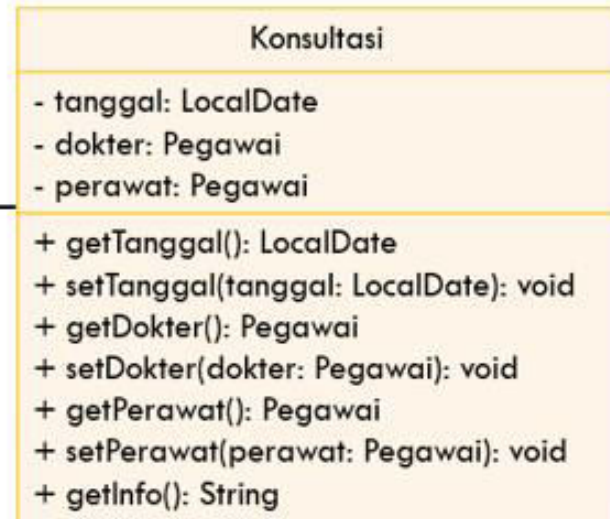
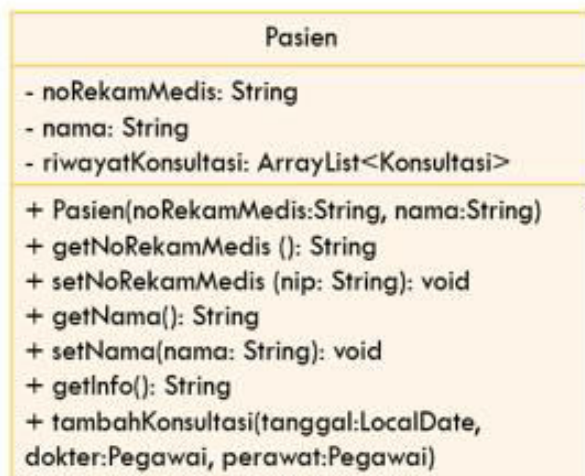


MULTIPLICITY NOTATION

Contoh	Keterangan
1	Tepat satu
0..1	Nol atau satu
0..*	Nol atau lebih
1..*	Satu atau lebih
3..5	3 hingga 5

MULTIPLICITY NOTATION





1

0..*

0..*

merawat

1

0..*

memeriksa

1

LATIHAN

- Buat sebuah studi kasus dengan minimal 3 class yang saling berasosiasi.
- Buat minimal 1 atribut bertipe ArrayList of objects
- Tentukan atribut dan method pada masing-masing class
- Gambarkan class diagramnya.



PEMROGRAMAN BERBASIS OBJECT

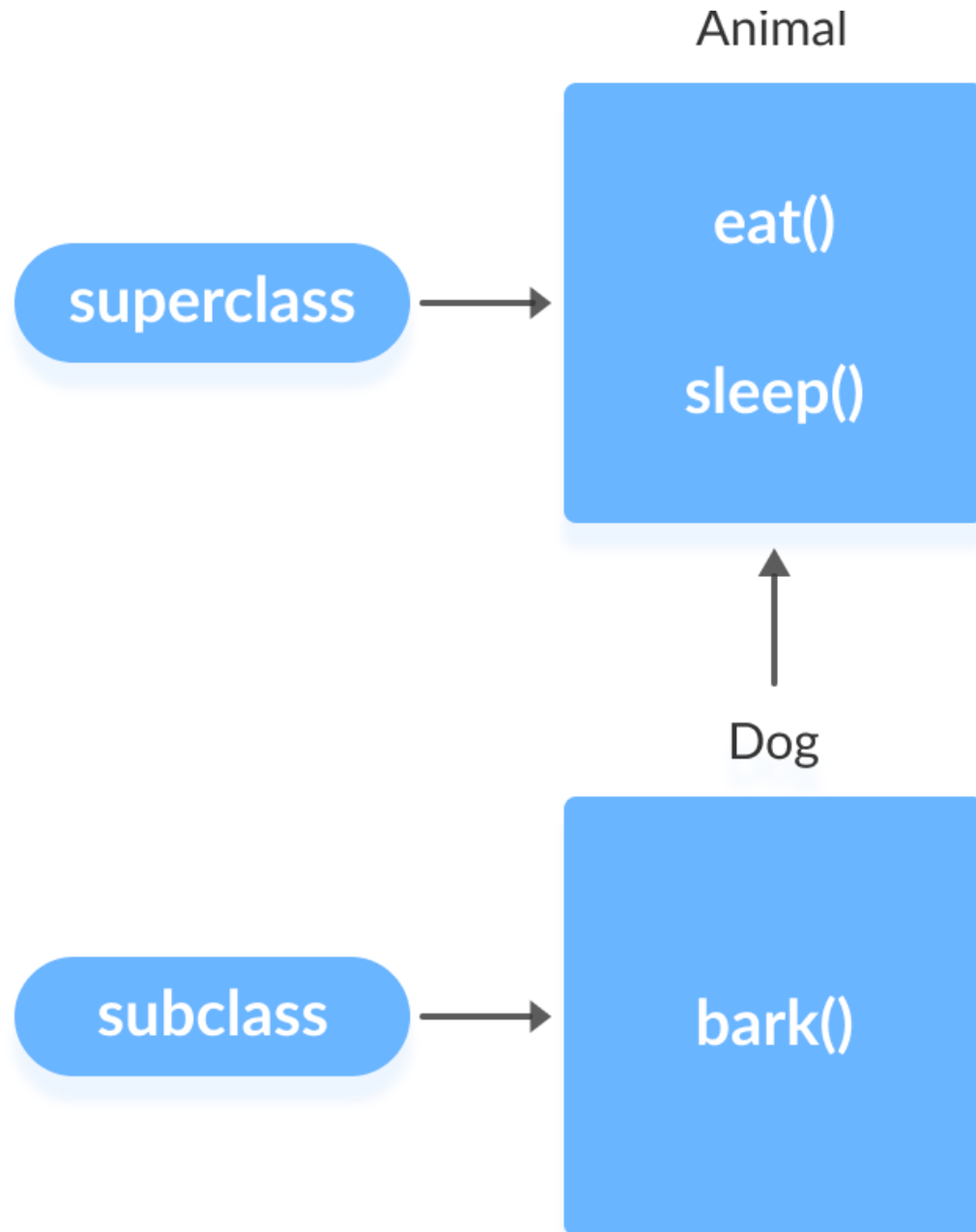
Minggu 6: Inheritance

Outline

- Pengertian inheritance
- Deklarasi inheritance
- Access Control
- Super keyword
- Jenis inheritance

Pengertian Dasar

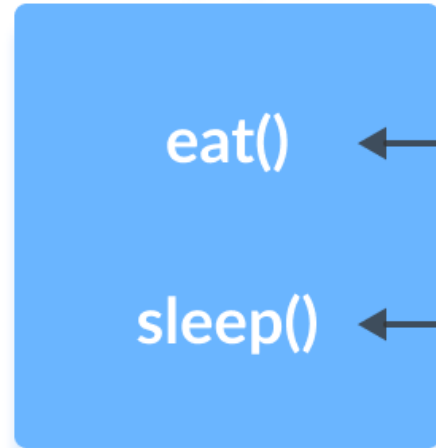
- ❑ Inheritance (Pewarisan) merupakan salah satu konsep dasar OOP.
- ❑ Konsep inheritance ini mengadopsi dunia nyata, dimana suatu entitas/obyek dapat mempunyai entitas/obyek turunan.
- ❑ Dengan konsep inheritance, sebuah class dapat mempunyai class turunan.



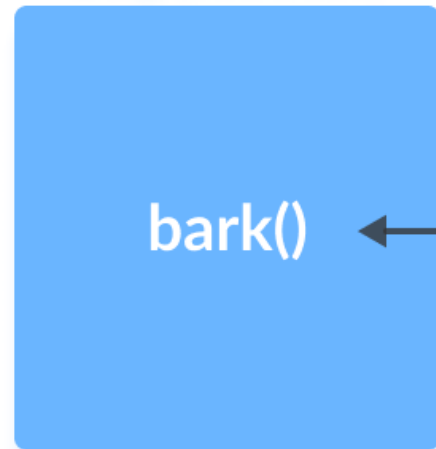
Pengertian

- Suatu class yang mempunyai class turunan dinamakan **superclass/parent class/base class**.
- Sedangkan class turunan itu sendiri seringkali disebut **subclass/child class/derived class**.
- Suatu subclass dapat mewarisi apa-apa yang dipunyai oleh parent class.

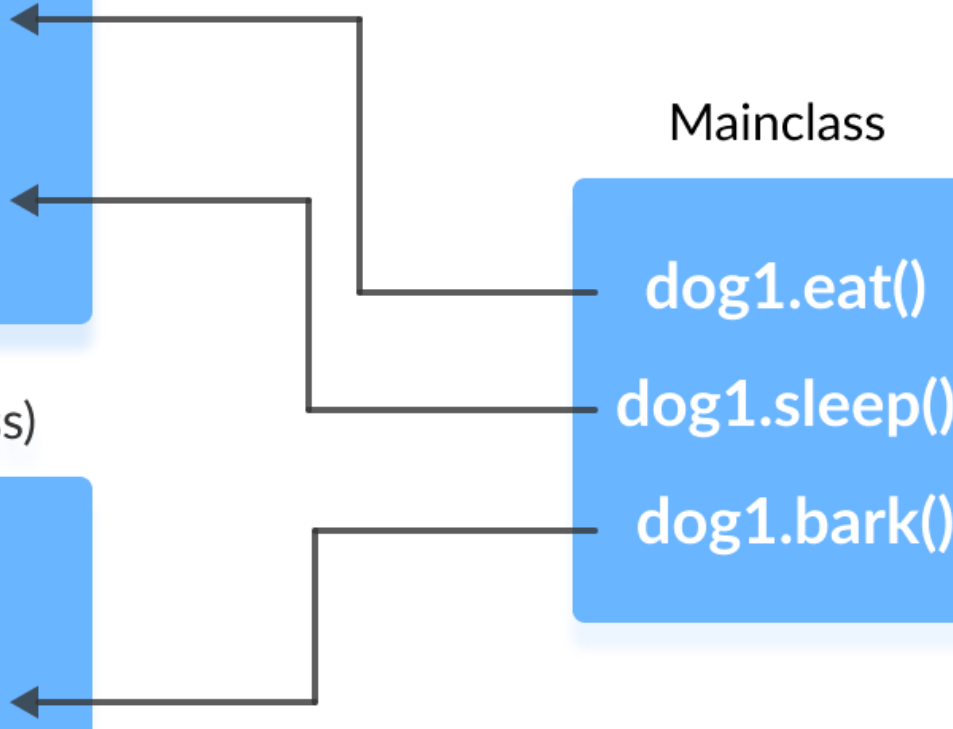
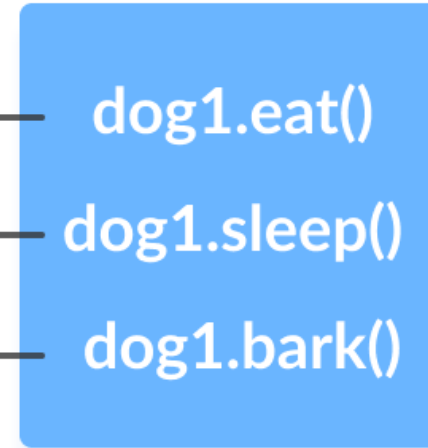
Animal (superclass)



Dog (subclass)



Mainclass



Deklarasi inheritance

- ❑ Dengan menambahkan kata kunci **extends** setelah deklarasi nama class, kemudian diikuti dengan nama parent class-nya.

```
public class B extends A
{
    ...
}
```

Kapan kita menerapkan inheritance?

- Kita baru perlu menerapkan inheritance pada saat dijumpai ada suatu class yang dapat diperluas dari class lain

class Pegawai

```
public class Pegawai{  
    public string nip;  
    public String nama;  
    public double gaji;  
}
```

class Dosen

```
public class Dosen{  
    public string nip;  
    public String nama;  
    public double gaji;  
    public String NIDN;  
}
```

- Dari 2 class di atas, kita lihat class **Dosen** memiliki atribut yang identik sama dengan class **Pegawai**, hanya saja ada tambahan atribut **NIDN (Nomor Induk Dosen Nasional)**.
- Sebenarnya yang terjadi disana adalah class Dosen merupakan **extension/perluasan** dari class Pegawai berupa tambahan atribut NIDN.
- Dengan konsep inheritance, deklarasi class Dosen dapat dituliskan sebagai berikut:

```
public class Dosen extends Pegawai {  
    public String NIDN;  
}
```

Access Control

- Dalam dunia riil, suatu entitas induk bisa saja tidak mewariskan sebagian dari apa-apa yang ia punyai kepada entitas turunan karena sesuatu hal.
- Demikian juga dengan konsep inheritance dalam OOP.
- Suatu parent class bisa saja tidak mewariskan sebagian atribut atau method kepada subclass-nya.
- Hal ini dilakukan dengan memberikan access level modifier **private**

Kata Kunci “super”

- Kata kunci **super** dipakai untuk merujuk pada **parent class/object**
- Sebagaimana kata kunci **this** yang dipakai untuk merujuk pada **current class/object**
- Format penulisannya adalah sebagai berikut :
 - **super()**
→ merujuk pada constructor parent
 - **super.<namaAttribute>**
→ merujuk pada atribut dari parent
 - **super.<namaMethod>()**
→ merujuk pada method dari parent

Kesimpulan

- **x**
→ merujuk pada x terdekat, yaitu parameter method info()
- **this.x**
→ merujuk pada atribut x pada class Child
- **super.x**
→ merujuk pada atribut x pada class Parent

Single Inheritance

- Konsep inheritance dengan subclass yang mempunyai satu parent class.



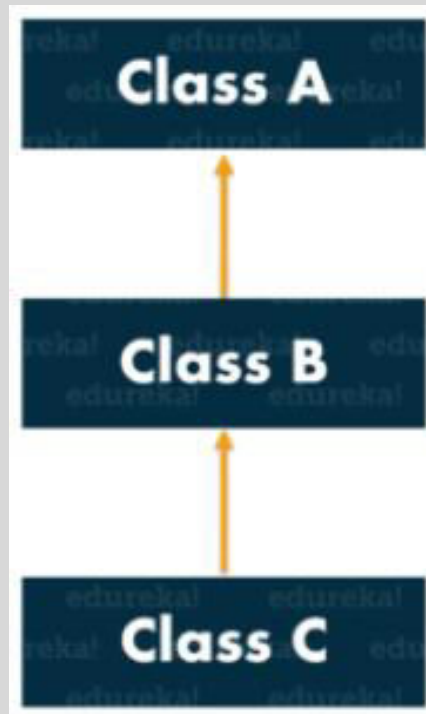
```
class Animal{  
void eat(){System.out.println("eating...");}  
}
```

```
class Dog extends Animal{  
void bark(){System.out.println("barking...");}  
}
```

```
class TestInheritance{  
    public static void main(String args[]){  
        Dog d=new Dog();  
        d.bark();  
        d.eat();  
    }  
}
```


Multilevel Inheritance

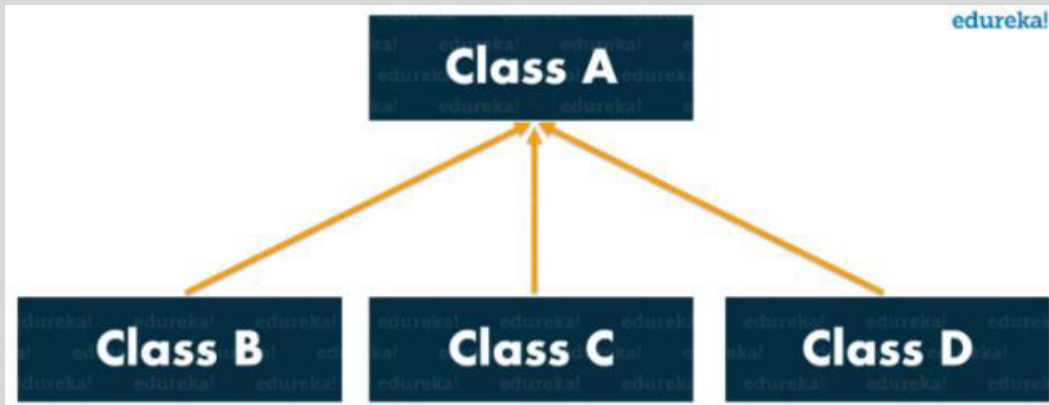
- Ketika kelas diturunkan dari kelas yang juga diturunkan dari kelas lain, yaitu kelas yang memiliki lebih dari satu kelas induk tetapi pada tingkat yang berbeda



```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
    void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
    public static void main(String args[]){
        BabyDog d=new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}
```

Hierarchical Inheritance

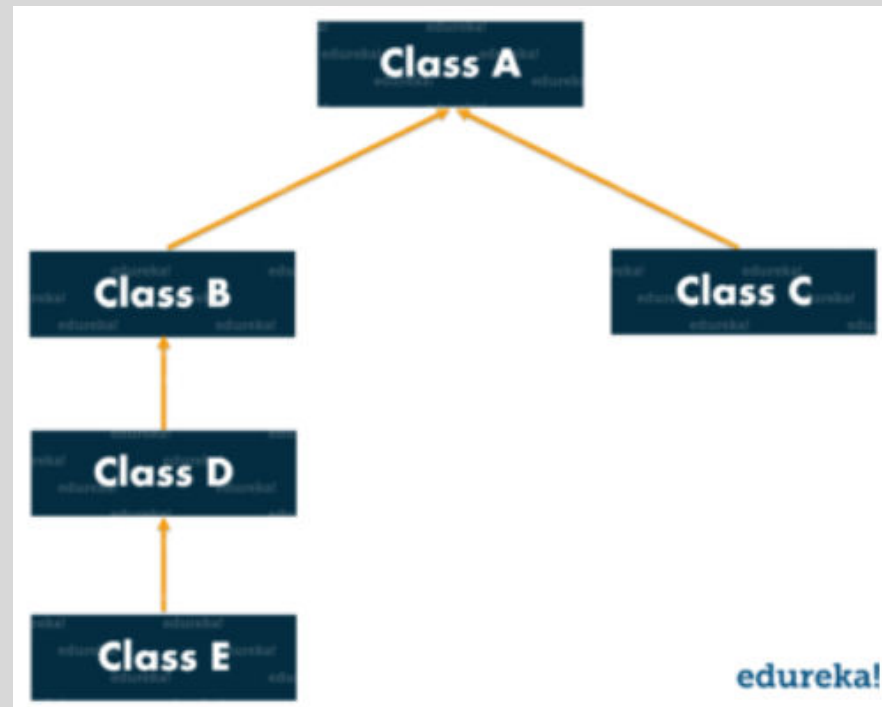
- Ketika sebuah kelas memiliki lebih dari satu kelas turunan (*subclass*) atau dengan kata lain, lebih dari satu kelas turunan memiliki kelas induk yang sama.



```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
    void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
    public static void main(String args[]){
        Cat c=new Cat();
        c.meow();
        c.eat();
        //c.bark();//C.T.Error
    }
}
```

Hybrid Inheritance

- Kombinasi dua atau lebih jenis inheritance.



Hybrid Inheritance

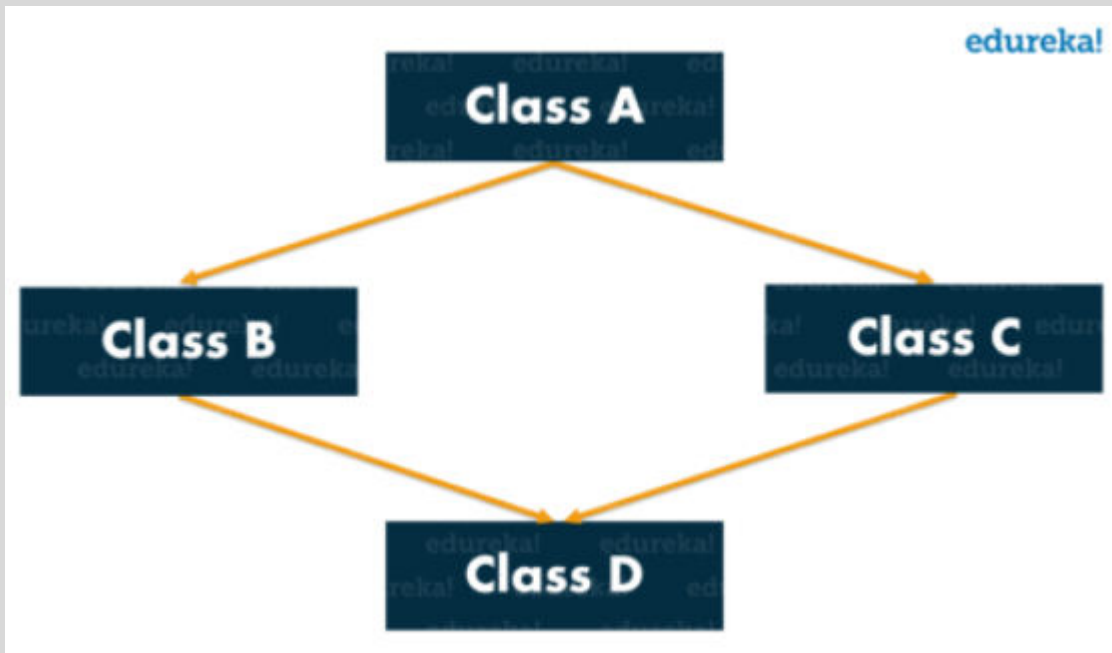
```
class A
{
    public void display()
    {
        System.out.println("A");
    }
}
```

```
class B extends A
{
    public void display()
    {
        System.out.println("B");
    }
}
```

```
class C extends A
{
    public void display()
    {
        System.out.println("C");
    }
}
```

```
class D extends B
{
    public void display()
    {
        System.out.println("D");
    }
    public static void main(String args[]){
        D obj = new D();
        obj.display();
    }
}
```

Multiple Inheritance



- Multiple inheritance: satu kelas turunan mencoba untuk **memperluas (extend) lebih dari satu kelas induk**.
- Misal terdapat method *show()* pada kelas B dan C dengan fungsi yang berbeda. Kemudian kelas D meng-extend kelas B dan C. Ketika objek dari kelas D mencoba memanggil method *show()*, kompilator akan bingung method di kelas mana yang akan dieksekusi (dari kelas B atau C)
- Mengarah pada ambiguitas.

Mengapa Multiple Inheritance tidak bisa diimplementasikan di Java?

```
class A{
    void message(){System.out.println("Hello");}
}
class B extends A{
    void show(){System.out.println("Welcome");}
}
class C extends A{
    void show(){System.out.println("Good Morning");}
}
```

```
class D extends B,C{
    void print(){System.out.println("This is multiple inheritance");}
}
class TestInheritance4{
    public static void main(String args[]){
        D obj=new D();
        obj.print();
        obj.show();//method show() mana yang akan dipanggil
    }
}
```

 **TERIMA KASIH!** 



PEMROGRAMAN BERBASIS OBJECT

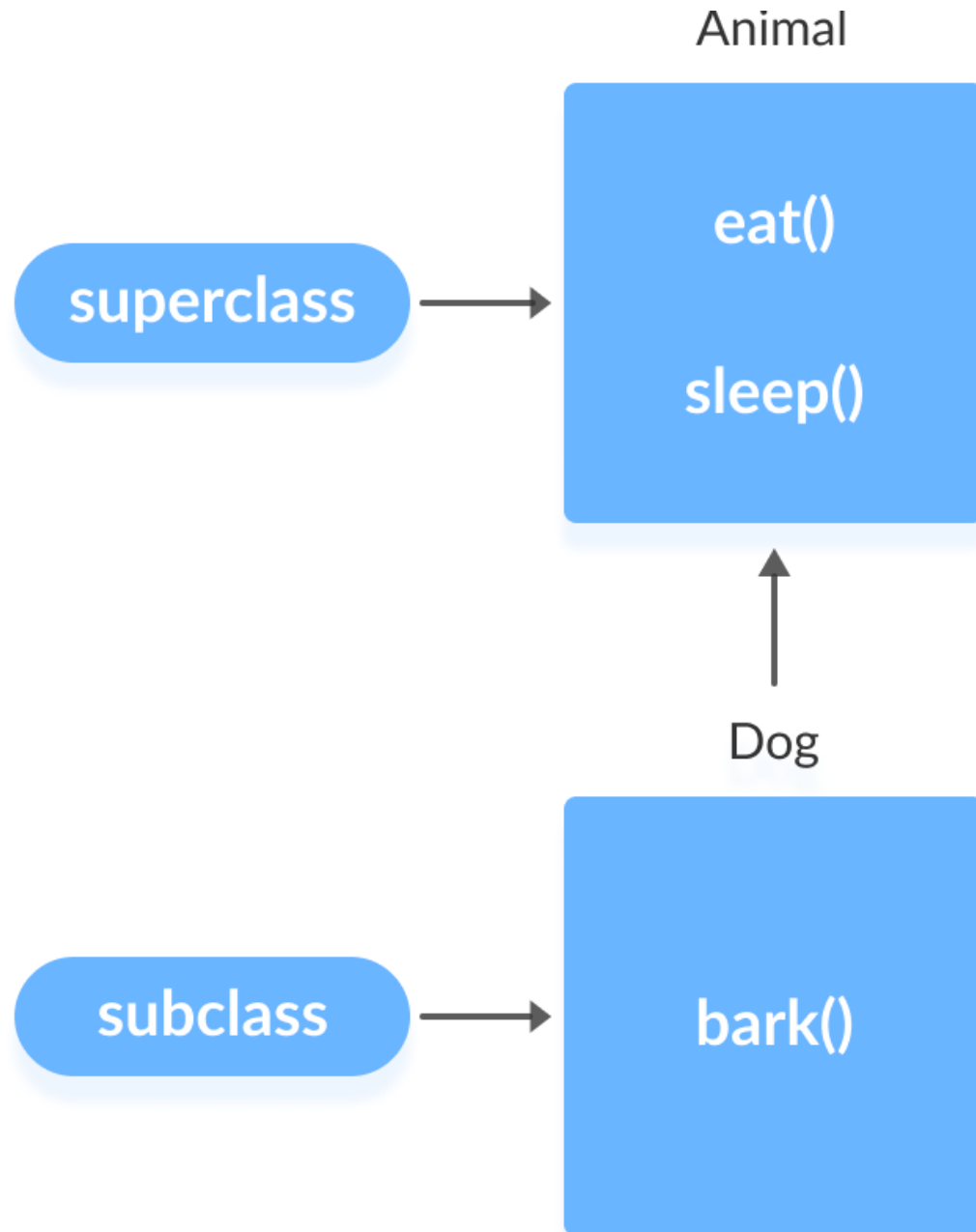
Minggu 6: Inheritance

Outline

- Pengertian inheritance
- Deklarasi inheritance
- Access Control
- Super keyword
- Jenis inheritance

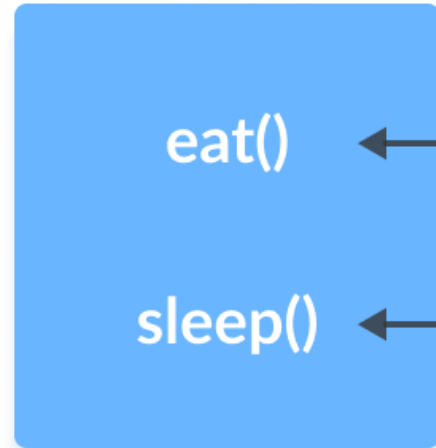
Inheritance

- ❑ Inheritance (Pewarisan) merupakan salah satu konsep dasar OOP.
- ❑ Konsep inheritance ini mengadopsi dunia nyata, dimana suatu entitas/obyek dapat mempunyai entitas/obyek turunan.
- ❑ Dengan konsep inheritance, sebuah class dapat mempunyai class turunan.
- ❑ Konsep inheritance digunakan untuk memanfaatkan fitur 'code reuse' untuk menghindari duplikasi kode program

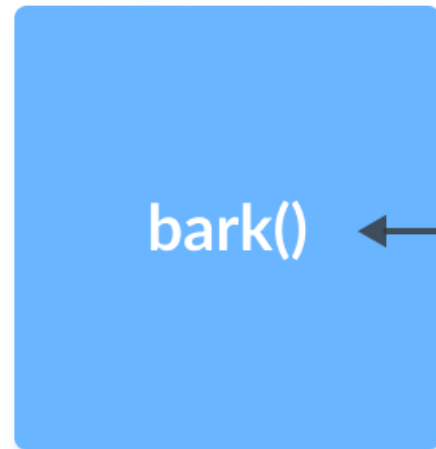


- Suatu class yang mempunyai class turunan dinamakan **superclass/parent class/base class**.
- Sedangkan class turunan itu sendiri seringkali disebut **subclass/child class/derived class**.
- Suatu subclass dapat mewarisi apa-apa yang dipunyai oleh parent class.

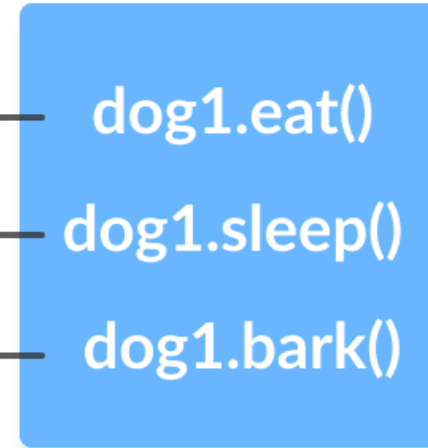
Animal (superclass)



Dog (subclass)



Mainclass



`eat()`

`sleep()`

`bark()`

`dog1.eat()`

`dog1.sleep()`

`dog1.bark()`



Deklarasi inheritance

- ❑ Dengan menambahkan kata kunci **extends** setelah deklarasi nama class, kemudian diikuti dengan nama parent class-nya.

```
public class <Subclass> extends <Superclass>
{
    ...
}
```

Kapan kita menerapkan inheritance?

- Kita baru perlu menerapkan inheritance pada saat dijumpai ada suatu class yang dapat di-extend/diperluas/dikembangkan dari class lain

class Pegawai

```
public class Pegawai {  
    public String nip;  
    public String nama;  
    public double gaji;  
}
```

class Dosen

```
public class Dosen {  
    public String nip;  
    public String nama;  
    public double gaji;  
    public String NIDN;  
}
```

- Dari 2 class di atas, kita lihat class **Dosen** memiliki atribut yang identik sama dengan class **Pegawai**, hanya saja ada tambahan atribut **NIDN (Nomor Induk Dosen Nasional)**.
- Sebenarnya yang terjadi disana adalah class Dosen merupakan **extension/perluasan** dari class Pegawai berupa tambahan atribut NIDN.
- Dengan konsep inheritance, deklarasi class Dosen dapat dituliskan sebagai berikut:

```
public class Dosen extends Pegawai {  
    public String NIDN;  
}
```


class Pegawai

```
public class Pegawai {  
    public String nip;  
    public String nama;  
    public double gaji;  
}
```

class Dosen

```
public class Dosen extends Pegawai {  
    public String NIDN;  
}
```

Kata Kunci “super”

- Kata kunci **super** dipakai untuk merujuk pada **parent class/object**
- Kata kunci **this** dipakai untuk merujuk pada **current class/object**
- Format penulisannya adalah sebagai berikut:
 - **super()**
→ merujuk pada constructor parent
 - **super.<namaAttribute>**
→ merujuk pada atribut dari parent
 - **super.<namaMethod>()**
→ merujuk pada method dari parent

Variable Scope

```
public class Kendaraan {  
    public int x = 5;  
}
```

```
public class Mobil extends Kendaraan{  
    public int x = 10;  
  
    public void displayInfo(int x){  
        System.out.println("Nilai x: " + x);  
        System.out.println("Nilai this.x: " + this.x);  
        System.out.println("Nilai super.x: " + super.x);  
    }  
}
```

```
public class Demo {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil();  
        mobil1.displayInfo(12);  
    }  
}
```

```
Nilai x: 12  
Nilai this.x: 10  
Nilai super.x: 5
```

Variable Scope (2)

```
public class Kendaraan {  
    public int x = 5;  
}
```

```
public class Mobil extends Kendaraan{  
    public int x = 10;  
  
    public void displayInfo(){  
        System.out.println("Nilai x: " + x);  
        System.out.println("Nilai this.x: " + this.x);  
        System.out.println("Nilai super.x: " + super.x);  
    }  
}
```

```
public class Demo {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil();  
        mobil1.displayInfo();  
    }  
}
```

```
Nilai x: 10  
Nilai this.x: 10  
Nilai super.x: 5
```

Variable Scope (3)

```
public class Kendaraan {  
    public int x = 5;  
}
```

```
public class Mobil extends Kendaraan {  
    public String platNomor;  
  
    public void displayInfo() {  
        System.out.println("Nilai x: " + x);  
        System.out.println("Nilai this.x: " + this.x);  
        System.out.println("Nilai super.x: " + super.x);  
    }  
}
```

```
public class Demo {  
    Run | Debug  
    public static void main(String[] args) {  
        Mobil mobil1 = new Mobil();  
        mobil1.displayInfo();  
    }  
}
```

```
Nilai x: 5  
Nilai this.x: 5  
Nilai super.x: 5
```

```
public class Kendaraan {  
    private int x = 5;  
  
    public void displayInfo()  
    {  
        System.out.println("Nilai this.x: " + this.x);  
    }  
}
```

```
public class Mobil extends Kendaraan{  
    public int x = 10;  
  
    public void displayInfo(int x){  
        System.out.println("Nilai x: " + x);  
        System.out.println("Nilai this.x: " + this.x);  
        System.out.println("Nilai super.x: " + super.x);  
    }  
}
```

```
public class Demo {  
    Run | Debug  
    public static void main(String[] args) {  
        Kendaraan kendaraan1 = new Kendaraan();  
        kendaraan1.displayInfo();  
    }  
}
```

Nilai this.x: 5

Kesimpulan

- **x**
 - merujuk pada x terdekat, cari dulu variable/parameter. Jika tidak ada berarti merujuk pada atribut class tersebut. Jika tidak ada berarti merujuk pada atribut parent class
- **this.x**
 - merujuk pada atribut x pada current class/object. Jika tidak ada berarti merujuk pada atribut parent class
- **super.x**
 - merujuk pada atribut x pada parent class/object

Access Control

- Dalam dunia nyata, suatu entitas induk bisa saja tidak mewariskan sebagian dari apa-apa yang ia miliki kepada entitas turunan karena sesuatu hal.
- Demikian juga dengan konsep inheritance dalam OOP.
- Suatu parent class bisa saja **tidak mewariskan** sebagian atribut atau method kepada subclass-nya. Hal ini dilakukan dengan memberikan access level modifier **private**

Access Control

```
public class Kendaraan {  
    private int x = 5;  
  
    public void displayInfo()  
    {  
        System.out.println("Nilai this.x: " + this.x);  
    }  
}
```

```
public class Mobil extends Kendaraan{  
    public int x = 10;  
  
    public void displayInfo(int x){  
        System.out.println("Nilai x: " + x);  
        System.out.println("Nilai this.x: " + this.x);  
        System.out.println("Nilai super.x: " + super.x);  
    }  
}
```

The field Kendaraan.x is not visible Java(33554503)

int x

[View Problem \(Alt+F8\)](#) [Quick Fix... \(Ctrl+.\)](#)

Single Inheritance

- Konsep inheritance dengan subclass yang mempunyai satu parent class.



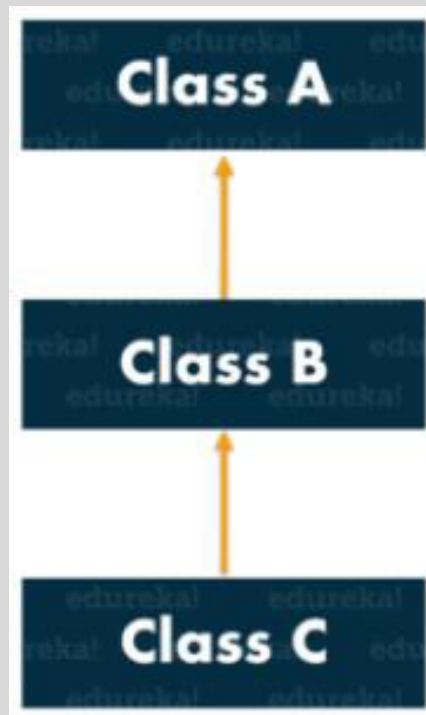
```
class Animal {  
    void eat() {  
        System.out.println("eating...");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("barking...");  
    }  
}
```

```
class TestInheritance{  
    public static void main(String args[]) {  
        Dog d=new Dog();  
        d.bark();  
        d.eat();  
    }  
}
```

Multilevel Inheritance

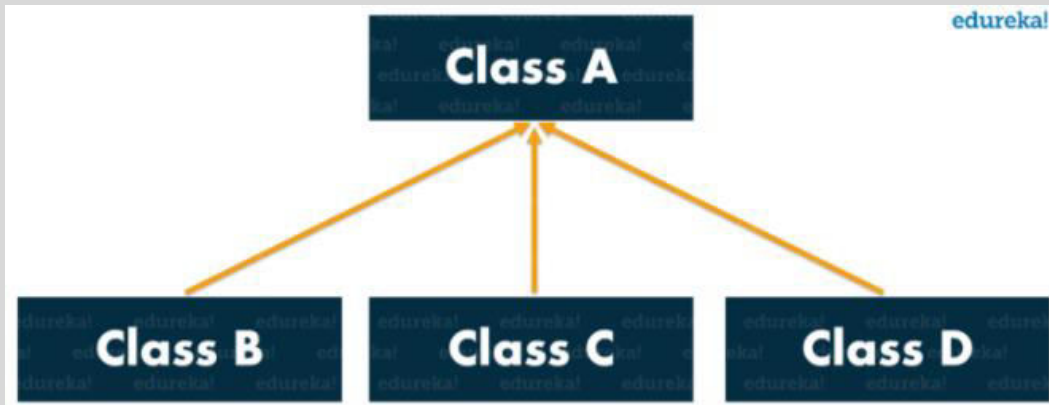
- Ketika kelas diturunkan dari kelas yang juga diturunkan dari kelas lain, yaitu kelas yang memiliki lebih dari satu kelas induk tetapi pada tingkat yang berbeda



```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class Puppy extends Dog{
    void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
    public static void main(String args[]){
        Puppy puppy1=new BabyDog();
        puppy1.weep();
        puppy1.bark();
        puppy1.eat();
    }
}
```

Hierarchical Inheritance

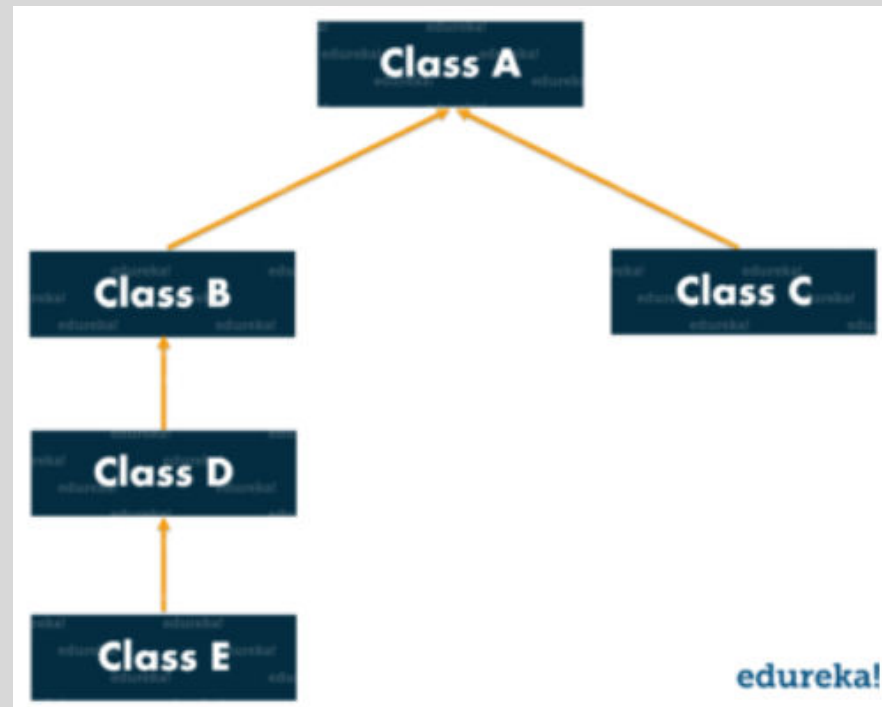
- Ketika sebuah kelas memiliki lebih dari satu kelas turunan (*subclass*) atau dengan kata lain, lebih dari satu kelas turunan memiliki kelas induk yang sama.



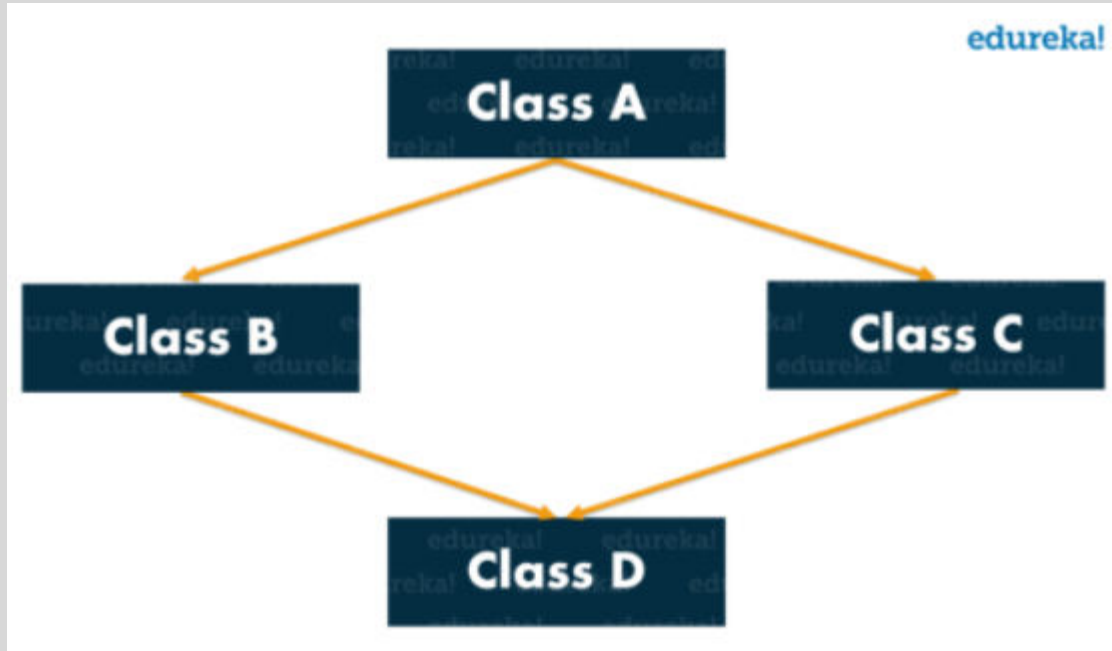
```
class Animal{
    void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
    void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
    void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
    public static void main(String args[]){
        Cat cat1=new Cat();
        cat1.meow();
        cat1.eat();
        //cat1.bark(); //C.T.Error
    }
}
```

Hybrid Inheritance

- Kombinasi dua atau lebih jenis inheritance.



Multiple Inheritance



- Multiple inheritance: satu kelas turunan mencoba untuk **memperluas (extend) lebih dari satu kelas induk**.
- Misal terdapat method *show()* pada kelas B dan C dengan fungsi yang berbeda. Kemudian kelas A meng-extend kelas B dan C. Ketika objek dari kelas A mencoba memanggil method *show()*, Java compiler akan bingung method di kelas mana yang akan dieksekusi (dari kelas B atau C)
- Mengarah pada ambiguitas.

Mengapa Multiple Inheritance tidak bisa diimplementasikan di Java?

```
class B extends D{
    void show(){
        System.out.println("Welcome");
    }
}

class C extends D{
    void show(){
        System.out.println("Good Morning");
    }
}
```

```
class A extends B,C{
    void print(){System.out.println("This is multiple inheritance");}
}

class TestInheritance4{
    public static void main(String args[]){
        A obj=new A();
        obj.print();
        obj.show(); //method show() mana yang akan dipanggil
    }
}
```

Tugas

- Buatlah sebuah contoh hierarchical inheritance dengan 1 parent class dan 2 child class. Pada parent class terdapat minimal 3 atribut dan pada setiap child class terdapat minimal 1 atribut tambahan. Buatlah class diagramnya

Best Practice

Kendaraan
kecepatan: int



Mobil
warna: String

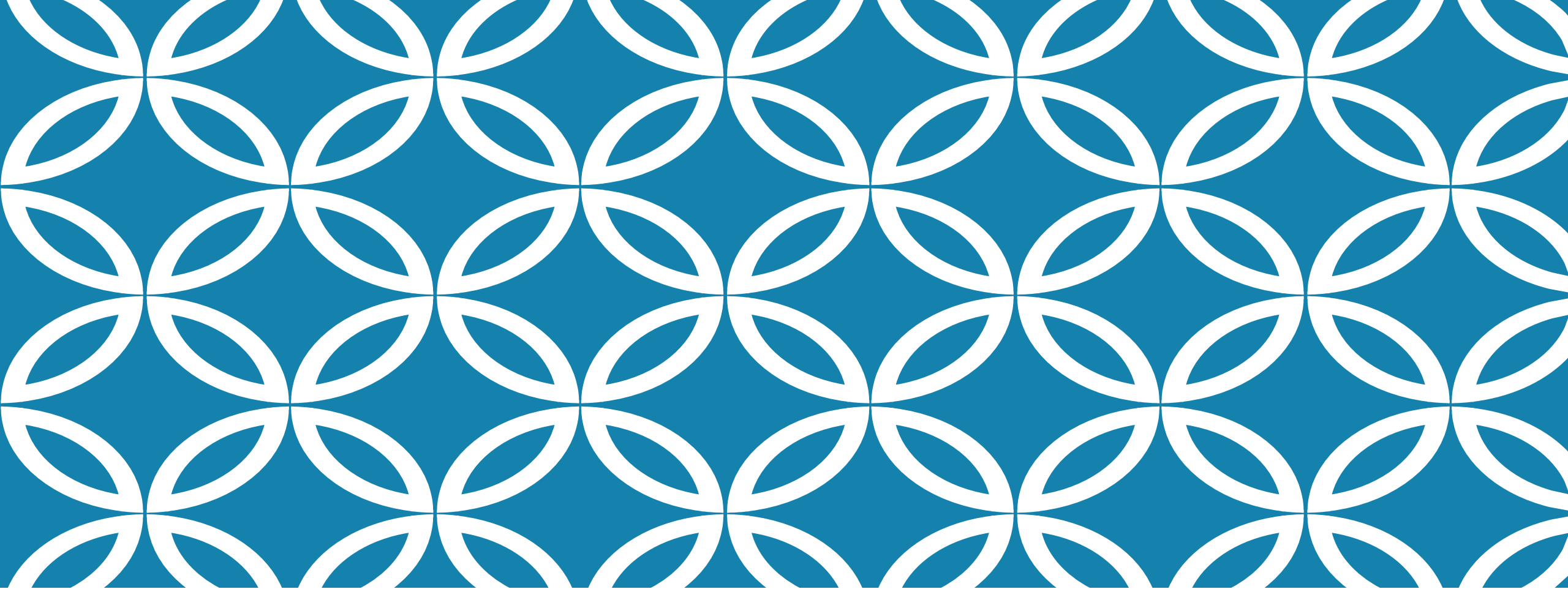
Becak
warna: String

Motor
warna: String



TERIMA KASIH





OVERLOADING & OVERRIDING



POLYMORPHISM

- Polymorphism: poly (banyak), morph (bentuk)
- Konsep polimorfisme pada OOP membolehkan sebuah aksi diimplementasikan secara berbeda

POLYMORPHISM

1. Overloading

- JVM menentukan method mana yang akan dipanggil pada **compile-time** → compile-time polymorphism
- Disebut juga static binding atau early binding

2. Overriding

- JVM menentukan method mana yang akan dipanggil pada saat **run-time** → run-time polymorphism
- Disebut juga dynamic binding atau late binding

OVERLOADING

- Method overloading berarti kondisi dimana ada method dengan nama yang sama, tetapi memiliki method signature yang berbeda.
- **Method signature**: jumlah, tipe data dan susunan parameter
- Method overloading dapat terjadi pada kelas yang sama atau kelas lain yang terkait dalam hierarki pewarisan.
- Karakteristik overloading:
 1. Nama method sama.
 2. Method signature berbeda.
 3. Return type boleh sama atau berbeda.

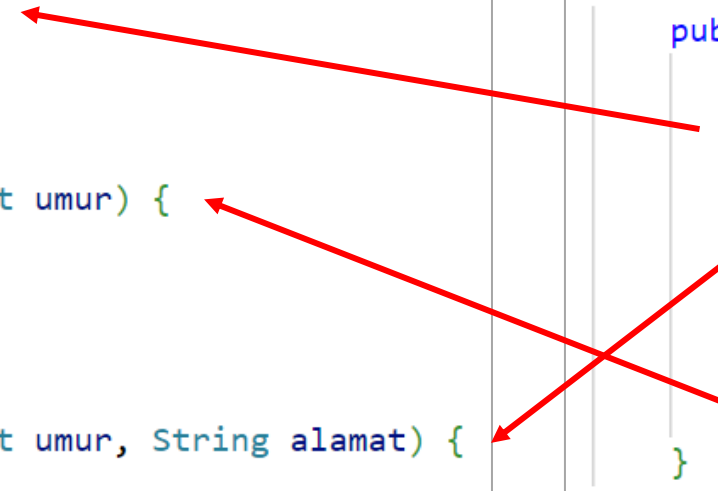
OVERLOADING EXAMPLE

```
public class Mahasiswa {  
    public String nama;  
    public int umur;  
    public String alamat;  
  
    public void setBiodata(String nama) {  
        this.nama = nama;  
    }  
  
    public void setBiodata(String nama, int umur) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
  
    public void setBiodata(String nama, int umur, String alamat) {  
        this.nama = nama;  
        this.umur = umur;  
        this.alamat = alamat;  
    }  
}
```

Di kelas Mahasiswa, terdapat 3 method dengan nama setBiodata() tetapi dengan signature yang berbeda.

```
public class Mahasiswa {  
    public String nama;  
    public int umur;  
    public String alamat;  
  
    public void setBiodata(String nama) {  
        this.nama = nama;  
    }  
  
    public void setBiodata(String nama, int umur) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
  
    public void setBiodata(String nama, int umur, String alamat) {  
        this.nama = nama;  
        this.umur = umur;  
        this.alamat = alamat;  
    }  
}
```

```
public class Test {  
    Run | Debug  
    public static void main(String[] args) {  
        Mahasiswa mahasiswa1 = new Mahasiswa();  
        mahasiswa1.setBiodata("Ani");  
  
        Mahasiswa mahasiswa2 = new Mahasiswa();  
        mahasiswa2.setBiodata("Budi", 18, "Batu");  
  
        Mahasiswa mahasiswa3 = new Mahasiswa();  
        mahasiswa3.setBiodata("Cica", 19);  
    }  
}
```



METHOD SIGNATURE

- Salah satu karakteristik overloading adalah method signature yang berbeda
- **Method signature**: jumlah, tipe data dan susunan parameter
- Method signature disebut berbeda jika:
 - ☐ Jumlahnya berbeda, *atau*
 - ☐ Jumlahnya sama tapi tipe data berbeda , *atau*
 - ☐ Jumlah dan tipe data sama tapi susunannya berbeda
- Perhatikan bahwa **nama parameter tidak diperhitungkan**

METHOD SIGNATURE (2)

```
public class Mahasiswa {  
    public String nama;  
    public int umur;  
    public String alamat;  
  
    public void setBiodata(String nama, int umur) {  
        this.nama = nama;  
    }  
  
    public void setBiodata(String alamat, int umur) {  
        this.alamat = alamat;  
    }  
}
```

Jumlah, tipe data,
susunan sama

Compile error:

Duplicate method setBiodata(String, int)

METHOD SIGNATURE (3)

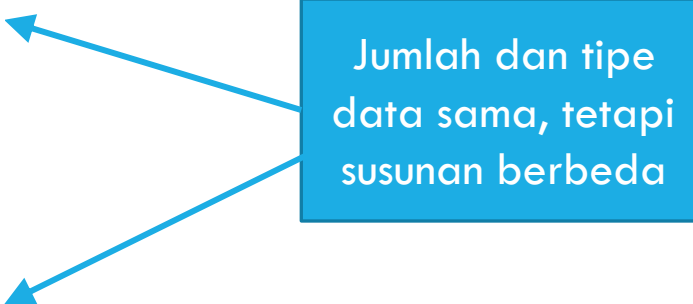
```
public class Mahasiswa {  
    public String nama;  
    public int umur;  
    public String alamat;  
  
    public void setBiodata(String nama, int umur) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
  
    public void setBiodata(String alamat, int umur) {  
        this.alamat = alamat;  
        this.umur = umur;  
    }  
}
```

```
public class Test {  
    Run | Debug  
    public static void main(String[] args) {  
        Mahasiswa mahasiswa1 = new Mahasiswa();  
        mahasiswa1.setBiodata("abcdefg", 19);  
    }  
}
```

METHOD SIGNATURE (4)

```
public class Mahasiswa {  
    public String nama;  
    public int umur;  
    public String alamat;  
  
    public void setBiodata(String nama, int umur) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
  
    public void setBiodata(int umur, String nama) {  
        this.nama = nama;  
        this.umur = umur;  
    }  
}
```

Jumlah dan tipe
data sama, tetapi
susunan berbeda



OVERLOADING OF A CONSTRUCTOR

```
public class Donat {  
    public String topping;  
  
    public Donat() {  
  
    }  
  
    public Donat(String topping) {  
        this.topping = topping;  
    }  
}
```

OVERLOADING OF A CONSTRUCTOR

```
public class Donat {  
    public String topping;  
  
    public Donat() {  
  
    }  
  
    public Donat(String topping) {  
        this.topping = topping;  
    }  
}
```

```
public class DemoDonat {  
    Run | Debug  
    public static void main(String[] args) {  
        Donat donat1 = new Donat("Choco");  
  
        Donat donat2 = new Donat();  
        donat2.topping = "Strawberry";  
    }  
}
```

WHEN DO WE USE OVERLOADING?

- Terkadang kita perlu membuat beberapa method yang memiliki fungsi/tujuan yang mirip, tetapi dengan parameter yang berbeda atau implementasi yang berbeda
- Menyederhanakan kode program dan memunculkan konsistensi dalam penamaan method

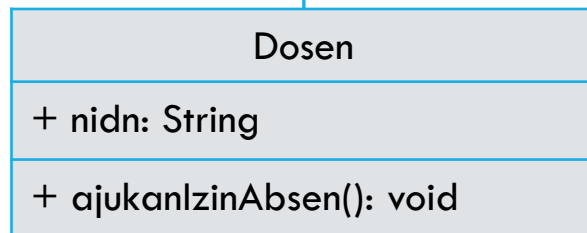
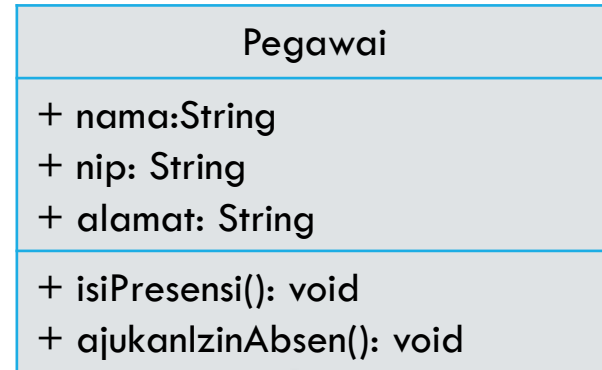
OVERRIDING

Overriding terjadi ketika subclass memiliki method dengan nama dan signature yang sama dengan superclass nya.

Karakteristik:

1. Dilakukan oleh method pada subclass terhadap method pada superclass
2. Nama method sama
3. Method signature sama

Pada class Dosen terdapat method
ajukanIzinAbsen() dengan signature yang sama
dengan class Pegawai → *Overriding*

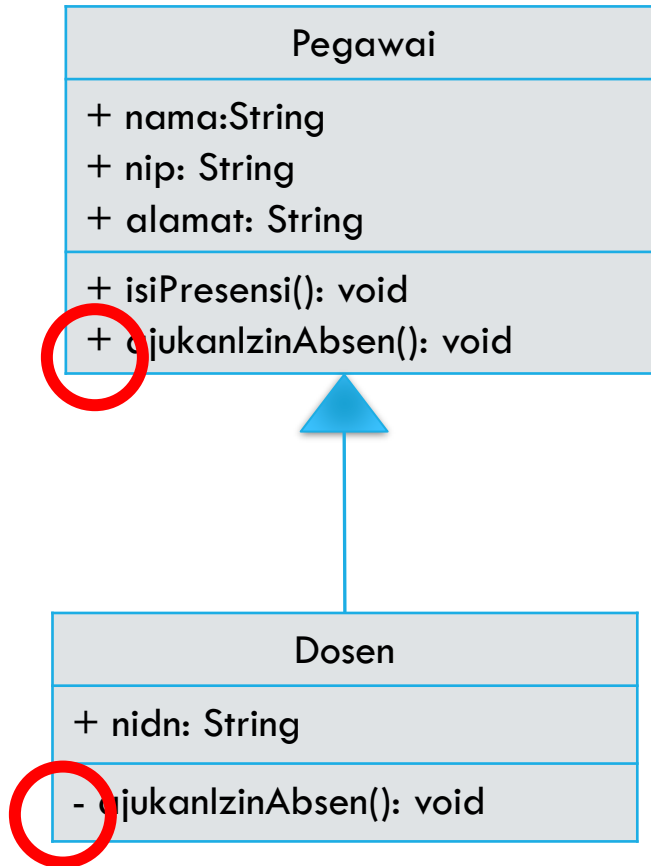


```
public void ajukanIzinAbsen() {  
    System.out.println("Menghubungi Biro Kepegawaian");  
}
```

```
public void ajukanIzinAbsen() {  
    System.out.println("Menghubungi Biro Kepegawaian");  
    System.out.println("Menginfokan kepada ketua kelas");  
}
```

ACCESS LEVEL MODIFIER

Modifier	Class	Package	Subclass	Outside Package
public	v	v	v	v
protected	v	v	v	
no modifier	v	v		
private	v			



Method yang di-override (method di superclass) tidak boleh memiliki access level modifier yang lebih luas daripada method yang meng-override (method di subclass).

Jika aturan tersebut dilanggar, akan muncul compile error: *Cannot reduce the visibility of the inherited method from Pegawai*

WHEN TO USE OVERRIDING

Overriding digunakan jika terdapat suatu kelas yang merupakan turunan dari kelas lain, tetapi implementasi suatu method berbeda

FINAL KEYWORD

Kata kunci **final** merupakan modifier yang digunakan untuk membuat sebuah class, atribut, atau method tidak dapat diubah

- ✓ Final class → tidak dapat diturunkan/di-extend
- ✓ Final atribut → nilainya tidak dapat dimodifikasi
- ✓ Final method → tidak dapat di-override

FINAL CLASS

```
public final class Pegawai {  
    public String nip;  
    public String nama;  
    public double gaji;  
}
```

```
public class Dosen extends Pegawai {  
    public String nidn;  
}
```

Final class → tidak dapat diturunkan/di-extend

The type Dosen cannot subclass the final
class Pegawai

FINAL ATRIBUT

```
public class Pegawai {  
    public final String nip;  
    public String nama;  
    public double gaji;  
  
    public Pegawai(String nip, String nama, double gaji) {  
        this.nip = "jsdfhsd";  
        this.nama = nama;  
        this.gaji = gaji;  
    }  
  
    public void setNIP(String nip) {  
        this.nip = nip;  
    }  
}
```

Final atribut → nilainya tidak dapat dimodifikasi

The final field
Pegawai.nip cannot be
assigned

FINAL ATRIBUT (2)

```
public class Lingkaran {  
    public final double phi = 3.14;  
    public double jariJari;  
  
    public Lingkaran(double jariJari) {  
        this.jariJari = jariJari;  
    }  
}
```

Final atribut → nilainya tidak dapat dimodifikasi

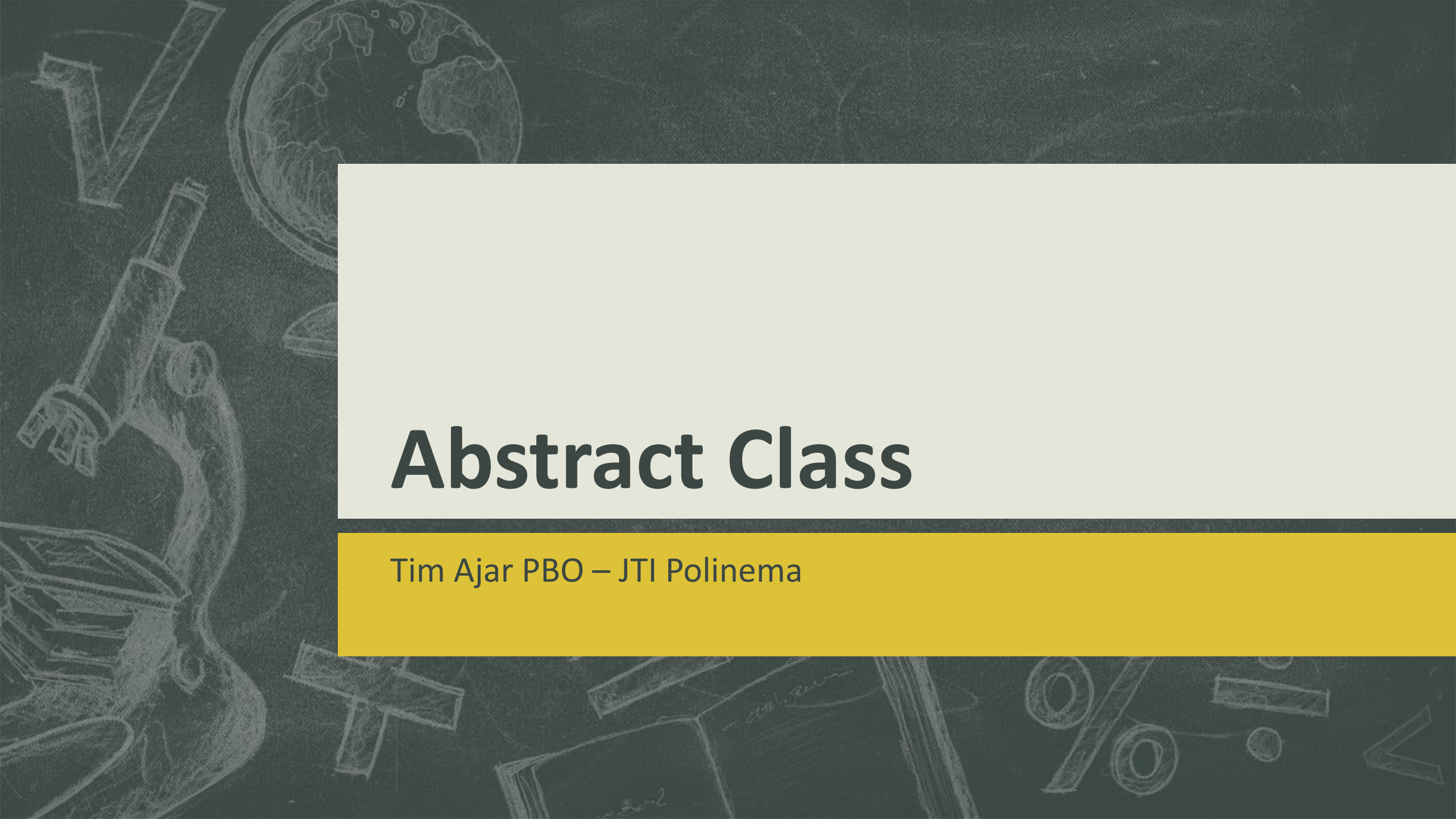
FINAL METHOD

Final method → tidak dapat di-override

Cannot override the final method from Pegawai

```
public class Pegawai {  
    public String nip;  
    public String nama;  
    public double gaji;  
  
    public final void ajukanIzin() {  
        System.out.println("Izin ke biro kepegawaian");  
    }  
}
```

```
public class Dosen extends Pegawai {  
    public String nidn;  
  
    public void ajukanIzin() {  
        System.out.println("Izin ke biro kepegawaian");  
        System.out.println("Menginfokan ketua kelas");  
    }  
}
```



Abstract Class

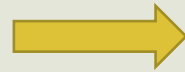
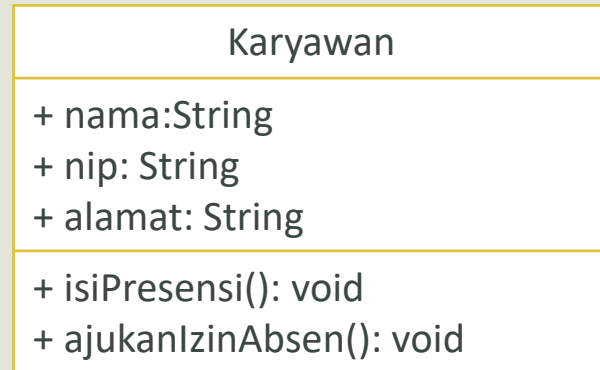
Tim Ajar PBO – JTI Polinema

TUJUAN

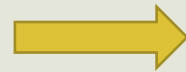
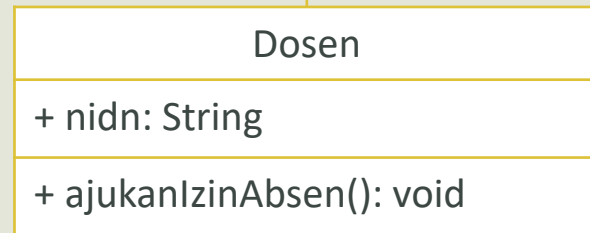
- Mahasiswa mampu memahami konsep abstract class
- Mahasiswa mampu membuat class diagram untuk studi kasus dengan abstract class

OUTLINE

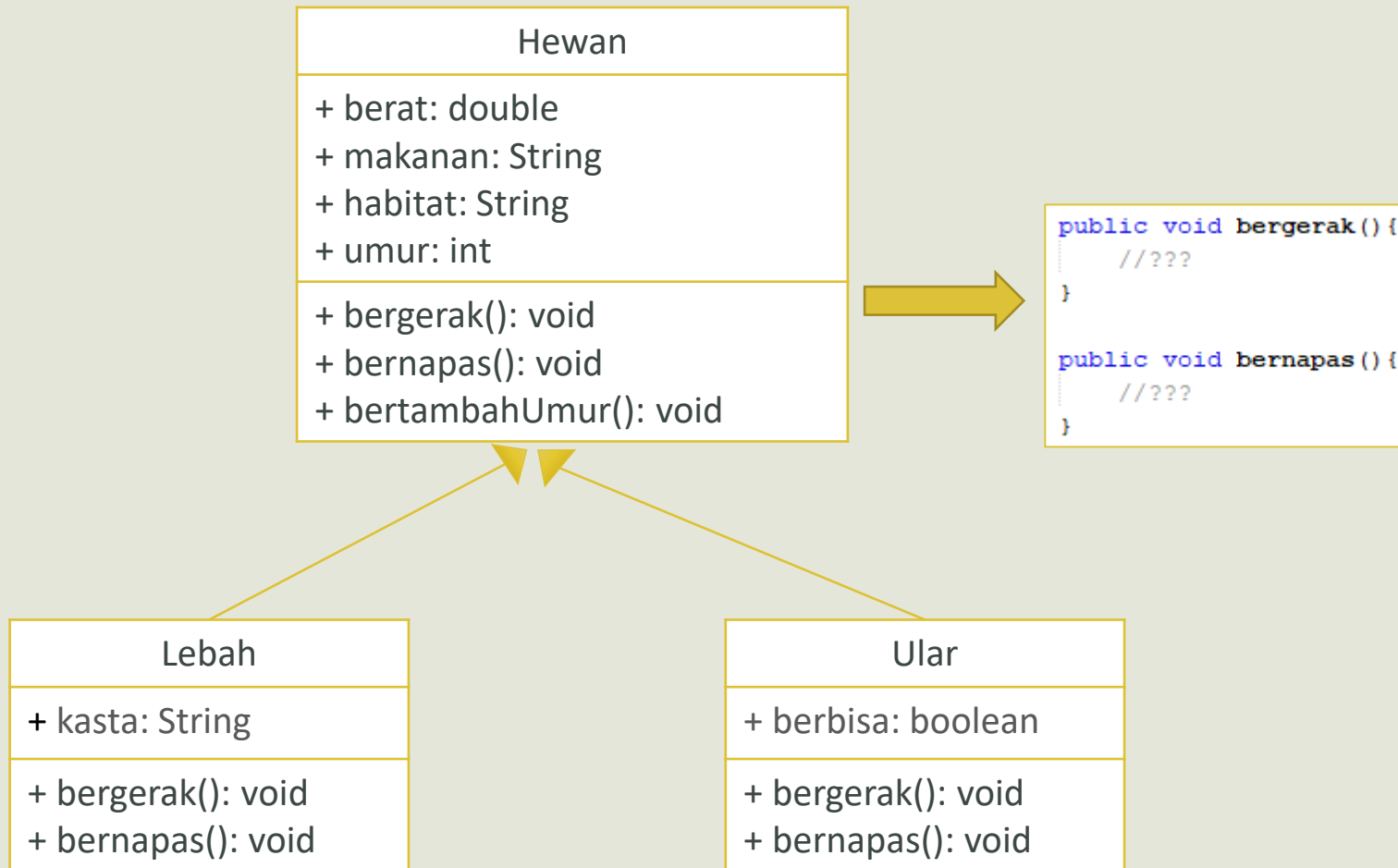
- Studi Kasus
 - Concrete Superclass
 - Abstract Superclass
- Definisi Abstract Class
- Abstract Method
- Syntax
- Notasi dalam Class Diagram
- Penggunaan



```
public void ajukanIzinAbsen() {  
    System.out.println("Menghubungi Biro Kepegawaian");  
}
```



```
public void ajukanIzinAbsen() {  
    System.out.println("Menghubungi Biro Kepegawaian");  
    System.out.println("Menginfokan kepada ketua kelas");  
}
```



Method yang belum dapat diimplementasikan sebaiknya dideklarasikan sebagai ***abstract method***.

Akibatnya class-nya menjadi ***abstract class***

```
public class Ular extends Hewan {  
    public boolean berbisa;  
  
    public void bergerak() {  
        System.out.println("Otot-otot di bagian samping tubuh ular berkontraksi, sehingga tubuh ular menjadi memanjang");  
        System.out.println("tot-otot di bagian samping tubuh ular berelaksasi, sehingga tubuh ular kembali.");  
    }  
  
    public void bernapas() {  
        System.out.println("Melebarkan tulang rusuk untuk menciptakan ruang hampa");  
        System.out.println("Menghirup udara melalui hidung -> tenggorokan -> trakea -> paru-paru");  
        System.out.println("Menyempitkan tulang rusuk, mendorong udara keluar");  
    }  
}
```



```
public class Lebah extends Hewan {  
    public String kasta;  
  
    public void bergerak() {  
        System.out.println("Mengepakkan sayap mereka ke depan dan ke belakang");  
        System.out.println("Tekanan udara di bagian atas lebah lebih rendah dibanding tekanan udara di sekitarnya");  
        System.out.println("Tubuh lebah terangkat ke atas");  
    }  
  
    public void bernapas() {  
        System.out.println("Sebuah katup di setiap spirakel lebah terbuka dan menyedot udara segar");  
        System.out.println("Udara ditransfer melalui lengan trakea ke dalam kantung udara");  
        System.out.println("Kantung udara berkontraksi, memaksa udara melewati trakeol dan masuk ke jaringan lebah");  
        System.out.println("karbon dioksida dipompa kembali melalui trakeol dan keluar melalui spirakel");  
    }  
}
```


Class/Concrete Class

```
public class Hewan {  
    public double berat;  
    public String makanan;  
    public String habitat;  
    public int umur;  
  
    public void bergerak() {  
        //???  
    }  
  
    public void bernapas() {  
        //???  
    }  
  
    public void bertambahUmur() {  
        this.umur += 1;  
    }  
}
```



Abstract Class

```
public abstract class Hewan {  
    public double berat;  
    public String makanan;  
    public String habitat;  
    public int umur;  
  
    public abstract void bergerak();  
    public abstract void bernapas();  
  
    public void bertambahUmur() {  
        this.umur += 1;  
    }  
}
```

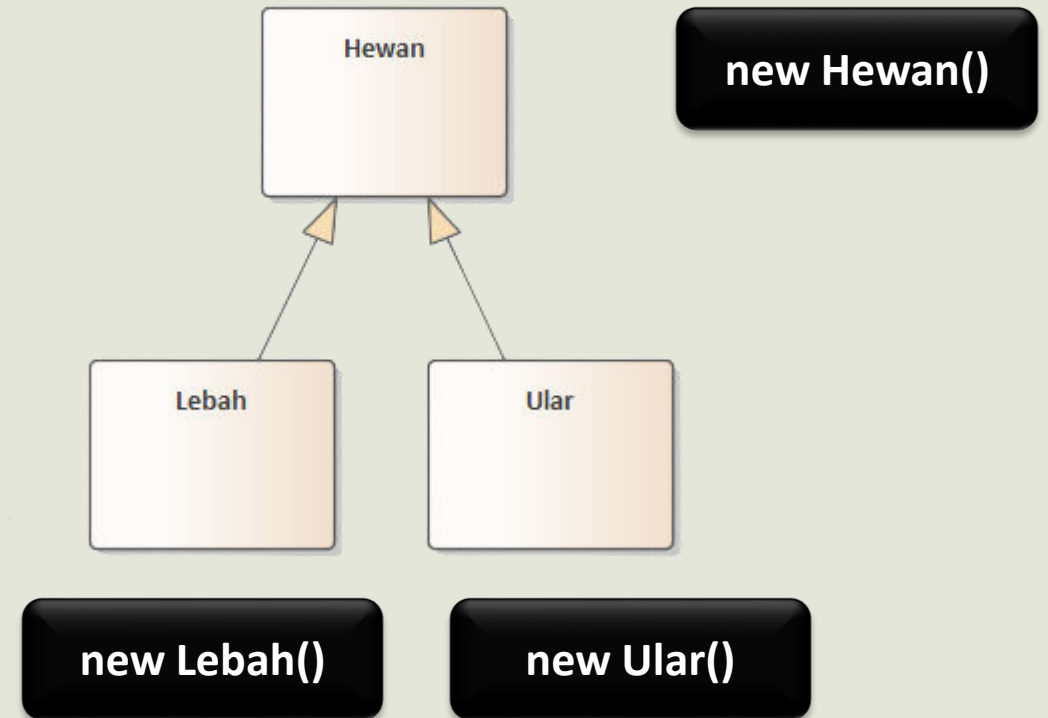
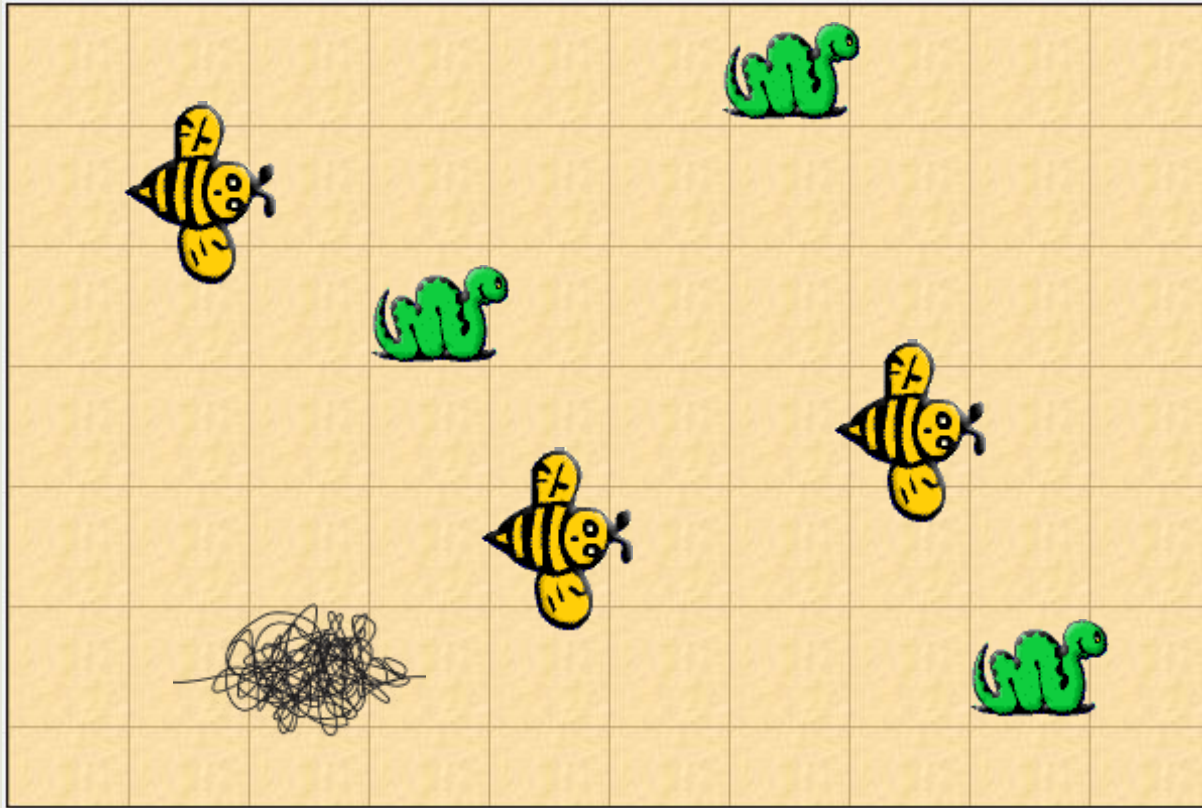


```
public static void main(String[] args) {  
    Hewan h = new Hewan();  
}
```

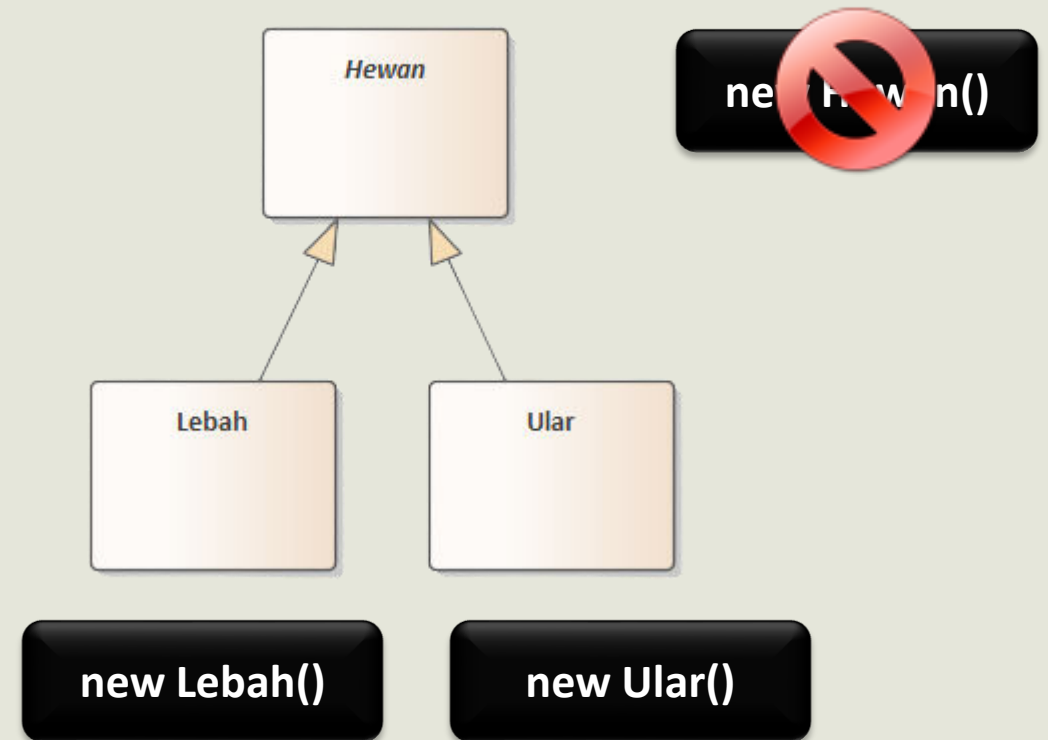
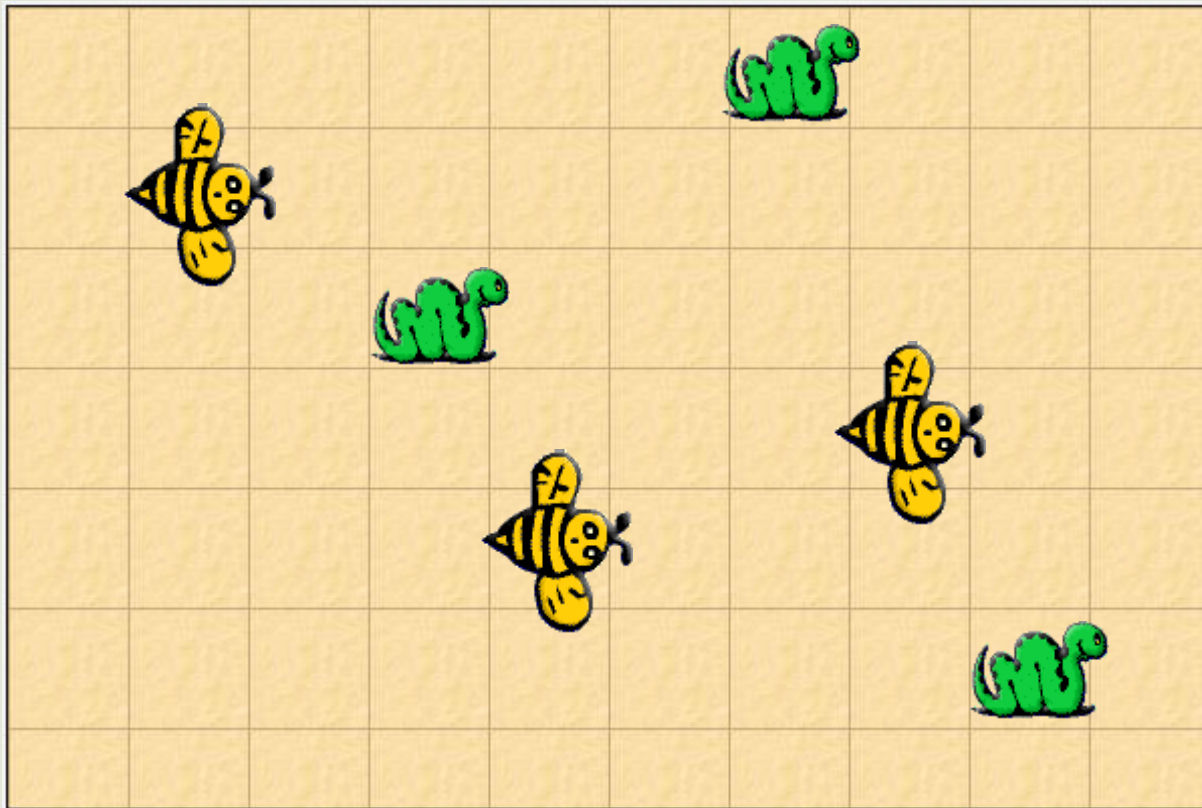


```
public static void main(String[] args) {  
    Hewan h = new Hewan();  
}
```

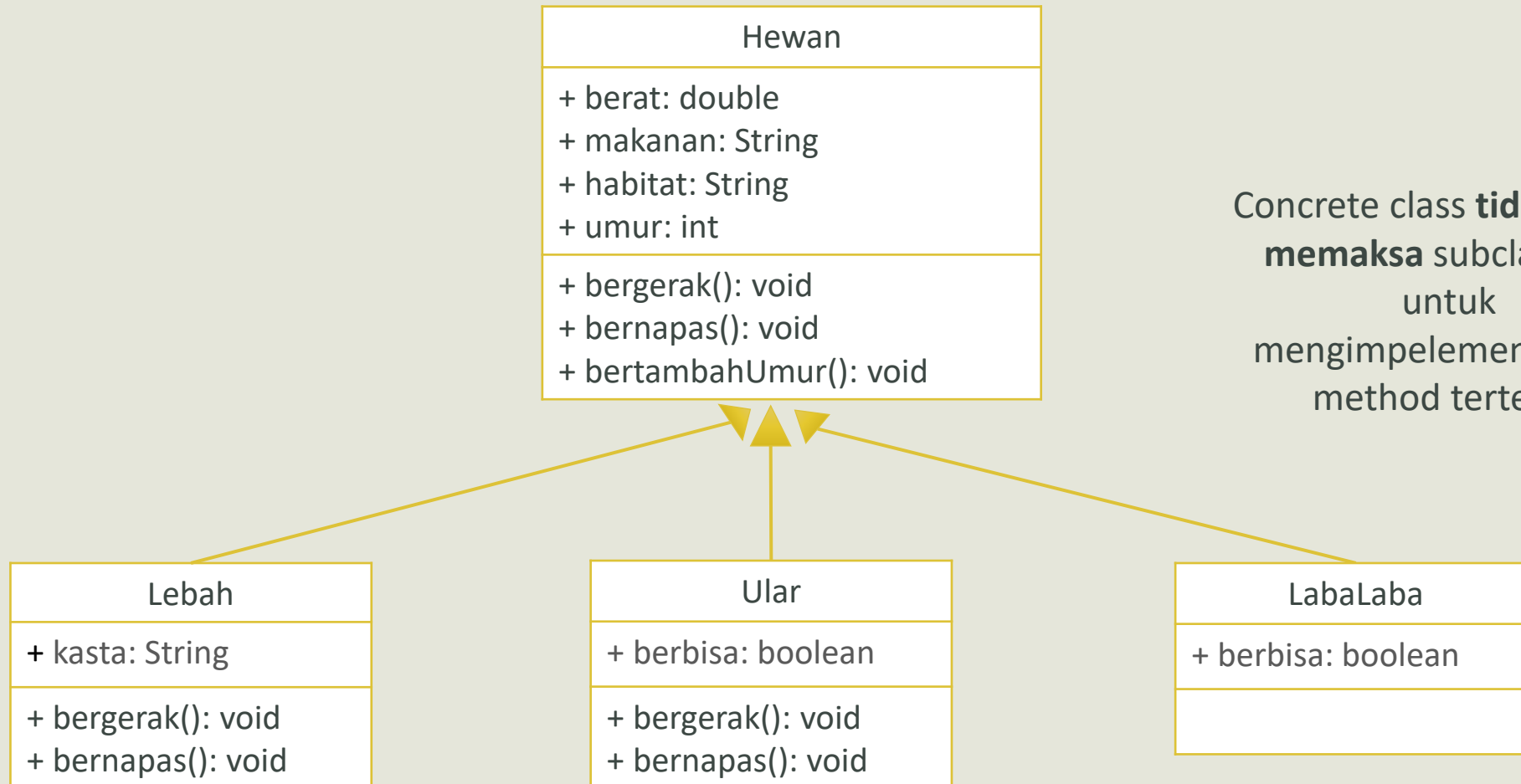
Concrete Superclass



Abstract Superclass



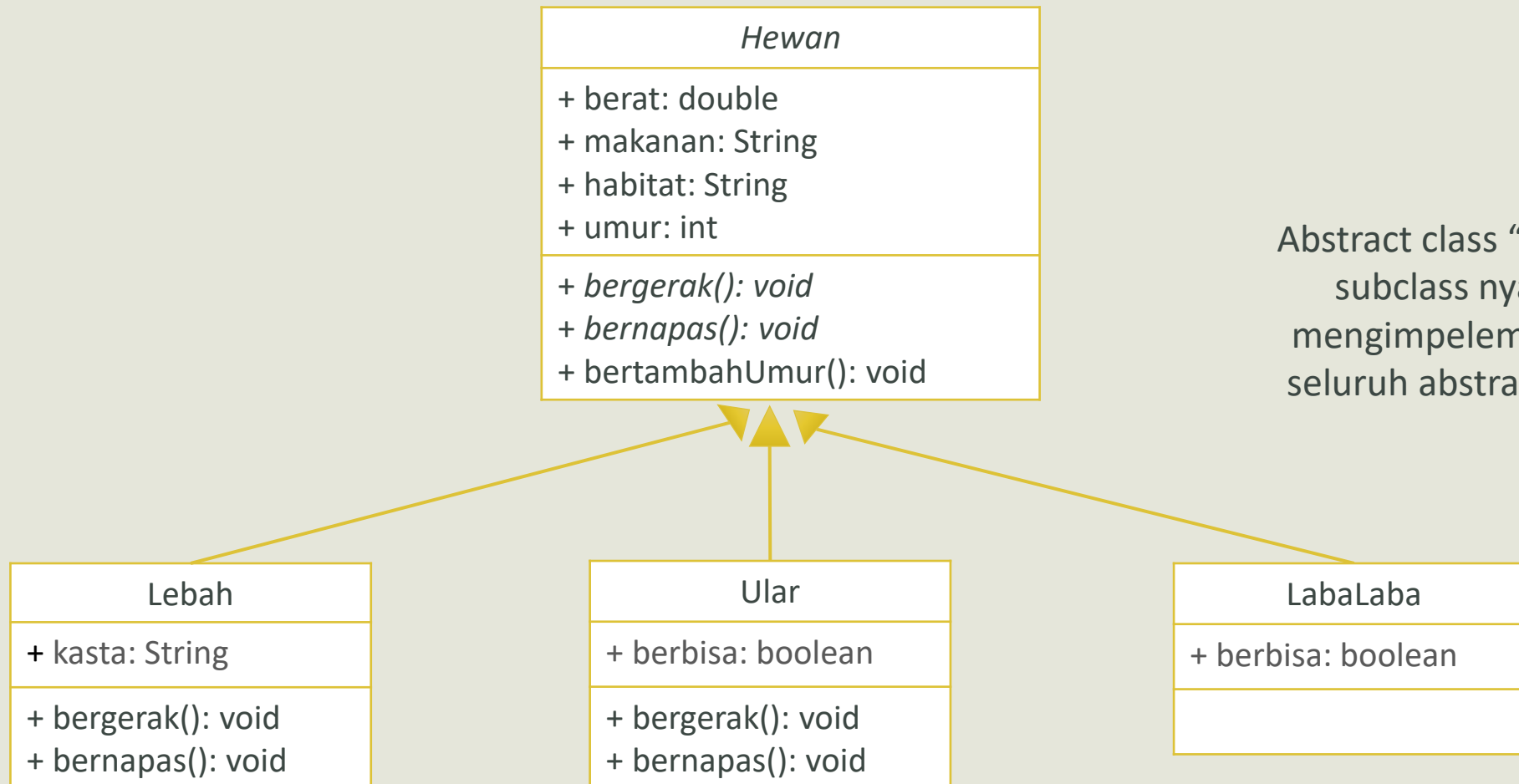
Concrete Superclass



Concrete class **tidak dapat memaksa** subclass nya untuk mengimplementasikan method tertentu



Abstract Superclass



Abstract class “memaksa” subclass nya untuk mengimplementasikan seluruh abstract method



DEFINISI

- *Abstract class* merupakan class yang **tidak dapat diinstansiasi** namun dapat di-***extend***.
- Kegunaan:
 - Sebagai **generalisasi** subclass atau guideline untuk subclass
- Karakteristik:
 - **Selalu** dideklarasikan dengan menggunakan keyword '**abstract class**'
 - **Dapat** memiliki properties dan methods seperti *concrete class*
 - Class yang memiliki abstract method harus dideklarasikan sebagai abstract class. Tetapi suatu class bisa dideklarasikan sebagai abstract tanpa memiliki abstract method

ABSTRACT METHOD

- Method yang hanya dideklarasikan tetapi tidak memiliki implementasi (*body*)
- *Abstract method* dibuat dengan keyword ***abstract***
- Abstract method hanya menjelaskan apa saja yang bisa dilakukan oleh sebuah class namun tidak ada detail bagaimana cara melakukannya

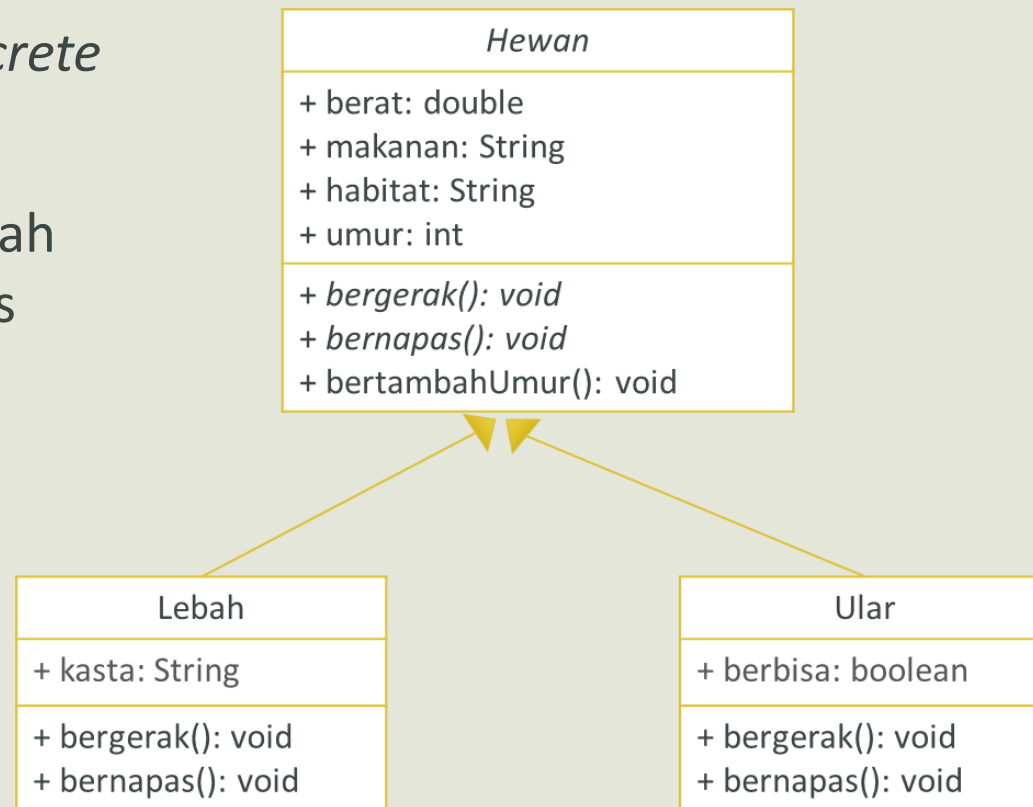
SYNTAX

- Untuk mendeklarasikan abstract class:
 - **<modifier> abstract class** <NamaClass>
- Untuk mendeklarasikan abstract method:
 - **<modifier> abstract <return_type>** <namaMethod>();
- Contoh:

```
public abstract class Hewan {  
    public abstract void bergerak();  
    public abstract void bernapas();  
}
```


NOTASI CLASS DIAGRAM

- Secara umum sama dengan notasi *concrete class*
- Nama kelas dicetak miring atau ditambah anotasi <<abstract>> di atas nama kelas
- Abstract method juga dicetak miring



Penggunaan Abstract Class

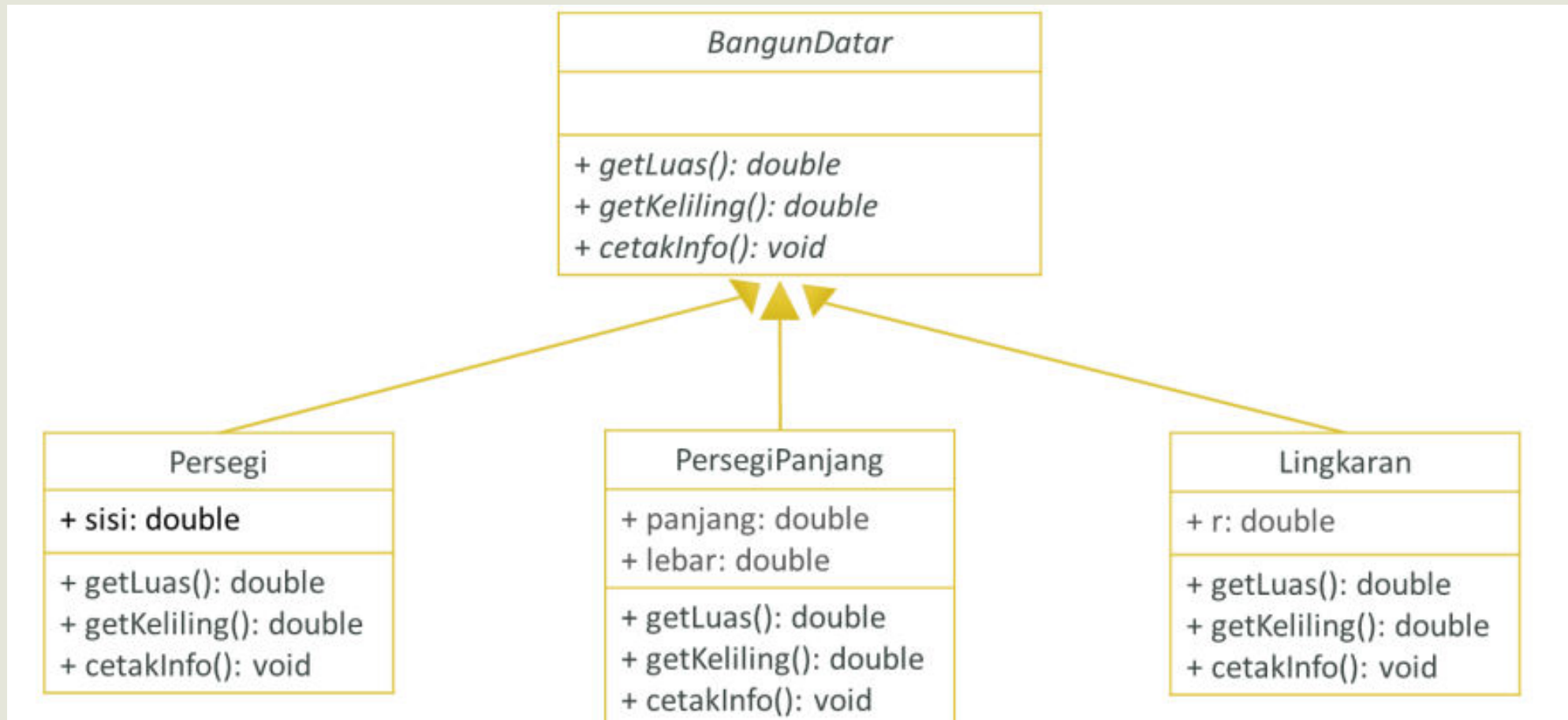
- Abstract class tidak dapat diinstansiasi (tidak dapat dibuat objectnya)
- Baris kode berikut akan memunculkan compilation error “*Hewan is abstract; cannot be instantiated*”

```
Hewan hewan1 = new Hewan();
```

- Untuk menggunakan *abstract class*, dibuat concrete class yang meng-extend *abstract class* tersebut
 - concrete class menggunakan `extends` keyword
 - concrete class harus mengimplementasi semua abstract method
- Class yang menge-extend abstract class tetapi tidak mengimplementasi seluruh abstract method nya maka harus dideklarasikan sebagai abstract class juga

Kesimpulan

- Abstract class dapat digunakan untuk mencegah suatu class diinstansiasi atau dibuat objeknya
- Abstract class umumnya digunakan sebagai generalisasi/superclass pada class hierarki.
- Abstract class berlaku sebagai guideline untuk subclass dengan cara memaksa subclass untuk mengimplementasikan abstract method



Latihan

- Cari sebuah studi kasus dari abstract class kemudian gambarkan UML class diagramnya.
 - Superclass merupakan abstract class dengan minimal 1 concrete method dan 1 abstract method.
 - Terdapat minimal 2 subclass

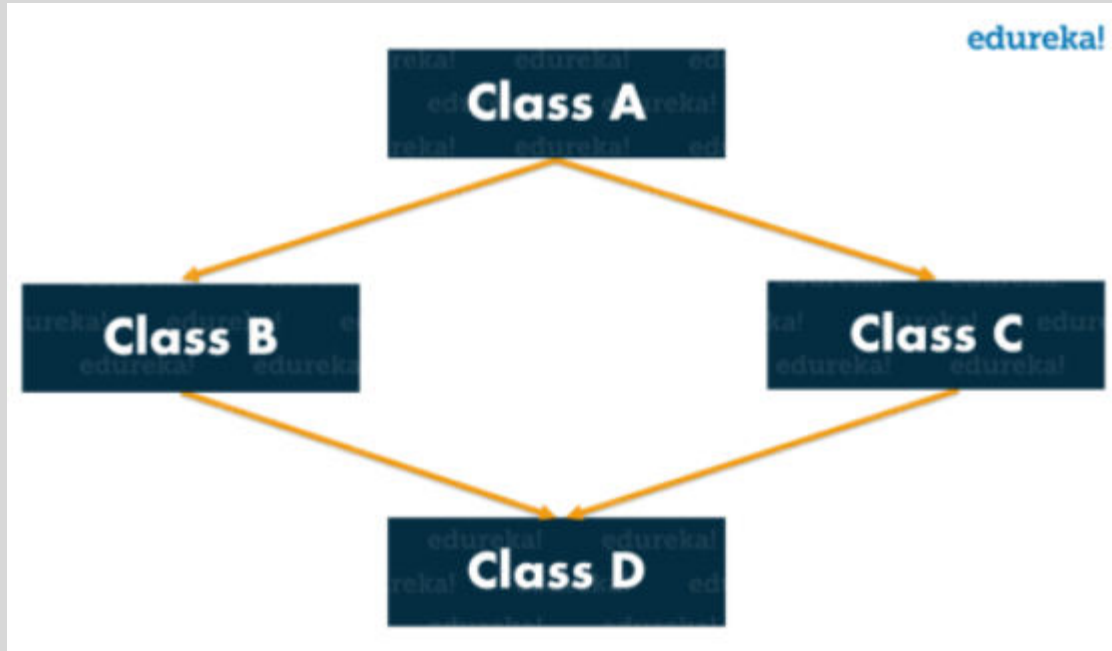
Final & Static

- Final → tidak dapat berubah
- Static → variable yang tidak terikat pada objek tertentu

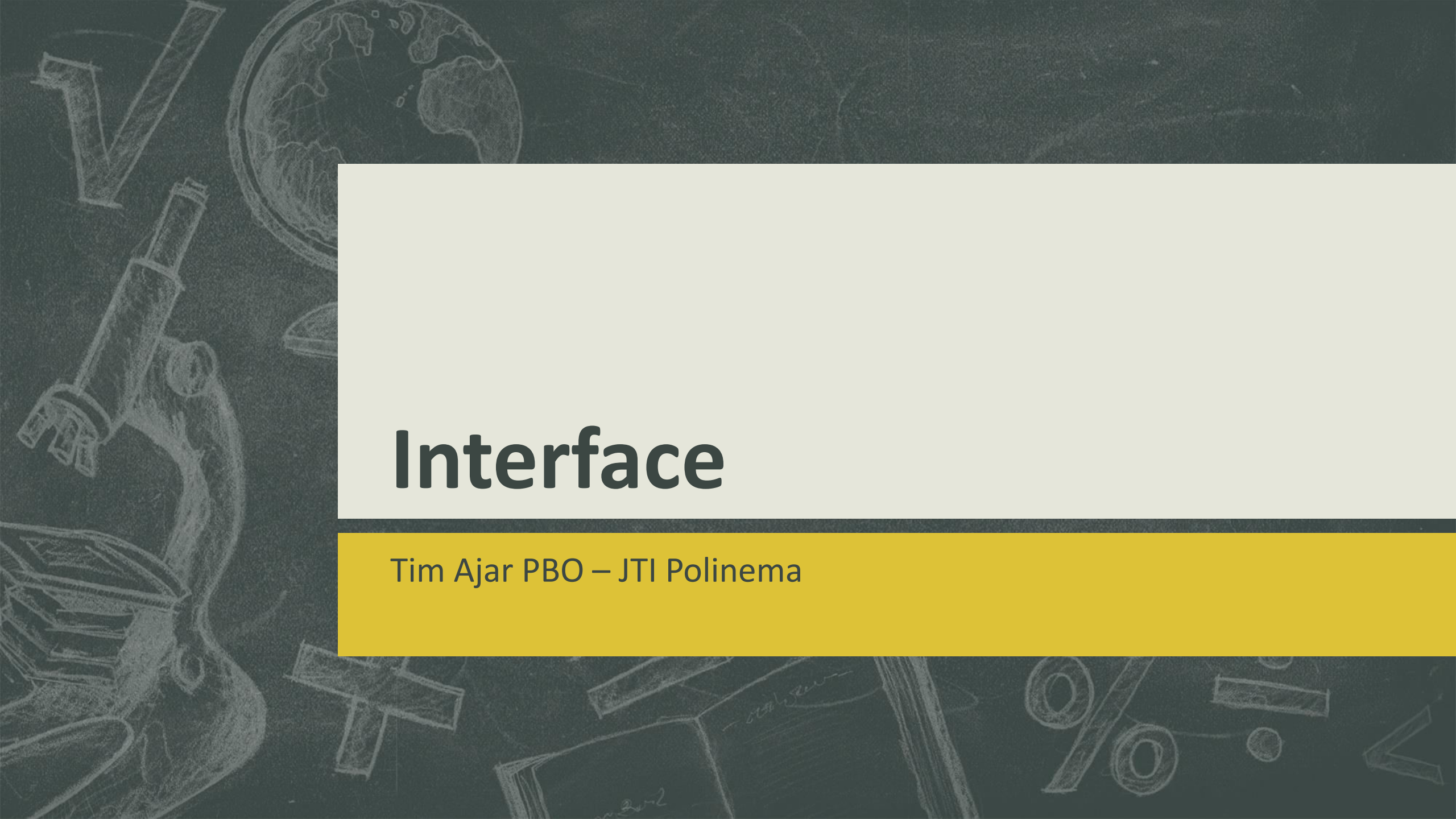
```
public class Lingkaran {  
    public static final double phi = 3.14;  
    public double jariJari;  
  
    public double getLuas() {  
        return phi * jariJari * jariJari;  
    }  
  
    public static void welcome() {  
        System.out.println("Selamat datang...");  
    }  
}
```

```
Lingkaran lingkaran1 = new Lingkaran();  
lingkaran1.jariJari = 10;  
System.out.println(lingkaran1.getLuas());  
  
System.out.println(Lingkaran.phi);  
Lingkaran.welcome();  
Lingkaran.phi = 4; //cannot assign value to a final variable
```

Multiple Inheritance

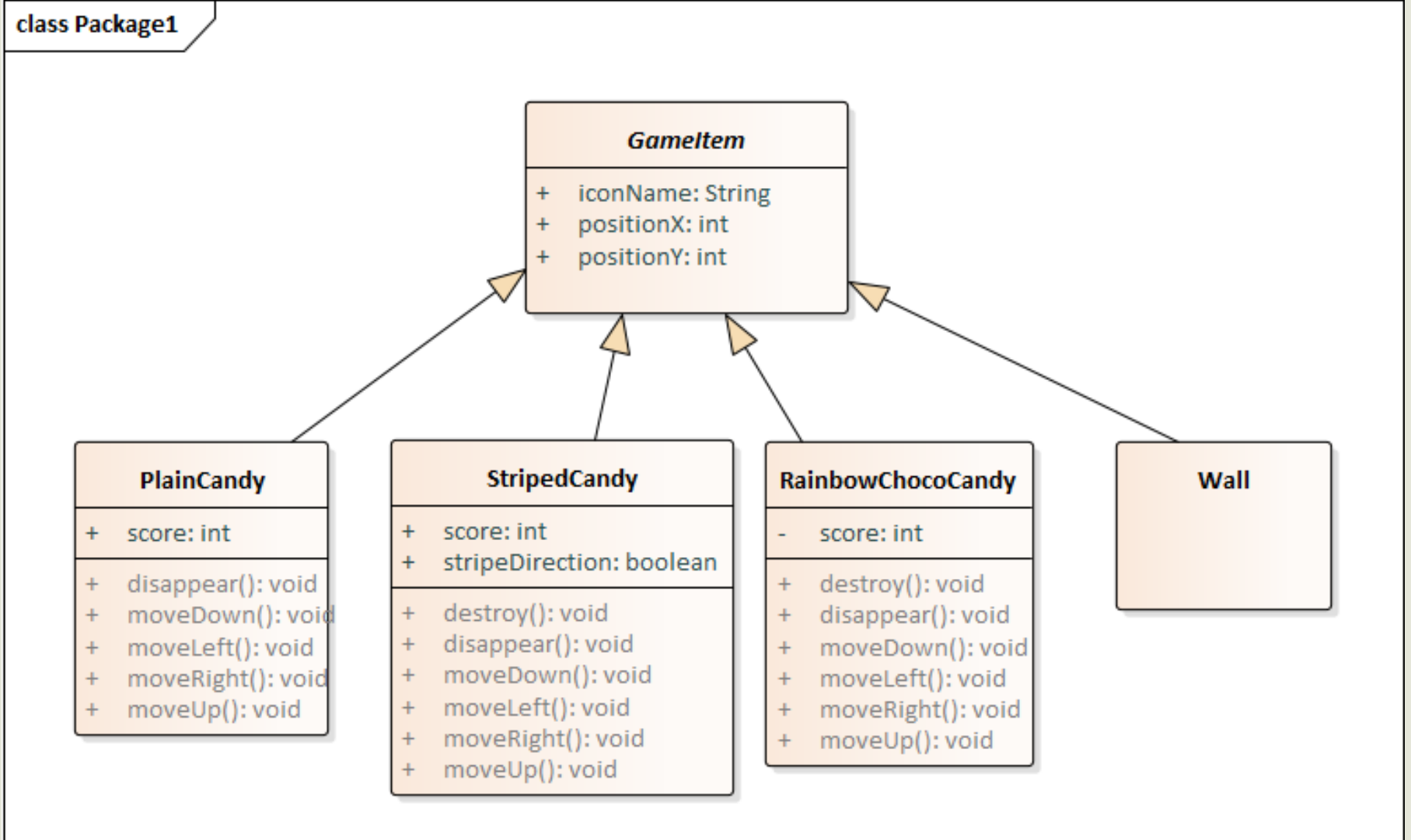


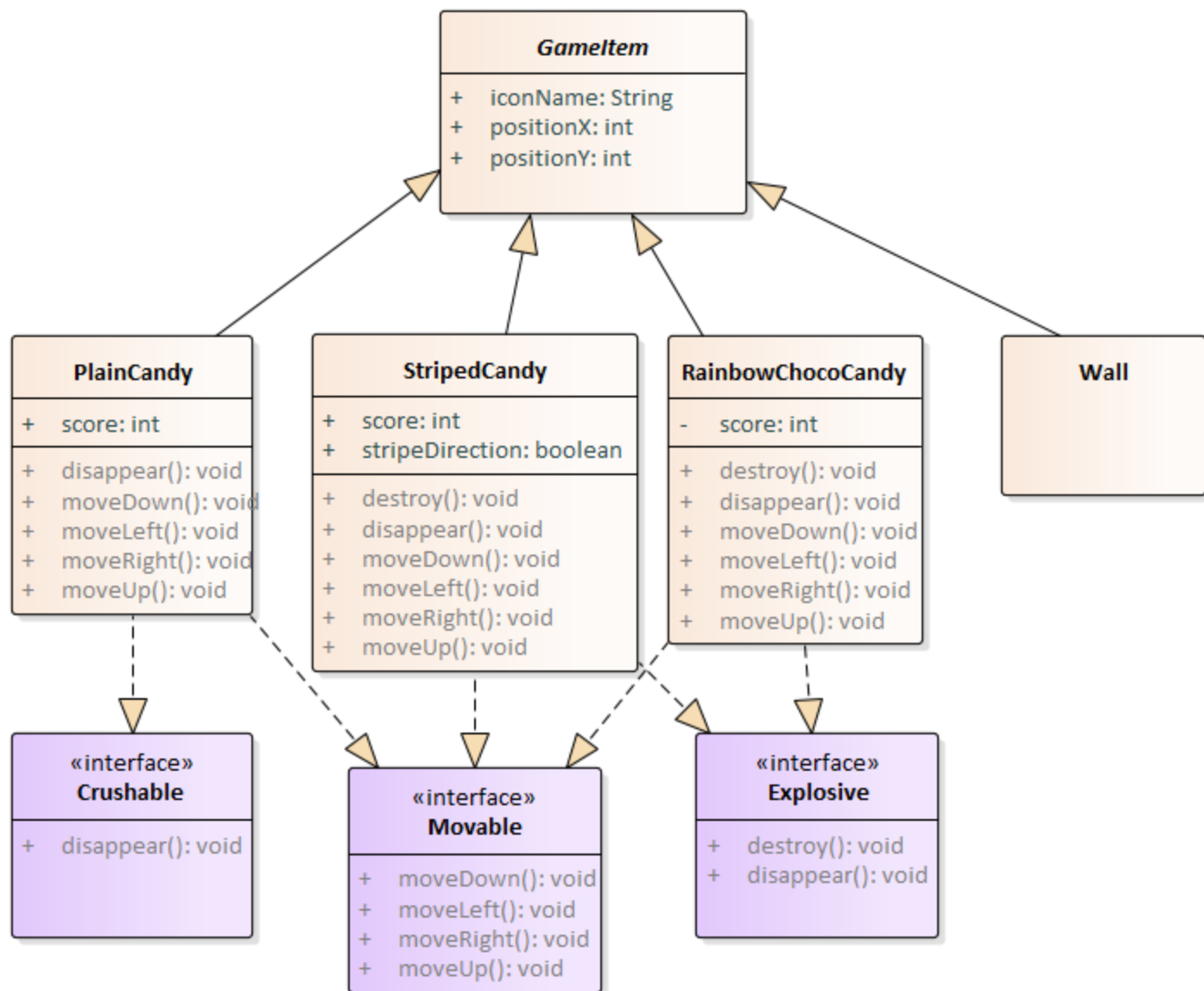
- Multiple inheritance: satu kelas turunan mencoba untuk **memperluas (extend) lebih dari satu kelas induk**.
- Misal terdapat method *show()* pada kelas B dan C dengan fungsi yang berbeda. Kemudian kelas A meng-extend kelas B dan C. Ketika objek dari kelas A mencoba memanggil method *show()*, Java compiler akan bingung method di kelas mana yang akan dieksekusi (dari kelas B atau C)
- Mengarah pada ambiguitas.



Interface

Tim Ajar PBO – JTI Polinema





Definisi Interface

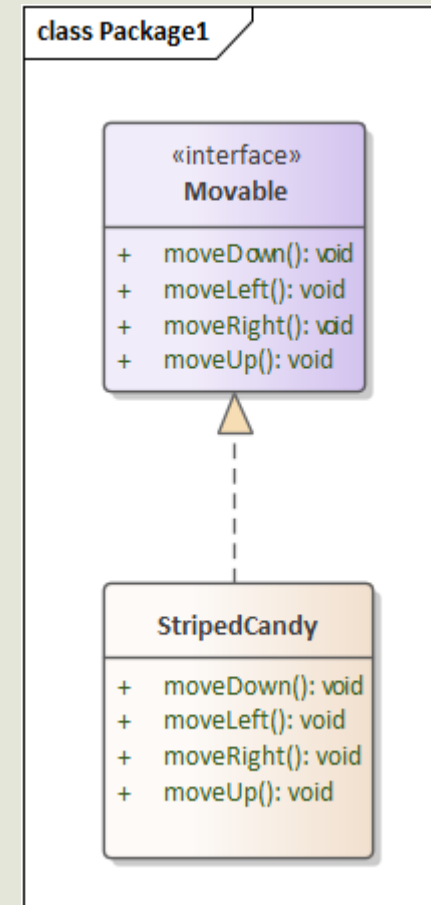
- Interface adalah sekumpulan method tanpa *body* yang saling berkaitan
- Karakteristik:
 - Umumnya hanya terdiri dari *abstract methods*
 - **Selalu** dideklarasikan dengan menggunakan *keyword* '**interface**'
 - Diimplementasikan dengan menggunakan *keyword* '**implements**'
 - Tidak memiliki constructor dan tidak dapat diinstansiasi. Yang dapat diinstansiasi adalah class yang meng-***implement*** interface tersebut
 - Dapat memiliki atribut tetapi hanya bersifat **public static final**

Kegunaan Interface

- Bertindak sebagai **kontrak/syarat** yang berisi sekumpulan *behavior/method* yang saling **terkait** untuk memenuhi suatu **kapabilitas**
- Dengan kata lain, interface memberikan panduan mengenai method apa saja yang perlu diimplementasikan untuk memenuhi kapabilitas tertentu

Notasi Class Diagram

- Nama interface **tidak** dicetak miring
- Keterangan <<interface>> di atas nama interface
- Nama method dicetak miring atau tidak
- **Implements** dilambangkan dengan garis panah putus-putus



Aturan Penulisan Interface

- Secara struktur hampir sama dengan class.
- Beberapa aturan:
 - **Tidak** memiliki concrete method (method biasa yang bukan abstract)
 - **Tidak** memiliki constructor
 - Dapat memiliki atribut, tapi hanya dapat bersifat **public, static, dan final**

Sintaks Interface

- Untuk mendeklarasikan suatu interface:
 - **public interface** <NamaInterface>
- Untuk mengimplementasikan interface:
 - **public class** <NamaClass> **implements** <NamaInterface>
- Nama interface sebaiknya dalam bentuk **adjective/kata sifat** jika merepresentasikan **kapabilitas**. Dapat juga menggunakan **kata benda**
- Contoh:

```
public interface Movable {  
    void moveLeft();  
    void moveRight();  
    void moveUp();  
    void moveDown();  
}
```

```
public class PlainCandy extends GameItem implements Movable{  
    @Override  
    public void moveLeft() {}  
    @Override  
    public void moveRight() {}  
    @Override  
    public void moveUp() {}  
    @Override  
    public void moveDown() {}  
}
```

Implementasi Interface

- Bila sebuah class mengimplementasikan suatu interface:
 - **Seluruh variable** dari interface akan dapat diakses oleh class tersebut
 - **Seluruh method** pada interface **harus diimplementasikan**
 - Bila class yang meng-implement interface **tidak mengimplementasikan semua method**, maka class tersebut harus dideklarasikan sebagai **abstract class**

Multiple Interface

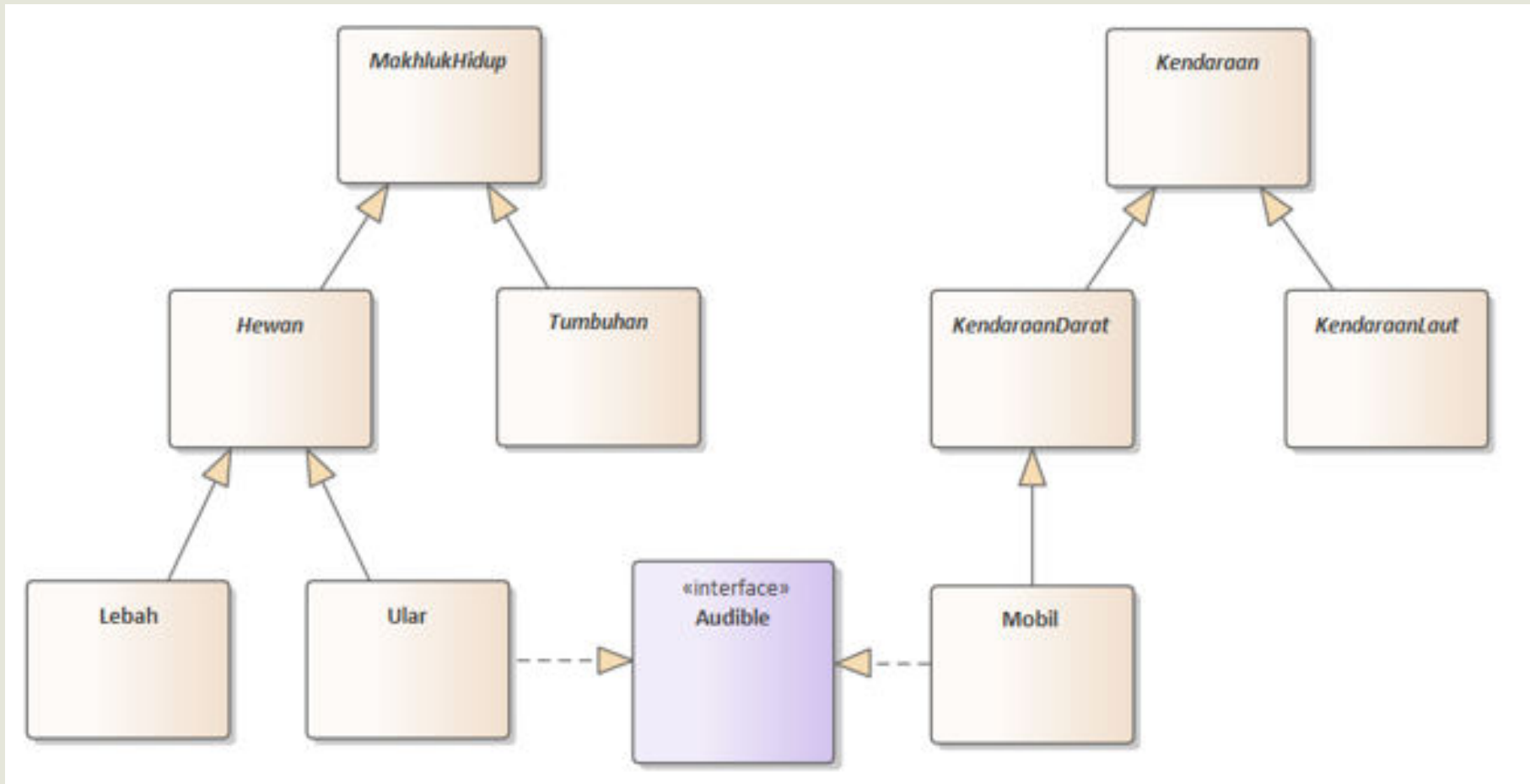
- Suatu class dapat meng-implement multiple interface
- Bila suatu class merupakan subclass dan meng-implement interface, maka keyword **extends** mendahului **implements**
- Contoh:

```
public class PlainCandy extends GameItem implements Crushable, Movable
```

Abstract Class vs Interface

Abstract Class	Interface
Dapat memiliki concrete method atau abstract method	Hanya dapat memiliki abstract method
Level modifier atribut dan method: public, protected, no-modifier, private	Level modifier variable dan method hanya public (boleh tidak dituliskan)
Dapat memiliki static/non-static, final/non final variable	Hanya dapat memiliki static dan final variable
Method boleh bersifat static/non-static dan final/non final	Method tidak boleh bersifat static dan final
Abstract class harus terdapat dalam hirarki yang sama dengan class yang meng-extend	Interface tidak terkait pada suatu hirarki

Interface tidak terikat pada hirarki



Penggunaan Abstract Class vs Interface

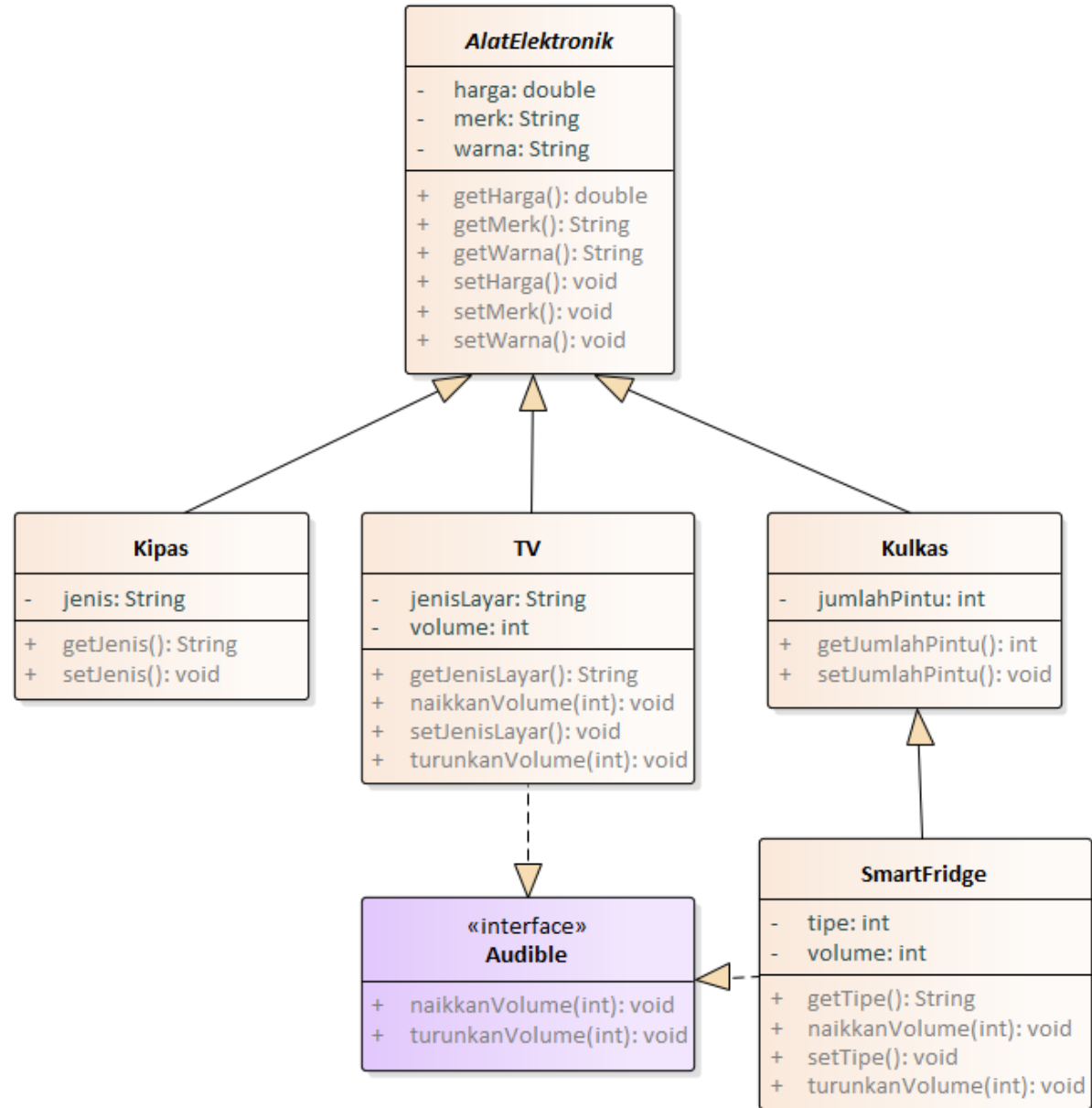
- Common properties (and methods) → abstract class
- Common methods → interface

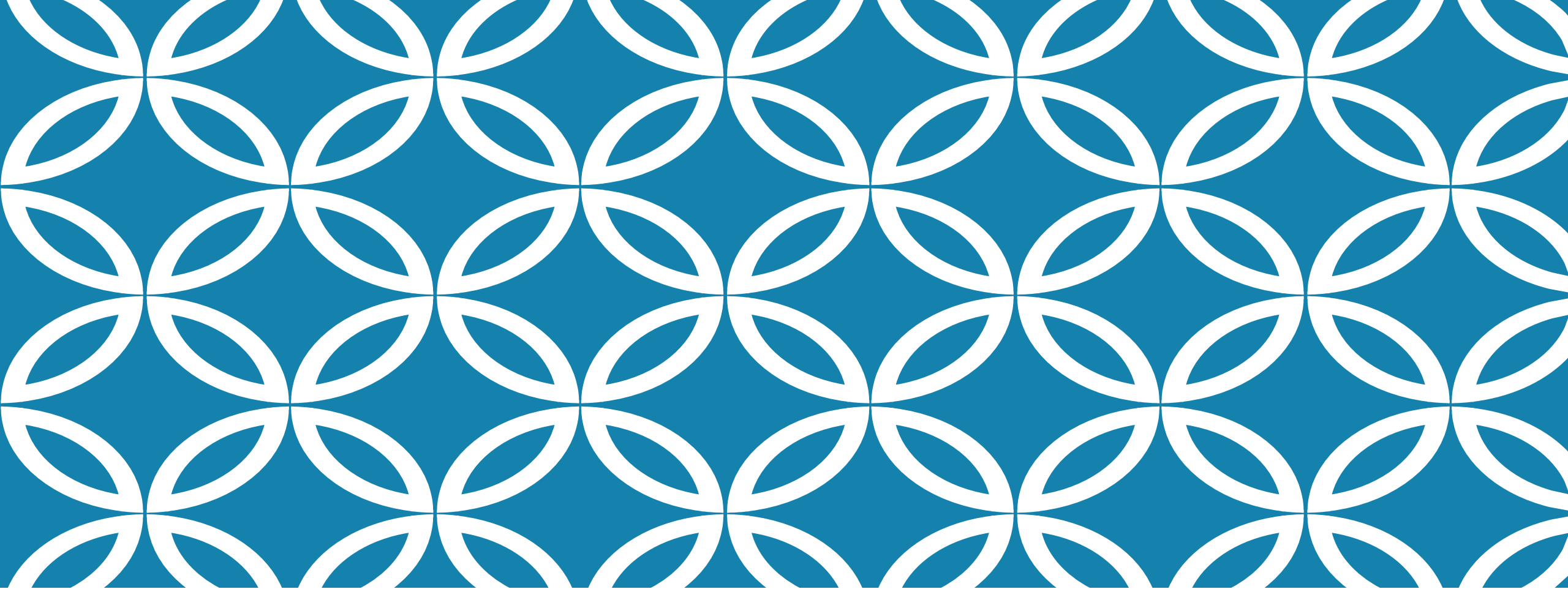
Kesimpulan

- Interface tidak dapat dibuat objek/*instance*-nya
- Interface merepresentasikan **kontrak/syarat** yang berisi sekumpulan *behavior* yang saling **terkait** untuk memenuhi suatu **kapabilitas**

Latihan

- Cari sebuah studi kasus dari interface kemudian gambarkan UML class diagramnya.





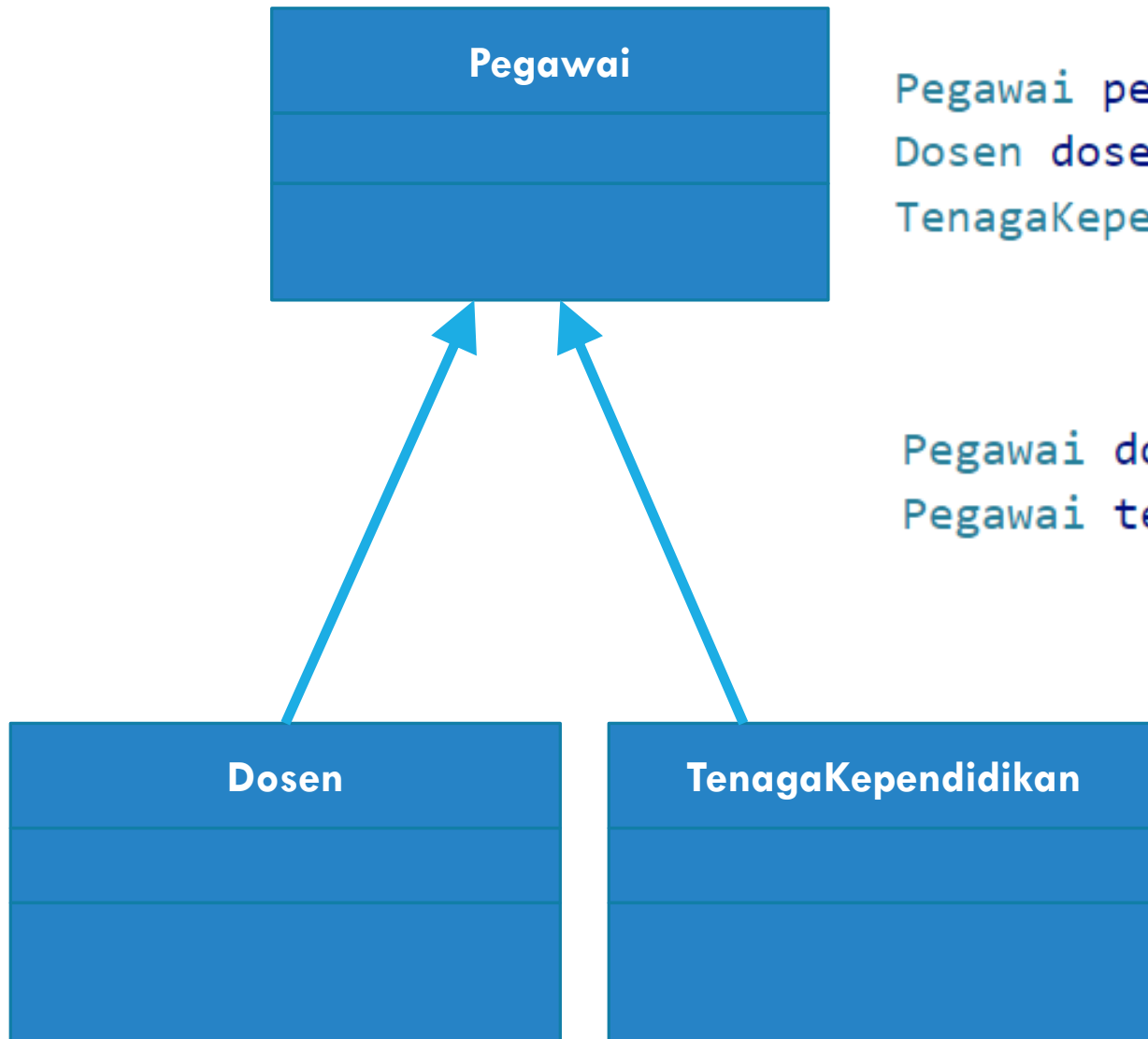
POLYMORPHISM

POLYMORPHISM

1. Heterogenous Collection
2. Object Casting
3. Polymorphic Arguments
4. InstanceOf

POLYMORPHISM

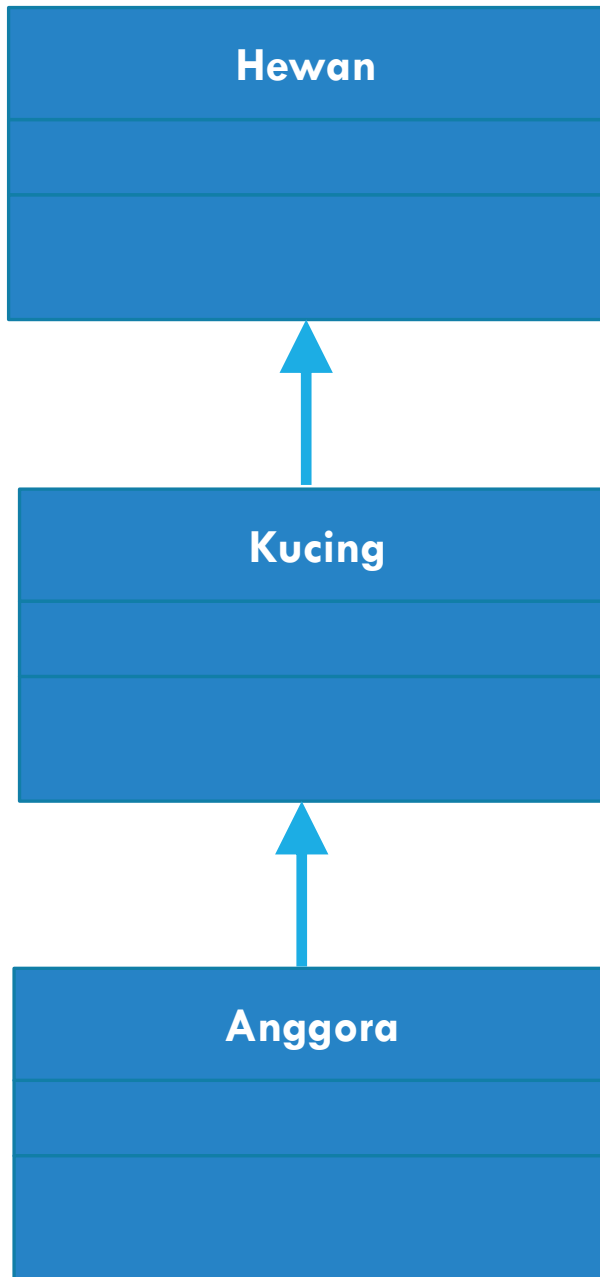
- Polymorphism: poly (banyak), morph (bentuk)
- Polimorfisme → konsep dalam OOP yang memperbolehkan sebuah aksi diimplementasikan secara berbeda
- Polimorfisme → konsep dalam OOP yang memperbolehkan sebuah objek untuk memiliki banyak bentuk



```
Pegawai pegawai1 = new Pegawai();
Dosen dosen1 = new Dosen();
TenagaKependidikan tendik1 = new TenagaKependidikan();
```

```
Pegawai dosen1 = new Dosen();
Pegawai tendik1 = new TenagaKependidikan();
```

dosen1 merupakan instance
dari class Dosen tapi dikenali
sebagai Pegawai



```
Anggora anggora1 = new Anggora();  
Kucing anggora2 = new Anggora();  
Hewan anggora3 = new Anggora();
```

CLASS *OBJECT*

Class `Object` merupakan root dari semua hirarki class. Artinya, semua object memiliki superclass dengan nama `Object`. Oleh karena itu, semua instance dapat dikenali sebagai `Object`

```
Object kucing1 = new Kucing();  
Object dosen1 = new Dosen();
```

HETEROGENOUS COLLECTION

- Dalam Java, setiap elemen dalam array harus memiliki tipe yang sama
- Konsep polimorfisme memungkinkan elemen dalam array bersifat heterogen

```
ArrayList<Pegawai> daftarPegawai = new ArrayList<Pegawai>();  
daftarPegawai.add(new Dosen());  
daftarPegawai.add(new TenagaKependidikan());  
daftarPegawai.add(new Dosen());
```

```
Ular ular1 = new Ular();  
Lebah lebah1 = new Lebah();  
Kucing kucing1 = new Kucing();  
Hewan[] daftarHewan = {ular1, lebah1, kucing1};
```

```
Object[] objects = {dosen1, ular1, tendik1, lebah1};
```

OBJECT CASTING

- Typecasting merupakan konversi variable dari suatu tipe data menjadi tipe data lainnya
- Typecasting juga dapat dilakukan terhadap object
- Terdapat 2 jenis object casting: **upcasting & downcasting**

UPCASTING

- Dilakukan untuk mengubah child object menjadi parent object
- Dapat dilakukan secara implisit

```
Pegawai dosen1 = new Dosen();  
Pegawai dosen2 = (Pegawai) new Dosen(); ← eksplisit  
  
TenagaKependidikan tendik1 = new TenagaKependidikan();  
Pegawai pegawai = tendik1;
```

- dosen1 merupakan instance dari class Dosen, tetapi setelah proses upcasting akan dikenali sebagai object bertipe Pegawai
- tendik1 merupakan instance dari class TenagaKependidikan, tetapi setelah upcasting akan dikenali sebagai object bertipe Pegawai

DOWNCASTING

- Dilakukan untuk mengubah parent object menjadi child object
- Tidak dapat dilakukan secara implisit

```
Hewan hewan1 = new Ular();
```

```
Hewan hewan2 = new Lebah();
```

```
Ular ular1 = (Ular) hewan1;
```

```
Lebah lebah1 = (Lebah) hewan2;
```

POLYMORPHIC ARGUMENTS

- Konsep polimorfisme juga memungkinkan **parameter** dari suatu method menerima **argument dengan berbagai bentuk object**
- Misalkan, pada class UnitKerja terdapat method train() dengan parameter pegawai bertipe Pegawai. Method ini akan dapat menerima argument bertipe Pegawai atau class lain yang merupakan subclass dari Pegawai
- Proses ini juga termasuk upcasting

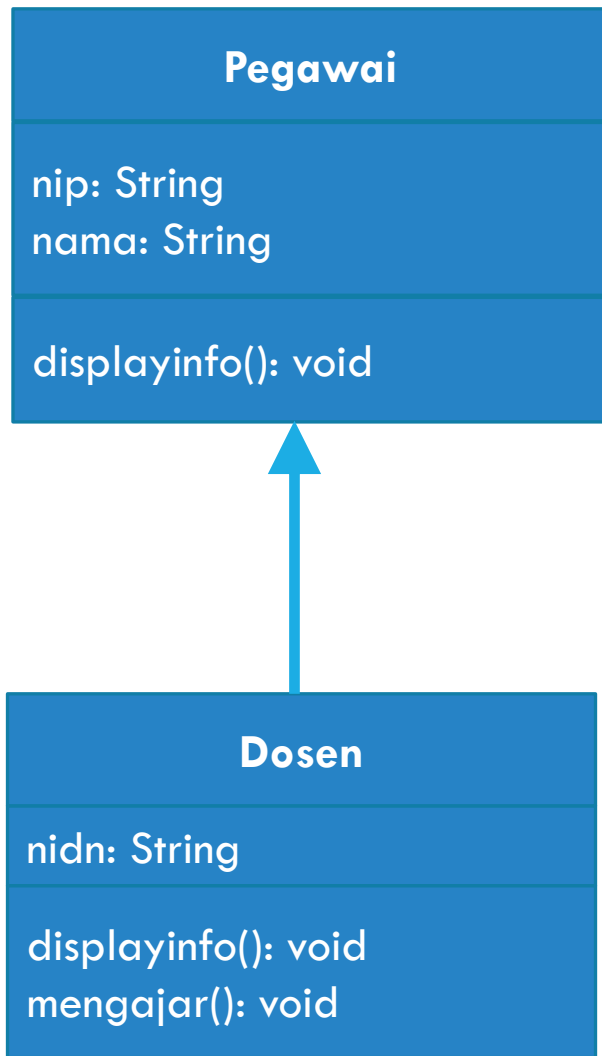
```
public class UnitKerja {  
    public void train(Pegawai pegawai){  
        System.out.println("Memberikan pelatihan untuk " + pegawai.nama);  
    }  
}
```

```
Dosen dosen1 = new Dosen();
TenagaKependidikan tendik1 = new TenagaKependidikan();

UnitKerja unitKerja1 = new UnitKerja();
unitKerja1.train(dosen1);
unitKerja1.train(tendik1);
```

Method train() dapat dipanggil dengan argument dosen1 bertipe **Dosen** maupun tendik1 bertipe **TenagaKependidikan** karena dilakukan **upcasting** ke class **Pegawai**

```
public class UnitKerja {
    public void train(Pegawai pegawai){
        System.out.println("Memberikan pelatihan untuk " + pegawai.nama);
    }
}
```



```
Pegawai dosen1 = new Dosen();
```

- `dosen1` merupakan instance dari class `Dosen`, tetapi setelah proses upcasting akan **dikenali sebagai object bertipe Pegawai**
- Oleh karna itu, `dosen1` tidak akan dapat mengakses atribut yang dideklarasikan di class `Dosen` (misalnya `nidn`) maupun memanggil method yang hanya ada di class `Dosen` (misalnya `mengajar()`)
- Method `displayInfo()` dapat dikenali karena ada di class `Pegawai`, tapi karena terdapat overriding maka yang akan dieksekusi adalah method `displayInfo()` pada class `Dosen`

```
Dosen dosen1 = new Dosen();
TenagaKependidikan tendik1 = new TenagaKependidikan();

UnitKerja unitKerja1 = new UnitKerja();
unitKerja1.train(dosen1);
unitKerja1.train(tendik1);
```

```
public class UnitKerja {
    public void train(Pegawai pegawai){
        System.out.println("Memberikan pelatihan untuk " + pegawai.nama);

        //test
        System.out.println(pegawai.nidn);           //tidak bisa mengakses atribut nidn
        pegawai.mengajar();                          //tidak bisa memanggil method mengajar()
        pegawai.displayInfo();                       //memanggil method displayInfo() nya class asli
    }
}
```

- Untuk dapat mengakses atribut `nidn` dan method `mengajar()`, lakukan downcasting

```
public class UnitKerja {  
    public void train(Pegawai pegawai){  
        System.out.println("Memberikan pelatihan untuk " + pegawai.nama);  
        Dosen dosen = (Dosen) pegawai;  
  
        //test  
        System.out.println(dosen.nidn);  
        dosen.mengajar();  
        dosen.displayInfo();  
    }  
}
```

Java.Lang.ClassCastException

```
unitKerja1.train(dosen1);  
unitKerja1.train(tendik1);    //runtime error
```

```
public class UnitKerja {  
    public void train(Pegawai pegawai){  
        System.out.println("Memberikan pelatihan untuk " + pegawai.nama);  
        Dosen dosen = (Dosen) pegawai;  
  
        //test  
        System.out.println(dosen.nidn);  
        dosen.mengajar();  
        dosen.displayInfo();  
    }  
}
```

INSTANCEOF

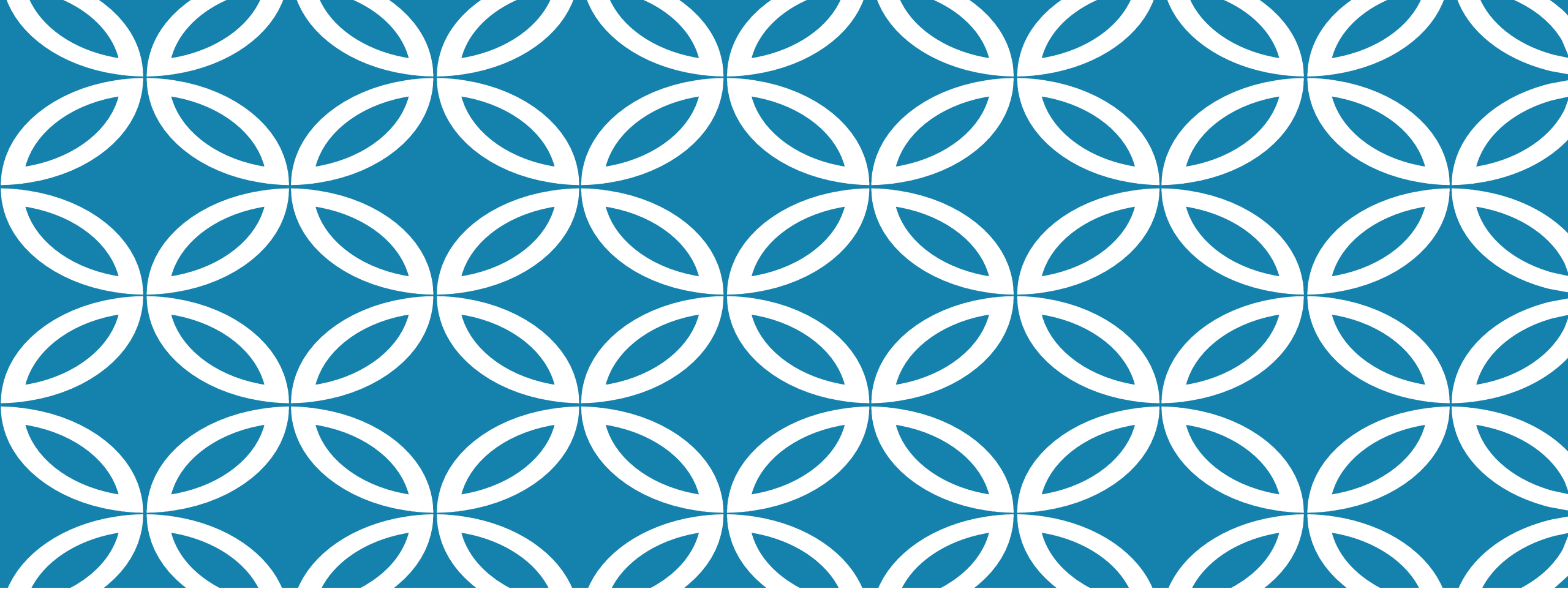
- Keyword `instanceOf` digunakan untuk mengidentifikasi apakah sebuah object merupakan instance dari suatu class/interface

```
Hewan hewan1 = new Ular();  
Hewan hewan2 = new Lebah();
```

```
System.out.println(hewan1 instanceof Ular);    //true  
System.out.println(hewan2 instanceof Ular);    //false
```



```
public class UnitKerja {  
    public void train(Pegawai pegawai){  
        System.out.println("Mengenalakan lingkungan kampus");  
        System.out.println("Menginfokan SOP/Juknis");  
  
        if (pegawai instanceof Dosen) {  
            System.out.println("Memberikan pelatihan pedagogik");  
        }  
    }  
}
```



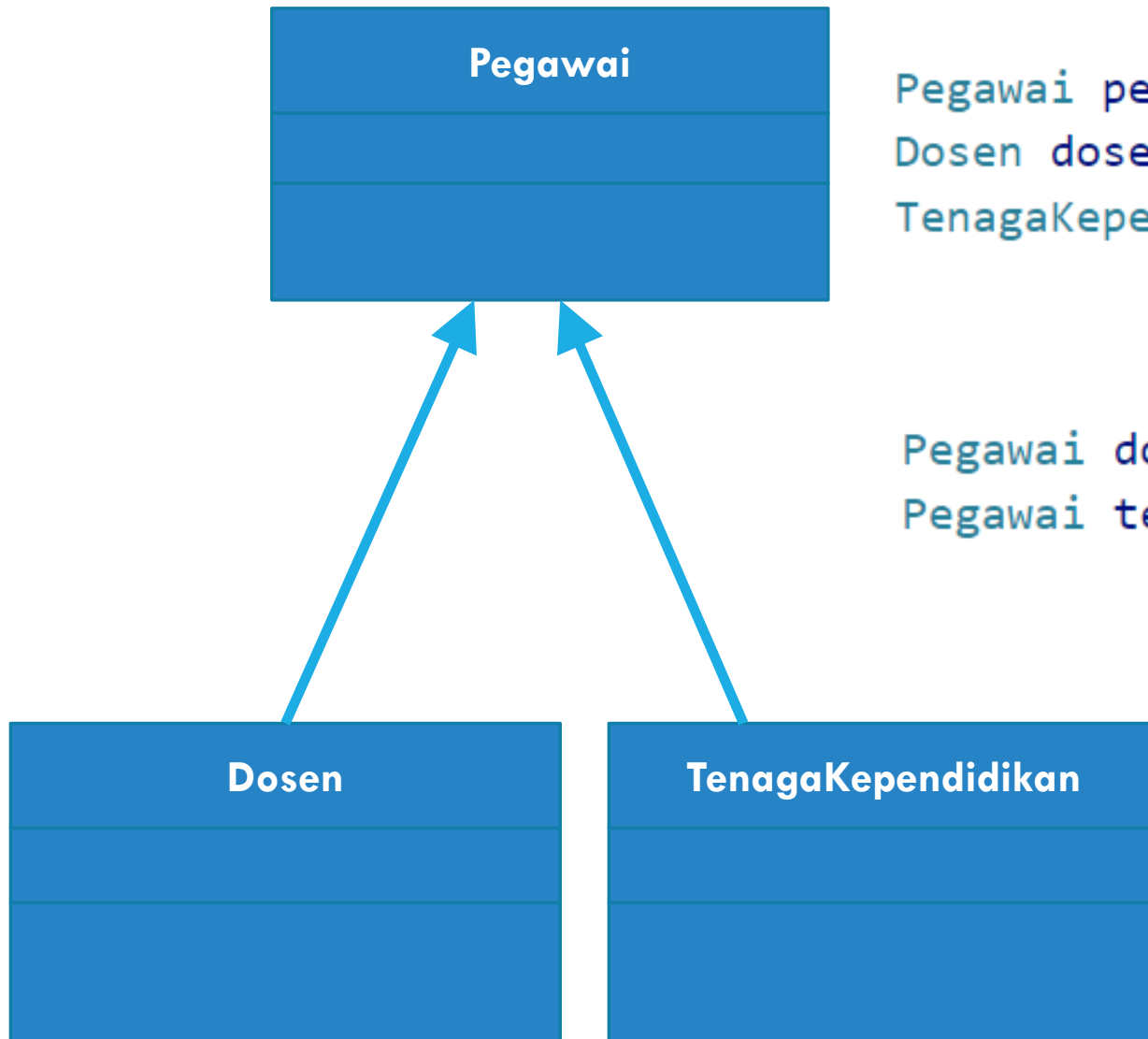
POLYMORPHISM

POLYMORPHISM

1. Heterogenous Collection
2. Object Casting
3. Polymorphic Arguments
4. InstanceOf

POLYMORPHISM

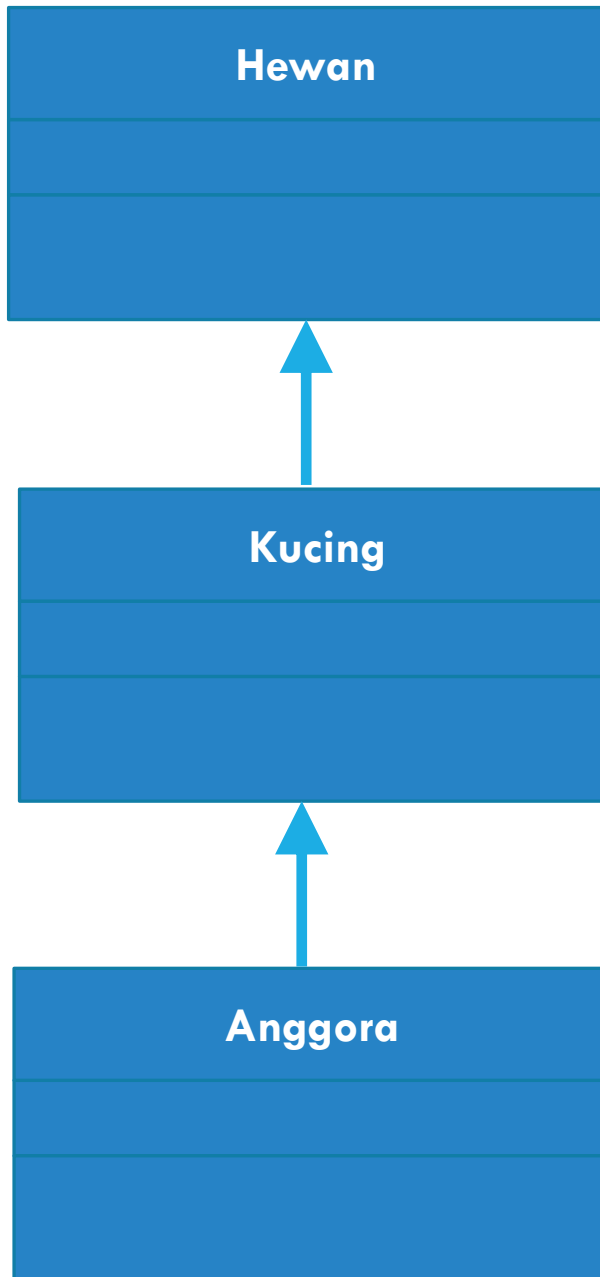
- Polymorphism: poly (banyak), morph (bentuk)
- Polimorfisme → konsep dalam OOP yang memperbolehkan sebuah aksi diimplementasikan secara berbeda
- Polimorfisme → konsep dalam OOP yang memperbolehkan sebuah objek untuk memiliki banyak bentuk



```
Pegawai pegawai1 = new Pegawai();
Dosen dosen1 = new Dosen();
TenagaKependidikan tendik1 = new TenagaKependidikan();
```

```
Pegawai dosen1 = new Dosen();
Pegawai tendik1 = new TenagaKependidikan();
```

dosen1 merupakan instance
dari class Dosen tapi dikenali
sebagai object bertipe
Pegawai



```
Anggora anggora1 = new Anggora();  
Kucing anggora2 = new Anggora();  
Hewan anggora3 = new Anggora();
```

CLASS *OBJECT*

Class `Object` merupakan root dari semua hirarki class. Artinya, semua object memiliki superclass dengan nama `Object`. Oleh karena itu, semua instance dapat dikenali sebagai `Object`

```
Object kucing1 = new Kucing();  
Object dosen1 = new Dosen();
```

HETEROGENOUS COLLECTION

- Dalam Java, setiap elemen dalam array harus memiliki tipe yang sama
- Konsep polimorfisme memungkinkan elemen dalam array bersifat heterogen

```
ArrayList<Pegawai> daftarPegawai = new ArrayList<Pegawai>();  
daftarPegawai.add(new Dosen());  
daftarPegawai.add(new TenagaKependidikan());  
daftarPegawai.add(new Dosen());
```

```
Ular ular1 = new Ular();  
Lebah lebah1 = new Lebah();  
Kucing kucing1 = new Kucing();  
Hewan[] daftarHewan = {ular1, lebah1, kucing1};
```

```
Object[] objects = {dosen1, ular1, tendik1, lebah1};
```


OBJECT CASTING

- Typecasting merupakan konversi variable dari suatu tipe data menjadi tipe data lainnya
- Typecasting juga dapat dilakukan terhadap object
- Terdapat 2 jenis object casting: **upcasting & downcasting**

UPCASTING

- Dilakukan untuk mengubah child object menjadi parent object
- Dapat dilakukan secara implisit

```
Pegawai dosen1 = new Dosen();  
Pegawai dosen2 = (Pegawai) new Dosen(); ← eksplisit  
  
TenagaKependidikan tendik1 = new TenagaKependidikan();  
Pegawai pegawai = tendik1;
```

- dosen1 merupakan instance dari class Dosen, tetapi setelah proses upcasting akan dikenali sebagai object bertipe Pegawai
- tendik1 merupakan instance dari class TenagaKependidikan, tetapi setelah upcasting akan dikenali sebagai object bertipe Pegawai

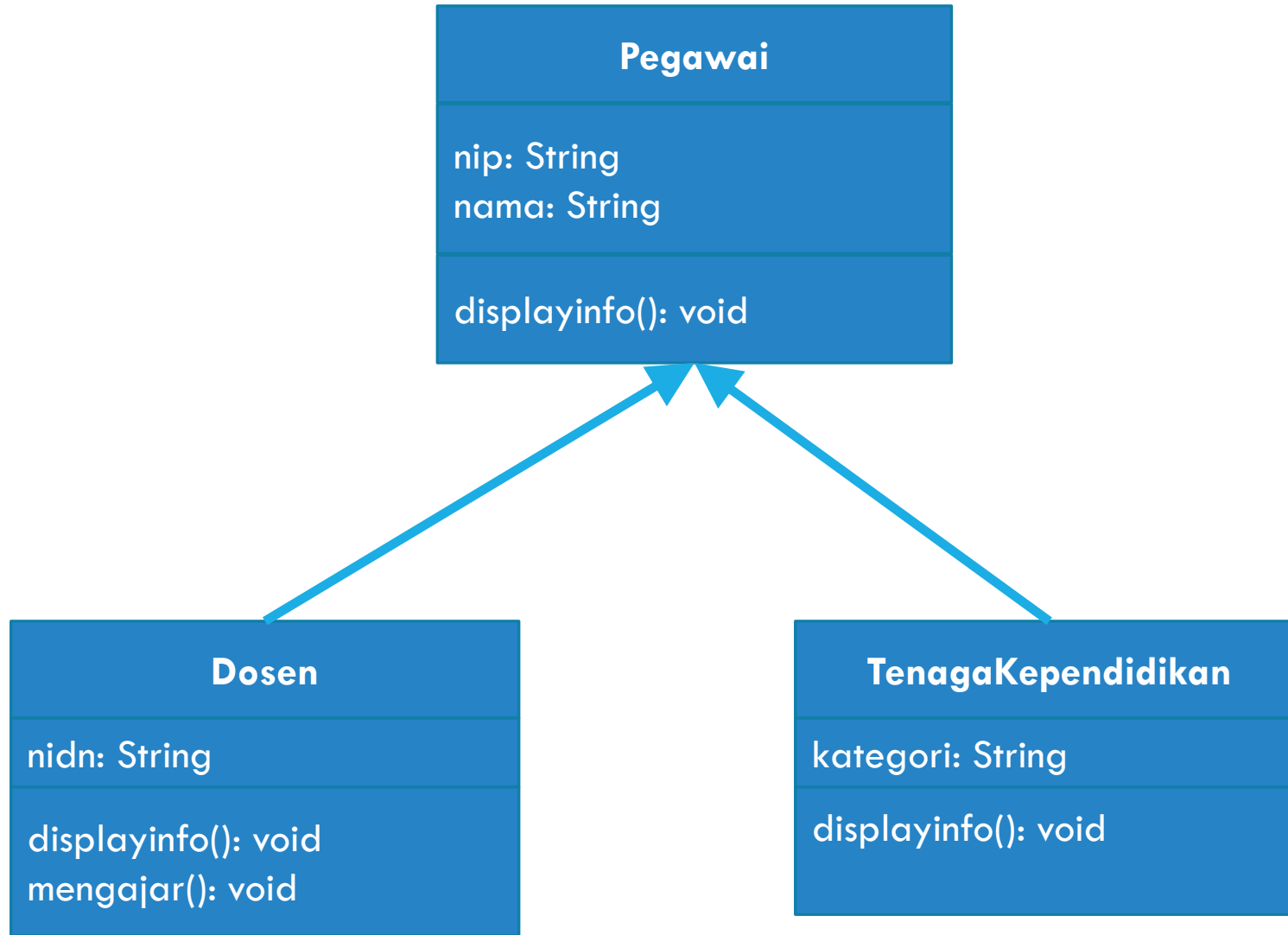
DOWNCASTING

- Dilakukan untuk mengubah parent object menjadi child object
- Tidak dapat dilakukan secara implisit

```
Hewan hewan1 = new Ular();  
Hewan hewan2 = new Lebah();
```

```
Ular ular1 = (Ular) hewan1;  
Lebah lebah1 = (Lebah) hewan2;
```

- Jika hewan2 didowncast menjadi object bertipe Ular maka akan muncul **Java.Lang.ClassCastException**



```
Dosen dosen1 = new Dosen("19940201", "Widia, S.Kom. M.Kom", "199402");
```

```
Pegawai pegawai1 = dosen1;
```

```
System.out.println(pegawai1.nip);
```

```
System.out.println(pegawai1.nama);
```

```
System.out.println(pegawai1.nidn);
```

```
pegawai1.mengajar();
```

- dosen1 merupakan instance dari class Dosen, tetapi setelah proses upcasting akan dikenali sebagai object bertipe Pegawai
- Oleh karna itu, dosen1 tidak akan dapat mengakses atribut yang dideklarasikan di class Dosen (misalnya nidn) maupun memanggil method yang hanya ada di class Dosen (misalnya mengajar())

```
Dosen dosen1 = new Dosen("19940201", "Widia, S.Kom. M.Kom", "199402");  
  
Pegawai pegawai1 = dosen1;  
  
System.out.println(pegawai1.nip);  
System.out.println(pegawai1.nama);  
pegawai1.displayInfo();
```

- Method displayInfo() dapat dikenali karena ada di class Pegawai, tapi karena terdapat overriding maka yang akan dieksekusi adalah method displayInfo() pada class Dosen

POLYMORPHIC ARGUMENTS

- Konsep polimorfisme juga memungkinkan **parameter** dari suatu method menerima **argument dengan berbagai bentuk object**
- Misalkan, terdapat method `train()` dengan parameter pegawai bertipe `Pegawai`. Method ini akan dapat menerima argument bertipe `Pegawai` atau class lain yang merupakan subclass dari `Pegawai`
- Proses ini juga termasuk **upcasting**

```
public static void train(Pegawai pegawai){  
    System.out.println("Memberikan pelatihan untuk pegawai");  
    pegawai.displayInfo();  
}
```

```
public static void main(String[] args) {  
    Dosen dosen1 = new Dosen("19940201", "Widia, S.Kom. M.Kom", "199402");  
    TenagaKependidikan tendik1 = new TenagaKependidikan("19750301", "Aida, A.Md.", "Tenaga Administrasi");  
  
    train(dosen1);  
    train(tendik1);  
}  
  
public static void train(Pegawai pegawai){  
    System.out.println("Memberikan pelatihan untuk pegawai");  
    pegawai.displayInfo();  
}
```

Method train() dapat dipanggil dengan argument dosen1 bertipe **Dosen** maupun tendik1 bertipe **TenagaKependidikan** karena dilakukan **upcasting** ke class **Pegawai**

INSTANCEOF

- Keyword `instanceOf` digunakan untuk mengidentifikasi apakah sebuah object merupakan instance dari suatu class/interface

```
Hewan hewan1 = new Ular();  
Hewan hewan2 = new Lebah();
```

```
System.out.println(hewan1 instanceof Ular);    //true  
System.out.println(hewan2 instanceof Ular);    //false
```

```
public static void train(Pegawai pegawai){  
    pegawai.displayInfo();  
    System.out.println("Mengenalkan lingkungan kampus");  
    System.out.println("Menginfokan SOP/Juknis");  
  
    if (pegawai instanceof Dosen) {  
        System.out.println("Memberikan pelatihan pedagogik");  
    }  
}
```

Constructor

```
public class Donat{  
    public String topping;  
}
```

Versi file yang ditulis
oleh developer

Jika sebuah class tidak memiliki
constructor secara eksplisit,
maka secara default Java
compiler akan menyediakan
constructor tanpa parameter

```
public class Donat{  
    public String topping;  
  
    public Donat(){  
    }  
}
```

Versi Java compiler

Instansiasi Objek

```
public class DemoDonat{
```

```
    public static void main(String[] args){
```

```
        Donat donat1 = new Donat();
```

```
        donat1.topping = "Strawberry sprinkles";
```

```
    }
```

```
}
```



Constructor berparameter

- Constructor berparameter digunakan untuk menginstansiasi objek baru dengan kondisi/nilai tertentu.
- Java compiler tidak akan menyediakan default constructor (tanpa parameter) jika constructor lain sudah dibuat oleh developer

```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```

Instansiasi Objek

```
public class DemoDonat{  
    public static void main(String[] args){  
        Donat donat1 = new Donat("Strawberry sprinkles");  
    }  
}
```



```
public class Donat{  
    public String topping;  
}
```



```
public class Donat{  
    public String topping;  
  
    public Donat(){  
    }  
}
```

Jika sebuah class tidak memiliki constructor secara eksplisit, maka secara default Java compiler akan menyediakan constructor tanpa parameter

```
public class DemoDonat{  
    public static void main(String[] args){  
        Donat donat1 = new Donat();  
        donat1.topping = "Choco";  
    }  
}
```

```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```

Versi file yang ditulis
oleh developer

Java compiler tidak akan menyediakan
default constructor (tanpa parameter)
jika constructor lain sudah dibuat oleh
developer

```
public class Donat{  
    public String topping;  
  
    Donat(String selectedTopping){  
        topping = selectedTopping;  
    }  
}
```

```
public class DemoDonat{  
    public static void main(String[] args){  
        Donat donat1 = new Donat("Choco");  
        Donat donat2 = new Donat();    //error  
    }  
}
```

Versi Java compiler